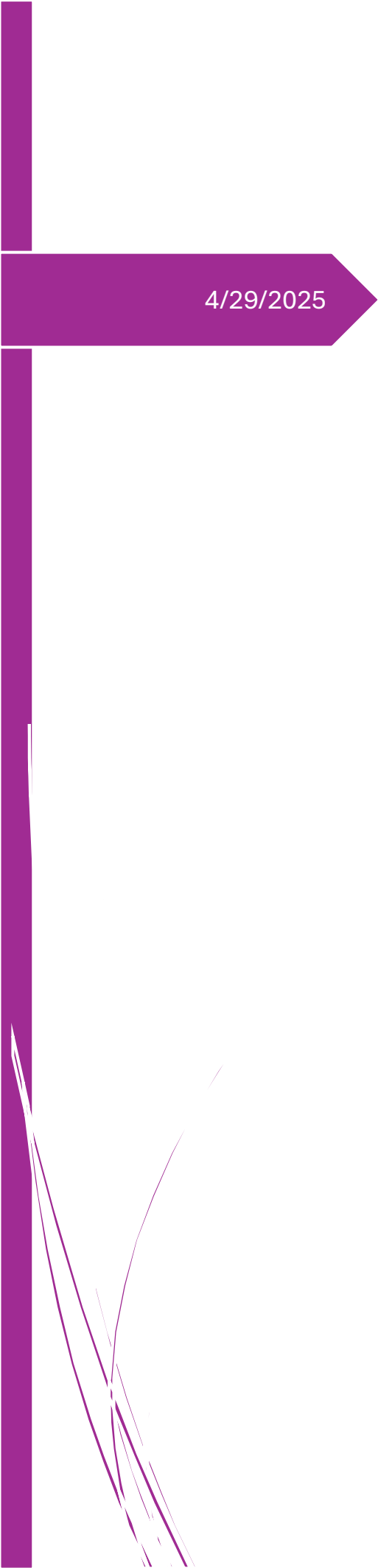


A thick purple vertical bar runs down the left side of the page. A purple arrow points to the right from the bar, containing the date.

4/29/2025

Nature in Motion

A predator-prey simulation

ATIKSH AGNIHOTRI

BOURNE GRAMMAR SCHOOL

CANDIDATE NUMBER: 6002

CENTRE NUMBER: 26202

QUALIFICATION CODE: 7517/C

Several thin, curved purple lines sweep upwards from the bottom left corner of the page.

Abstract

Rationale

Biology students are required to learn animal population dynamics that take place over prolonged timeframes, on an enormous scale. They can learn about these systems theoretically, but lack the means to fully comprehend these complex mechanisms. There are limited simulation tools available to help biology teachers bring these concepts to life for their students.

The purpose of this project is to create a tool (customisable simulation/model) for biology teachers to help them teach the dynamics of the predator-prey interaction by using a visual teaching aid to support learning.

Methodology

To develop the simulation, existing models were evaluated along with capturing user requirements through a questionnaire and end-user interviews. Prototypes were used to inform the development of the final solution. The design of the project provides an overview of how the different parts of the solution will interact. This includes algorithms, flowcharts, graphs and data tables.

Solution

A simulation was developed in Python, using digital creatures to recreate a natural environment and provide a graphical representation of the interaction between predators and prey. The simulation allows users to play around with different variables to model various scenarios that occur in the natural world.

This system uses the Lotka-Volterra differential equations to model the observed interaction between predators and prey in the simulation. A curve is produced, fitting the simulation data to the model's prediction for the population numbers of predators and prey.

End users (Teachers/tutors) can interact with the system by creating an account, where they can store previously generated figures and download the graphs generated, creating a physical repository for their students.

Key Words: Predator-Prey, Simulations, Algorithms

Table of Contents

ABSTRACT	I
TABLE OF CONTENTS	II
LIST OF FIGURES	IV
LIST OF TABLES.....	V
CHAPTER 1. NEA ANALYSIS.....	1
1.1. OVERVIEW OF THE PROBLEM	1
1.1.1. <i>Existing issues and challenges</i>	1
1.1.2. <i>End Users</i>	1
1.2. PROPOSED SOLUTION.....	2
1.3. RESEARCH METHODOLOGY	2
1.3.1. <i>Year 12 Biology Students Questionnaire</i>	3
1.3.2. <i>Questionnaire Updates:</i>	4
1.3.3. <i>Survey Results:</i>	4
1.3.4. <i>Key takeaways from the survey</i>	7
1.3.5. <i>Biology Teachers' Interviews</i>	8
1.3.6. <i>Key imperatives from the end-user interviews</i>	9
1.4. EXISTING SOLUTIONS.....	9
1.5. MODELLING	13
1.5.1. <i>Lotka-Volterra Model</i>	13
1.5.2. <i>IPSO Diagram</i>	14
1.5.3. <i>Entity Relationship Model</i>	15
1.5.4. <i>FSM Models</i>	15
1.5.5. <i>Prototyping</i>	16
1.6. POTENTIAL SOLUTIONS.....	17
1.7. OBJECTIVES OF THE SIMULATION PROGRAM.....	17
CHAPTER 2. NEA DESIGN.....	22
2.1. DESCRIPTION.....	22
2.2. FILE STRUCTURE	23
2.3. STRUCTURE/HIERARCHY CHART	24
2.4. ALGORITHMS AND FLOWCHART	25
2.4.1. <i>Random Walk</i>	25
2.4.2. <i>Biased Random Walk</i>	25
2.4.3. <i>Greedy Algorithm</i>	25
2.4.4. <i>Spiral Search</i>	25
2.4.5. <i>Levy Flights</i>	26
2.4.6. <i>Breadth-First Searching</i>	26
2.4.7. <i>Depth-First Searching</i>	26
2.5. ENTITY RELATIONSHIP DIAGRAMS.....	31
2.6. CLASS DIAGRAMS	32
2.7. DATA TABLES	34
2.8. HUMAN-COMPUTER INTERACTION	36
CHAPTER 3. NEA TECHNICAL SOLUTION	47
3.1. CODE LISTING	47

3.1.1.	<i>Main.py</i>	47
3.1.2.	<i>simulationManager.py</i>	63
3.1.3.	<i>tkManager.py</i>	82
3.1.4.	<i>dbManager.py</i>	106
3.1.5.	<i>EmailManager.py</i>	116
3.1.6.	<i>graphManager.py</i>	119
3.2.	LIBRARIES.....	124
3.3.	CODING STYLE.....	125
3.4.	TECHNICAL SKILL TABLES.....	127
CHAPTER 4. NEA TESTING:		131
4.1.	TEST TABLE.....	131
CHAPTER 5. EVALUATION		152
5.1.	KEY TAKEAWAYS.....	152
5.1.1.	<i>Quick Wins</i>	152
5.1.2.	<i>Objectives Accomplished</i>	152
5.1.3.	<i>Detailed User Feedback</i>	153
5.1.4.	<i>Limitations</i>	158
5.1.5.	<i>Further Improvements for the future</i>	158
CONCLUSION		159
BIBLIOGRAPHY		160

List of Figures

FIGURE 1: QUESTION 1 (SURVEY).....	4
FIGURE 2: QUESTION 2 (SURVEY).....	5
FIGURE 3: QUESTION 3 (SURVEY).....	5
FIGURE 4: QUESTION 4 (SURVEY).....	6
FIGURE 5: QUESTION 5 (SURVEY).....	6
FIGURE 6: QUESTION 6 (SURVEY).....	7
FIGURE 7: MINUTELABS EVOLUTION SIMULATOR.....	10
FIGURE 8: ECHALK PEPPERED MOTH GAME.....	11
FIGURE 9: THE LIFE ENGINE	12
FIGURE 10: PREDATOR-PREY CURVES.....	14
FIGURE 11: ENTITY RELATIONSHIP DIAGRAM.....	15
FIGURE 12: SIMPLE CREATURE FSM DIAGRAM	15
FIGURE 13: WOLF/DEER FSM DIAGRAM	16
FIGURE 14: PROJECT CODE FOLDER	23
FIGURE 15: RANDOM WALK ALGORITHM	27
FIGURE 16: RANDOMISED GREEDY ALGORITHM	27
FIGURE 17: RANDOMISED GREEDY ALGORITHM FLOWCHART	28
FIGURE 18: RANDOMISED GREEDY ALGORITHM WITH BIASED RANDOM WALK.....	29
FIGURE 19: RANDOMISED GREEDY ALGORITHM WITH BIASED RANDOM WALK FLOWCHART	30
FIGURE 20: USER DETAILS ERD	31
FIGURE 21: USER GRAPHS ERD	31
FIGURE 22: SIMULATION CLASS DIAGRAMS	32
FIGURE 23: CLIENT SERVER CLASS DIAGRAMS.....	33
FIGURE 24: GRAPHMANAGER CLASS DIAGRAM	34
FIGURE 25: WELCOME PAGE SCREEN	36
FIGURE 26: ACCOUNT CREATION SCREEN	37
FIGURE 27: EMAIL SCREEN	38
FIGURE 28: OTP SCREEN	38
FIGURE 29: ACCOUNT DETAILS SCREEN	39
FIGURE 30: LOGIN PAGE SCREEN	40
FIGURE 31: EXPLANATION PAGE.....	41
FIGURE 32: GRAPH DOWNLOAD SCREEN	41
FIGURE 33: CREATURE SELECTION SCREEN.....	42
FIGURE 34: SETTINGS SCREEN	43
FIGURE 35: SIMULATION SCREEN	43
FIGURE 36: PAUSED SCREEN.....	44
FIGURE 37: FINISHED SCREEN	44
FIGURE 38: GRAPH SAVED MESSAGE SCREEN.....	45
FIGURE 39: SAMPLE GRAPH 1.....	45
FIGURE 40: SAMPLE GRAPH 2.....	46

List of Tables

TABLE 1: IPSO DIAGRAM.....	14
TABLE 2: TEST PLAN TABLE.....	21
TABLE 3: USER ACCOUNT.....	34
TABLE 4: USER CONTACT	35
TABLE 5: USER DETAILS.....	35
TABLE 6: GRAPH DETAILS	35
TABLE 7: USER GRAPHS	35
TABLE 8: EXTERNAL LIBRARIES TABLE.....	124
TABLE 9: TECHNICAL SKILLS - MODEL	128
TABLE 10: TECHNICAL SKILLS - ALGORITHMS.....	130
TABLE 11: TEST TABLE WITH EVIDENCE AND RAG RATING	151

Chapter 1. NEA Analysis

1.1. Overview of the problem

This project was selected due to the lack of customisable teaching software to help teachers explore the population dynamics between predators and prey with students. Currently, students learn about the interaction between predators and prey by briefly discussing the theory and then drawing a graph. This does not engage students and fails to capture the complexity of the topic. Without observing these interactions occurring in real-time, students are expected to understand the model and how it is produced. The teaching resources are insufficient in providing students with a real comprehension of predator-prey dynamics.

1.1.1. Existing issues and challenges

Our school does not have their own software to teach these concepts. Using externally available software involves high subscription costs and expensive equipment. The outputs of relevant online simulations are available in the form of static images, rather than showing the emerging process. These systems are also inadequate in helping students understand, as they don't show the actual organisms interacting. These simulations usually take the parameters for the Lotka-Volterra to produce the simulation, which isn't appropriate, as this is using the model to make artificial representations of a real scenario. This program uses the interaction of creatures to produce parameters and a graph, which is more appropriate to the often-erratic nature of predator and prey populations in an area. Many biological simulations are focused on other areas, such as evolution and microscopic cell interactions. There is a lack of simulations which focus on modelling predator-prey populations. Of the few which exist, these do not provide an easy way to access data, which means students are left without a way to access any useful trends or data which emerged.

1.1.2. End Users

The end users for this project are biology teachers at my school, specifically Mr P, who requested this area to be explored. Other potential end users could be biology students, who will be able to access the simulation and make accounts as well, due to their student emails. Teachers outside of the school would also be interested in this software, as it is made to be secure, and databases are device-specific, which means that any user data stored is only present on the device where the simulation is run.

This program has been developed specifically as a teaching tool to help students visualise the mathematics which makes the predator-prey curves that students are taught about. This project has been built with teachers (end users) in mind, who will be supported through features such as providing graphs to the user as PNGS, which teachers can print and use as a resource in their lessons. This ensures the program

has usage beyond just running the simulation, with the added feature of creating a repository.

1.2. Proposed Solution

This project explores predator-prey dynamics, which have been modelled using digital creatures on a grid and producing graphs using the Lotka-Volterra differential equations. Predator-prey graphs show the relationship between the predators and prey in an environment, demonstrating a relationship between predator and prey populations over time, specifically due to their interaction. This interaction is explained below:

When prey becomes scarce, predators will die out, which leads to a boom in prey population, as they are not being predated upon, which in turn causes a predator boom, due to the abundance of prey.

This cyclical phenomenon will be replicated by using interactions between digital animals to visualise this phenomenon in real life. The program will simulate a user-specified population of digital animals on a grid with randomly placed food. Animals are programmed to move towards food if it is within their sense range, much like how real animals would be limited by their senses. Each movement costs a certain amount of energy, depending on the creature's type. Different search algorithms have been implemented to mimic different creatures, such as wolves and deer, so that it is even more accurate to real-life dynamics. Animals will regain energy if they land on a food object, and those which reach a certain energy threshold will reproduce, producing clones of themselves at a cost to their energy. Animals that run out of energy will die and be removed from the grid. The simulation will produce a graph of the populations of predators and prey, fitting the values to a system of Lotka-Volterra curves. Once the simulation ends, users should be able to store graphs produced at the end of the simulation, both saving to their downloads folder and a local database. To access this database, a user will have to make an account, where they will sign up with a username, password, name and email address. Users will be verified using their input email, as this confirms that users are real. Once the user has an account, they should be able to download a zip file of the graphs they have generated and saved.

1.3. Research Methodology

The methodology followed for this project involved undertaking primary data collection. This included administering a survey (using a short questionnaire) to current students to understand their experience, what they may have required from the program being developed, and to get a brief idea of the topics students found difficult. To narrow down the specific biological simulation to make, interviews with biology teachers were undertaken to understand the pedagogical challenges faced

when teaching. This provides an understanding of the tools and resources currently employed in teaching.

Initially, before the project details were finalised, I wanted to understand what type of biological visualisations students wanted to use. This questionnaire was conducted to pilot the kind of simulations to pursue. The pilot study data was useful to understand the system requirements.

1.3.1. Year 12 Biology Students Questionnaire

A Microsoft Forms questionnaire was sent to understand what Year 12 students would like to see in a simulation software application for biology. 24 responses were received, giving me a response rate of 50%. Opportunity sampling was used as there was a limited data set, and random sampling did not present as much of an advantage in this case, as fewer samples would have led to each response having considerably more weight. This could lead to a very biased set of respondents who may not be representative of the wider opinion. This meant I was able to obtain as many responses as possible in an attempt to accurately represent the views of biology students.

Visualisation: The representation of an object, situation, or set of information as an image, graph, or simulation.

1. What biological concept is most misunderstood by the public? (Options taken from Biology specification)
2. Do you find visualisations helpful for your learning? (Strongly Disagree, Neutral...)
3. Do you currently use any visualisation software to understand a topic? (Frequency)
4. What type of learner are you? (Visual, Auditory, Kinaesthetic...)
5. Do you use technology to supplement your revision? (Frequency)
6. What are some concepts you struggle with visualising in biology? (Open word, cloud)
7. Are there any biological concepts outside of your specification you are interested in learning about? (Open, word cloud)
8. Are there any features you would like to see included in my software? (Open, word cloud)
9. Would you find it helpful to see graphs while a simulation is underway e.g. Predator-Prey Cycle? (Yes/No)
10. On a scale of 1-10, how clear was the survey provided? (Likert Scale)
11. Are there any comments you wish to add? (Open)

1.3.2. Questionnaire Updates:

These were my initial questions; however, they were updated following advice received from a statistics teacher I consulted.

List of changes:

- Format of the questions – Some questions were more appropriate to ask in the format “To what extent do you agree with...” as this ensured no leading questions.
- Removal of a “neutral” option – The neutral option was removed from the opinion questions, forcing participants to pick a side. This was useful as neutral responses would not have been useful to collect and would have diluted the opinions expressed.
- Removal of Q11 in the student questionnaire – VAK learning styles are debunked and therefore would not have added value to the questionnaire.

1.3.3. Survey Results:

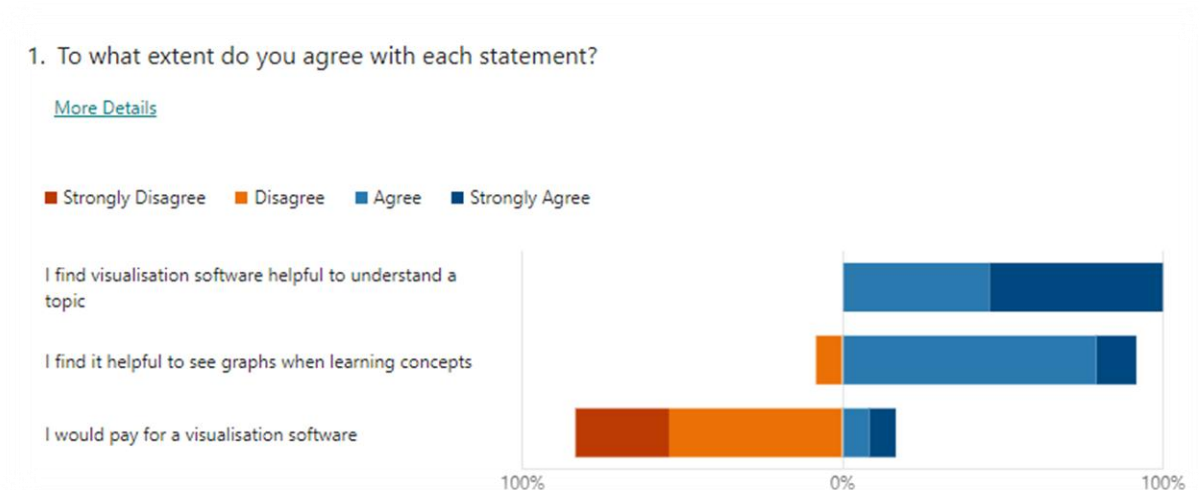


Figure 1: Question 1 (Survey)

These questions provided me with information on the opinions of students on visualisation software. Every participant responded with “Agree” or “Strongly Agree” when asked if visualisation software helped their understanding of topics. This shows that the end users viewed visualisation as a useful learning method. Moreover, an overwhelming majority of participants responded positively to graphs, a feature being considered for implementation. Despite the positive opinion regarding the use of visualisation software, many people responded with “Strongly Disagree” or “Disagree” when asked if they would pay for such software. This illustrates the need for free visualisation software which uses graphs and simulations to support students’ learning.

2. How frequently do you use visualisations to supplement your learning?

[More Details](#)

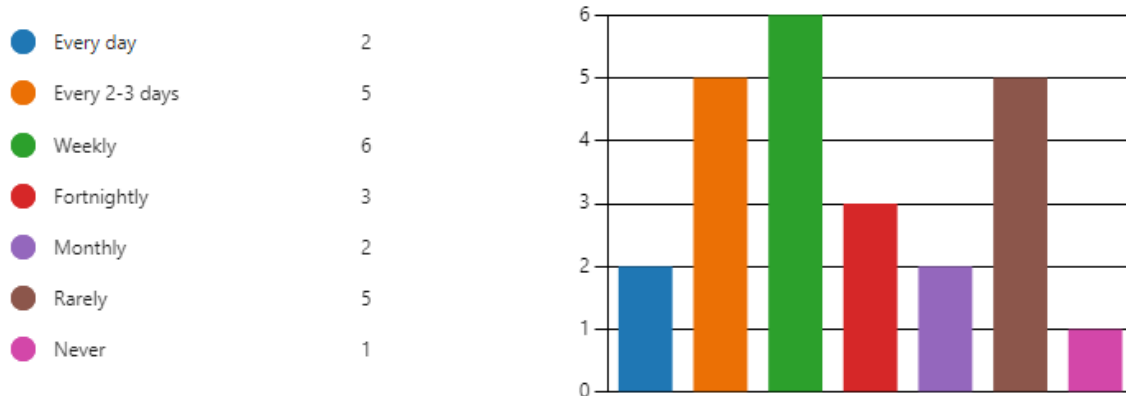


Figure 2: Question 2 (Survey)

3. How often do you use revision apps/software as part of your revision?

[More Details](#)

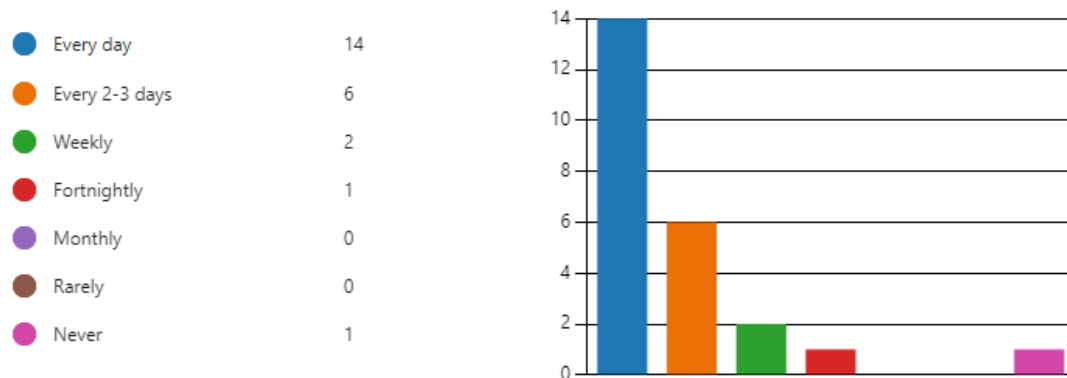


Figure 3: Question 3 (Survey)

These two questions were posed to see the difference in the use of other non-visual software and visualisation software to understand how often people used simulations and the value added to students' learning. While traditional revision apps, like flashcards and detailed note sites, were used by most students every day, visualisations such as animated YouTube videos and simulations were used less often on average. Results were also more varied for visualisations, showing that some students may benefit more from others.

4. Are there any features you really enjoy in the software/websites you use currently?

[More Details](#)

[Insights](#)

15

Responses

Latest Responses

"Flashcards"

"Flashcard"

"Easy to use"

4 respondents (27%) answered **Flashcards** for this question.

Word cloud showing features mentioned by respondents:

- Flashcards
- Spaced
- Exam questions
- Seneca style
- style features
- variety of questions
- factual recall
- large variety
- repetition on quizlet
- harder maths
- repetition flashcards
- maths problems
- range of things
- things available
- questions by topic
- working
- advanced calculator
- Easy to use
- pictures/animations

Figure 4: Question 4 (Survey)

This question did not provide me with very useful data, given that students may not have understood my intentions for the project. However, there was mention of an “advanced calculator”, “maths problems” and “harder maths”, which was noted as relevant. It seems that students found the mathematical elements of biology tough to understand.

5. Please rank these biology topics from the specification in order of which you would find most useful to visualise. (Drag topics to rank them, most useful at the top)

[More Details](#)



Figure 5: Question 5 (Survey)

This question aimed to narrow down which concepts students would like to see using simulation and visualisations. Cells and Exchange, and Transport emerged as the two most popular topics. Subjects like Classification and Genetics had not been taught in full yet, which explains their lower score.

6. What are some concepts outside of the topics mentioned in the above question that you are interested in visualising?

[More Details](#)

 Insights

24

Responses

Latest Responses

"None"

"No"

"Cells"

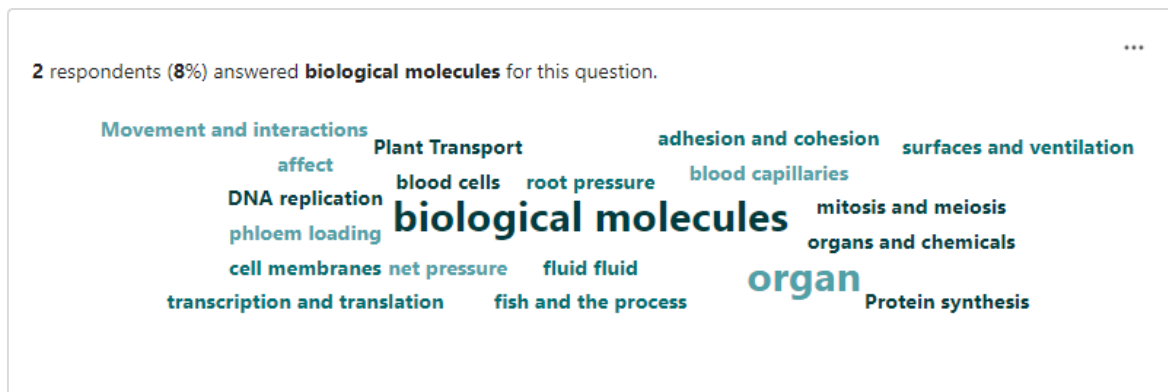


Figure 6: Question 6 (Survey)

This question was very useful in understanding the different concepts that people wanted to see in detail. Biological molecules emerged as a very popular concept. Most topics were related to cellular processes, which reflected the topics that students were studying at the time.

1.3.4. Key takeaways from the survey

This survey was useful to understand what students may find difficult to comprehend, but, upon reflection, it was ascertained that the Year 12 Biology students may not be the best resource, as they had not covered the entire course yet. This also limited my data collection to students who had chosen to take biology, and thus students who may not require visualisation tools. This was reflected by the responses, which mentioned features such as flashcards, and it seems these types of software may be better suited to them.

1. Students found visualisation software, specifically with graphs, helpful
2. Students wanted software to help understand the mathematics behind biology
3. There is a demand for free visualisation tools
4. To fully understand what students need, speaking to experts in more detail was required

Given that my survey was only partially successful at ascertaining the needs for a biological simulation software, I turned to interviewing the biology department for a more focussed idea.

1.3.5. Biology Teachers' Interviews

Interviews were conducted with the biology department, which involved asking the biology teachers a list of different questions. This enabled qualitative data collection, which was highly useful in informing the end user requirements. The teachers were asked the same questions, and the most interesting responses are included verbatim.

What are some biological concepts which are misunderstood by the public?

Mr P: I think there are a few topics which are hard for the public to conceptualise. The main reason is usually the scale over which these concepts occur. Disease is sometimes a topic which people can struggle with, as understanding how it spreads and how our immune system fights back can be difficult. It is complex, but I feel like there should be ways of teaching it better.

Who do you think would benefit the most from using visualisation?

Dr H: I think everyone could benefit from being able to see the concepts they are studying. We try to do this for all students with practicals like looking at cells through a microscope for Year 7s, all the way up to organ dissections for the older students. It's just an essential part of learning for everyone.

What software do we use in this school to visualise biological concepts?

Mr S: Well, we do use a software called eChalk. However, I've honestly given up on using simulations. Most of them are just slideshows, and I can't really use anything from the actual simulations, as they can be hyper-specific to historical scenarios we don't cover, or they just are hard to interact with and run. It's very frustrating.

Do you think other subjects are better or worse when it comes to simulations?

Mr S: Oh, I would definitely say so. Physics has a huge number of simulations to choose from for all types of concepts. I'm jealous, to be honest. Out of the sciences, I would say biology is pretty neglected when it comes to simulations.

What are some concepts outside of the course which you would be interested in understanding using visualisation?

Miss M: I remember using a software called PyMOL whilst doing a biological computation module, which opened my eyes to how proteins were structured. However, that might be a bit advanced for a student to program, and I wouldn't really have use for it outside of my own interests.

What would be the one concept you would most like to have a visualisation tool to explain?

Mr P: Good question. I recently had a class where I discussed the dynamics of predators and prey in the wild. I remember a student asking about how this actually worked, as they couldn't really understand why populations would vary so much. When I tried to explain it using the actual models which we use in Biology, Lotka-Volterra, they got even more confused because of the mathematical nature of the relationship between predators and prey. I wish I could have shown them an actual biological system where they could see the iconic predator-prey curves actually emerge. I think that would be pretty useful.

What sorts of features do you want to see in a simulation software?

Dr H: I think the most important thing is a way to get actually useful data out of the simulation. It is alright to use something as a demonstration, but without an actual output that we can use to teach students, videos or slideshows will be just as useful as any simulation.

1.3.6. Key imperatives from the end-user interviews

It became clear from these responses that teachers also felt there was a need for a visualisation software to help them teach effectively. There was a requirement for tools which not only demonstrated a concept but also generated relevant data in real-time. The other requirement was to have a user-friendly application. However, some of the suggestions regarding the application were too complex to implement in a limited time frame.

Particularly inspired by Mr P's response, the predator-prey dynamics were researched, specifically the Lotka-Volterra equations mentioned (see section 1.5.1). This was the key idea that the application is based upon.

1.4. Existing Solutions

Figure 7 below shows an interactive science app which simulates evolution in a randomly generated world of small blob creatures. These creatures have traits such as size, speed, and sense range, determined by a Gaussian distribution around a user-set mean value. This produces a realistic population of the species. The creatures are set in random motion from a point on a 2D plane with food placed at randomly generated locations. These creatures are drawn towards food within their sense range. If they eat one piece of food, they survive till the next generation. If they eat more than one piece of food in a single generation, a clone of the creature is produced in the next generation, mimicking asexual reproduction.

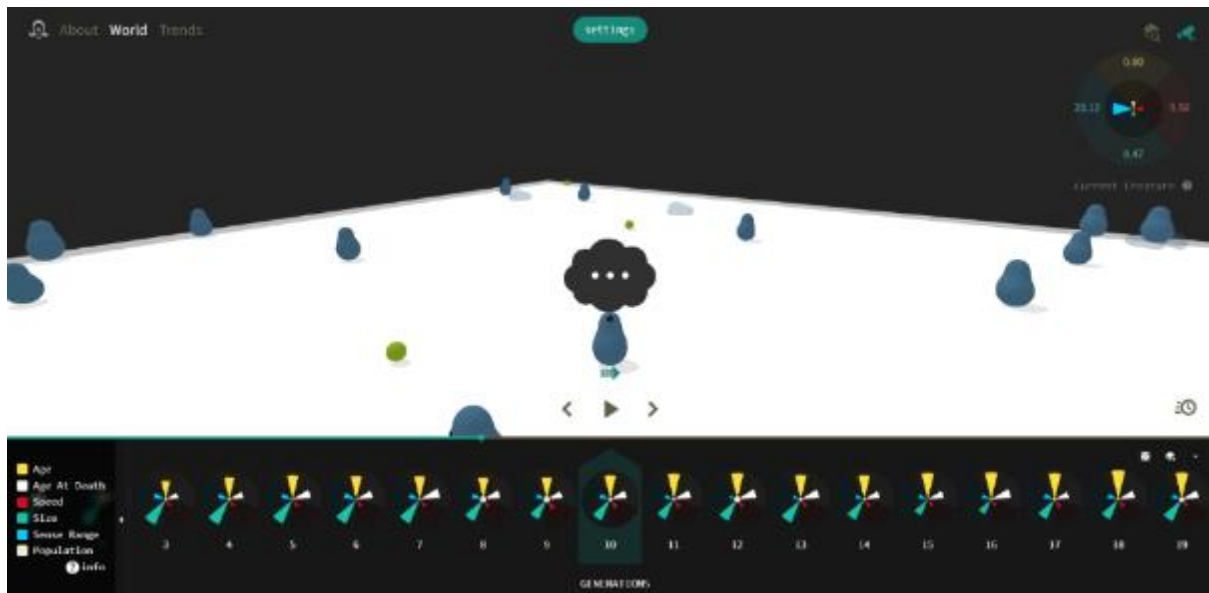


Figure 7: MinuteLabs Evolution Simulator

Users can set the number of generations and food per generation. I especially like the control that the simulation provides over the species themselves, as users can set the number of creatures per species, how varied a population is, and how much starting energy an organism has. This flexible approach helps users model evolution in a much more engaging way, as users can create highly specific scenarios. This simulation was very user-friendly and had the option to download the simulation data.

However, it didn't have any way to save graphs in the system itself, and CSV data is not immediately useful without more advanced data presentation techniques. It also unfortunately only modelled prey organisms and was more focused on showing the process of natural selection.

The feature of having a specific initial energy was implemented, as well as the sense range and speed.



Figure 8: eChalk peppered moth game

Another existing platform is eChalk, referenced earlier by Mr S, which is full of animations and simulations for a variety of different topics. Figure 8 shows an example of an eChalk activity, where users play a game to try and click on the famous peppered moths, which were one of the first species to be noticed as evolving. Users click on the moths using a bird cursor, with the proportion of the light and dark moths they click being updated dynamically as the program runs and as users click on moths. Users can also swap between two environments, one where the light moths blend in with the environment and one where darker moths blend in.

The interactive elements of this activity were good, but it wasn't very informative and worked more like a game. However, a good user interface and an account system made the system feel much more professional. As a result, an email-based login system and account system were incorporated in the solution.

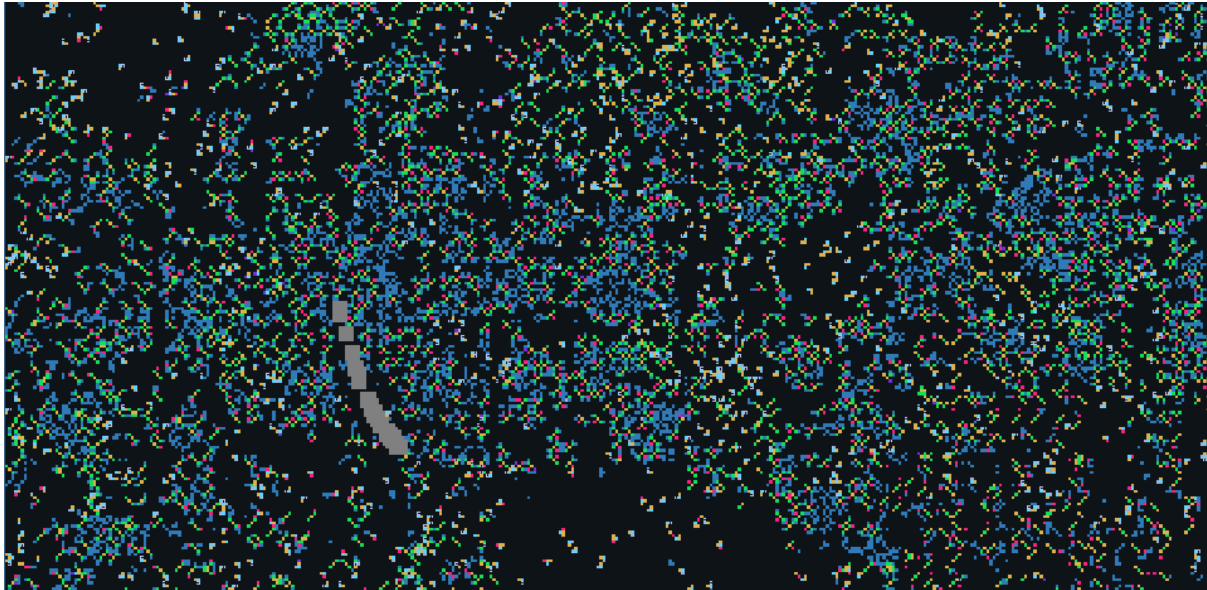


Figure 9: The Life Engine

Figure 9 above shows a large simulation of organisms composed of different components, designated by different colours. This is an example of a very complex simulation program which uses simple designs and biological components to create vast ecosystems of different organisms.

The simulation includes an editor to create a user's organisms to put into the program, which makes them interact with the environment as well. Additionally, wall and food tiles can be placed, and users can kill the existing organisms using a brush tool. The Life Engine is a project designed to observe simple multicellular organisms interact, modelling their success/failure through population-time graphs. Some features in this simulation were very effective, such as the ability to set the amount of food available to creatures, the ability to set a simulation speed and especially the pause feature, to be able to observe organisms when they are not moving.

However, some issues found were that the population graph continued to plot even when the simulation was paused. Understanding the controls of the simulation was very difficult as well.

The graph data also only represented the overall organism data and not the relationship between prey and predator organisms. It was decided to implement a pause feature and ensure that graphs do not continue plotting when the simulation is paused. The user will define the speed of the simulation and the food available at the start, like in this program.

1.5. Modelling

1.5.1. Lotka-Volterra Model

Building on Mr P's prompt, this phenomenon of predator-prey systems was further researched to ascertain if it was possible to build this system. Predator-prey systems principally rely on mathematics to model the relationship, with many different equations being used to model them, such as the Rosenzweig–MacArthur model and Arditi–Ginzburg equations. Such systems use differential equations, equations which describe quantities over time, to model aspects of the predators and prey. The most famous of these is the Lotka-Volterra equations, which are a pair of equations that model the change in predator and prey populations over time according to several parameters.

$$\begin{aligned}\frac{dx}{dt} &= \alpha x - \beta xy, \\ \frac{dy}{dt} &= \delta xy - \gamma y,\end{aligned}$$

Equation 1: Lotka-Volterra Differential Equations

Parameters,

dx/dt: This refers to the change in prey population over time. It represents the growth rate of the prey at the time

dy/dt: Similarly, this refers to the change in predator population over time. It represents the growth rate of predators at the time.

x: This is the current population level of the prey

y: This is the current population level of the predators

α: This is a constant parameter, which determines the growth rate of the prey

β: This is a constant parameter, describing the effect of the predators on the prey's death rate

γ: This is a constant parameter, which determines the death rate of the predators

δ: This is a constant parameter, which determines the effect of the prey on the predator's growth rate.

This model is one of the simplest that exists, but it is known to have good accuracy at predicting the general trend of the majority of actual ecosystems. It is also the model

which produces the predator-prey curves that students are expected to learn about when they first learn about predator-prey systems.

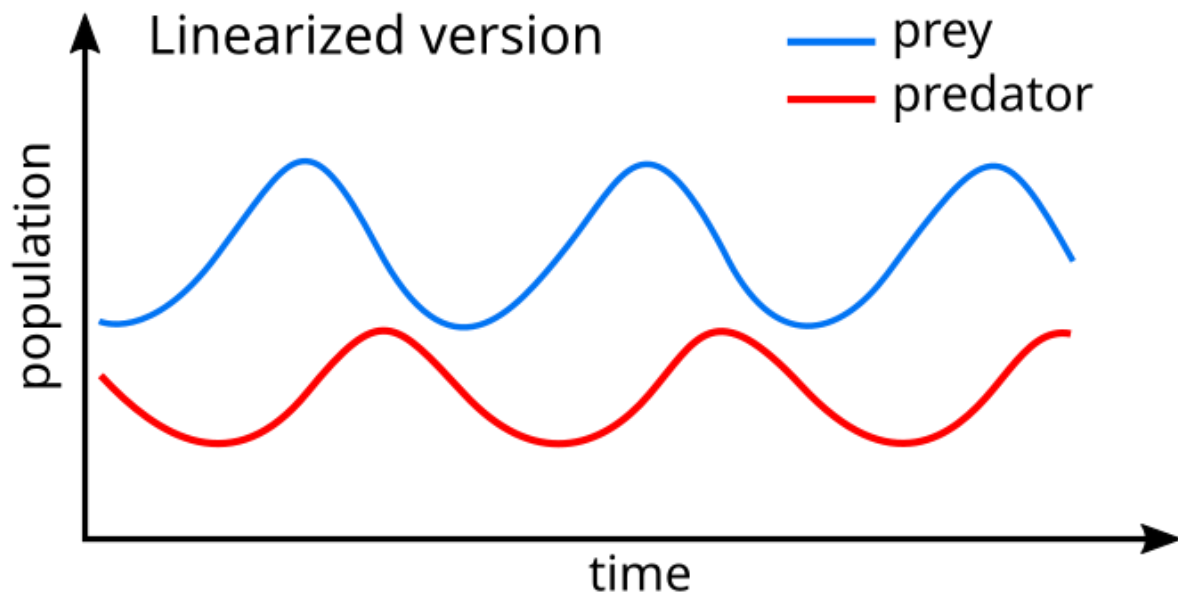


Figure 10: Predator-Prey Curves

A resource used to learn about these was the University of Queensland's Lotka-Volterra application: [Accessible by clicking here](#)

1.5.2. IPSO Diagram

IPSO	Information
Input	Username, Password, First Name, Last Name, Email
Input	Species choices, Simulation Speed, Starting Food number
Input	Simple prey number, Simple Predator number, Wolf number, Deer number
Input	OTP
Process	Use population numbers to calculate Lotka-Volterra values and plot graph
Process	Hash Password
Process	Calculate creature energy and determine if able to reproduce or if it is dead
Process	Generate a true random OTP by querying the ANU's quantum numbers API
Process	Create Local Database
Process	Check for creatures in sight/collisions using NumPy arrays
Store	Username, Hashed Password, First Name, Last Name, Email
Store	Graphs
Output	Graphs associated with the user

Table 1: IPSO Diagram

This IPSO Diagram was created to model the system's function, through the lens of inputs, processes and outputs. This helped inform the structure of the solution.

1.5.3. Entity Relationship Model

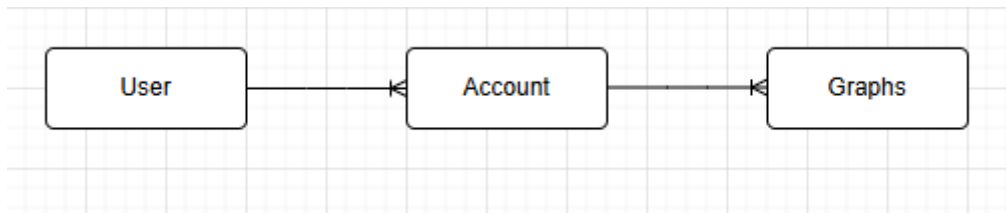


Figure 11: Entity Relationship Diagram

This model describes the basic structure of the database, where users are allowed to have multiple accounts, but each account is only linked to one user. Similarly, each account can have multiple graphs saved to it, but each graph can only be linked to one account. This also links the users and graphs, as users can have multiple graphs given that they are linked to graphs through the account interface.

1.5.4. FSM Models

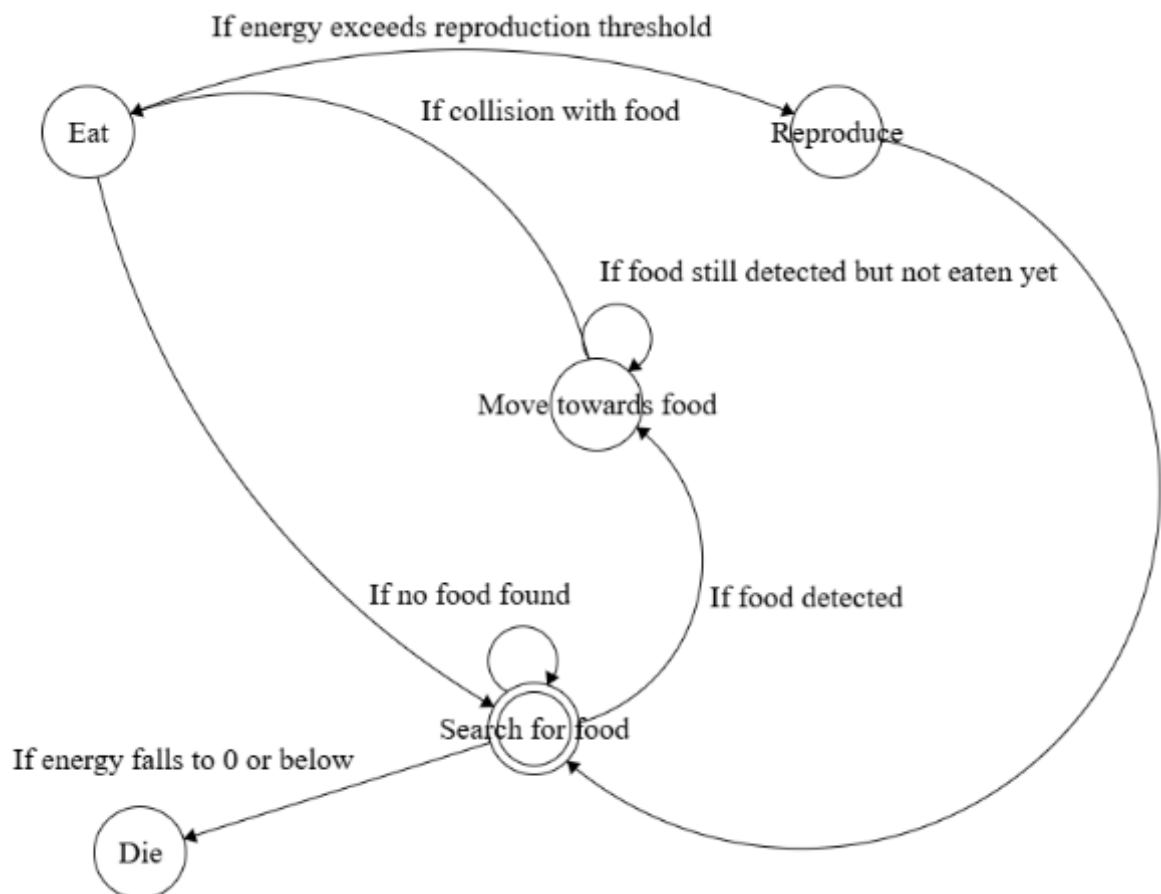


Figure 12: Simple Creature FSM Diagram

The FSM model above (Figure 12) represents the simple prey and predators which are included in the program and how they behave. An FSM was a good way to

represent the logic of the creatures initially, as it gave direction on what methods to develop and what states the creature exists in. The start state is searching for food, as this is the first action prey or predators perform once spawned in. The “Die” state has no arrows from it, as this is the creature’s dead state. In FSM terms, this is a state from which there can be no state changes. This is accurate for the program, where dead creatures are destroyed at the end of the program.

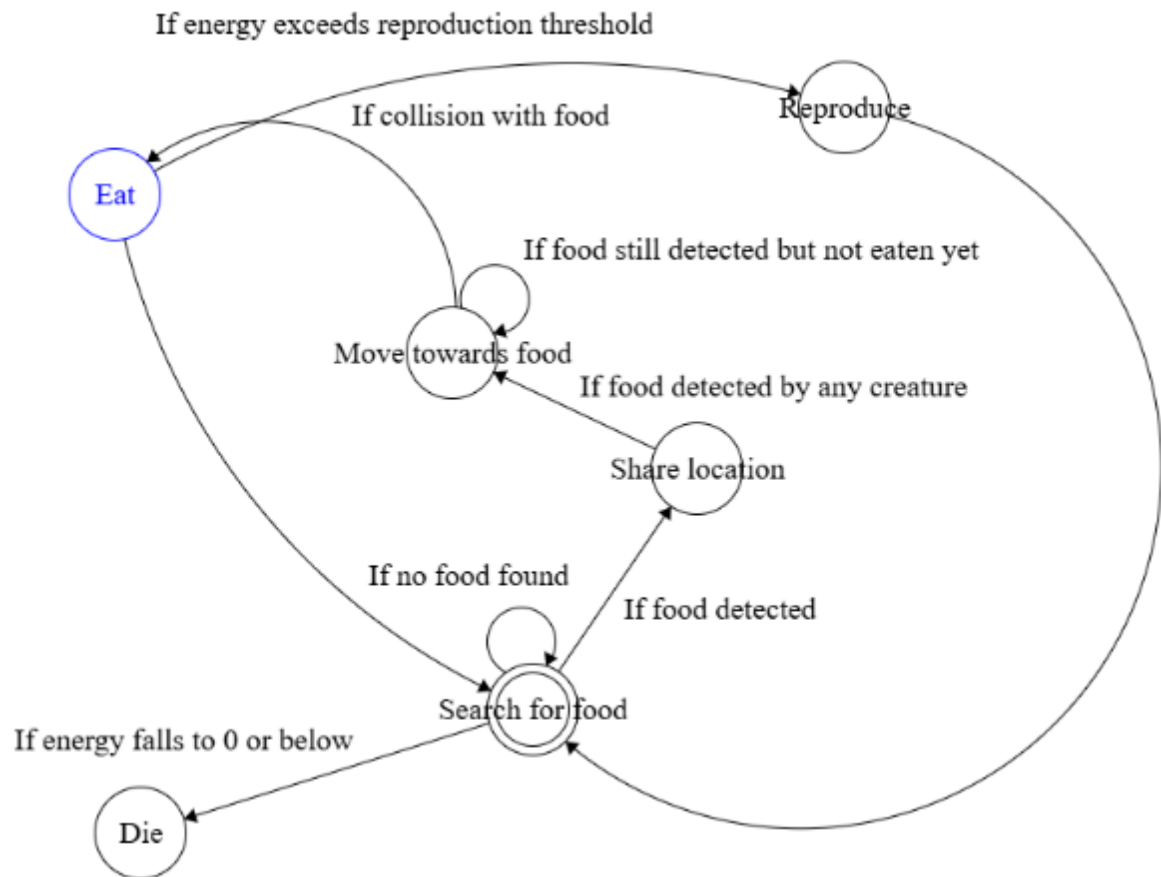


Figure 13: Wolf/Deer FSM Diagram

This model represents the wolves and deer, which were implemented. They are mostly similar, but have a “share location” state, which activates whenever food is detected. This means that if there is food detected, the rest of the organisms are also made aware, thus modelling how deer move in herds or how wolves move in packs.

1.5.5. Prototyping

To determine that this project was feasible, it was considered to build small prototypes of a system to graph the Lotka-Volterra equation. An application using matplotlib to plot these curves was developed. A major help here was this YouTube channel. [Accessible via this link here](#). This channel had a variety of videos on different models to test out, which were followed to make models following both Lotka-Volterra and Lotka-Volterra using Euler’s method, a method of fitting the

continuous equations with discrete time steps. This is the method chosen to use in the final program as well.

1.6. Potential Solutions

Python is the main programming language used in this application, as this is the one I am most proficient in, and it is the best at handling data. A wide array of libraries are accessible with Python, which can help me to control aspects of the program, such as NumPy and matplotlib.

SQL will be used to implement the login and database features, due to its industry-standard reputation. It was considered to use MongoDB, as there is a chance that tables can be subject to SQL injection, however, it was concluded that the structure of data tables was more useful for my project. Parameterised methods of passing in data will be used to ensure that any data input is not interpreted as a command.

Visual Studio Code will be used to create this application, particularly due to the multiple file view feature, as a modular system of file organisation was implemented, and the use of SQL in the project, which is available on Visual Studio Code through the Transact-SQL extension. The extensions available on Visual Studio Code are highly useful, helping to view tables at runtime and highlighting the different data types, making debugging easier

1.7. Objectives of the Simulation Program

When planning out this simulation, a few ideas on the exact system requirements were considered:

A secure system is required to make sure only the student is aware of their username and password, and that teachers/IT staff can use the simulation without requiring an account, if need be, i.e. there must be an admin account. The passwords will be secured by using hashing functions before storing them in the database.

User details must be accessible only through secure methods. The databases must be fully normalised to 3rd form to ensure the efficiency of the system.

A menu is required to ensure users can view and access the simulation. Each simulation must attempt to accurately model the relationship between predator and prey using the given parameters, and the details of the simulation must be clear to the user.

A dynamic system must be implemented to instantiate objects at runtime of specific classes which are created. These classes must be updated per generation, and this must occur while the program is running. If issues occur, they must be error handled to avoid the program crashing.

Graphs and data points are crucial to understanding the simulations, as these explain the concept being developed. These must be created and updated every few

generations to represent the population. The graphs will be created using theoretical models such as Lotka-Volterra, so the parameters must be visible, and the user should be informed of what these are

Using these thoughts, the following primary objectives have been developed for the proposed simulation program.

- A. Create a secure login system for students to ensure their details are protected using hashing to encrypt passwords
- B. Create local normalised databases to store, insert and delete user details using SQL, allowing users to download previously saved graphs
- C. Create a simulation based on biological concepts using user-defined algorithms to simulate the behaviour of animals
- D. Create a user interface to help users interact with the program
- E. Create a feature to track the data of organisms and create graphs as the simulations continue
- F. Allow users to compare searching methods and determine their effectiveness in a competitive natural environment, and demonstrate the predator-prey relationship

After developing the program further, the detailed testing objectives were developed:

Test No.	Test Description and Objectives
Test 1	The user should first be presented with the option to log in or make a new account.
Test 2	If the user clicks on the new account screen, they should be taken to a screen which asks them to input their email.
Test 3	If the user inputs an invalid email, display an error message.
Test 4	If the user keeps inputting an invalid email, close the program
Test 5	Once the user inputs a valid email, an email should be sent to the user.
Test 6	A completely random code should be generated to verify that the user is real and has access to the email account input

Test 7	The email should contain a unique code that the user must enter into the program to verify they are a real person and have access to the email account that was input.
Test 8	If the code entered is invalid, an error message should be displayed
Test 9	If the user keeps inputting an invalid code, close the program
Test 10	Once the code is validated, the user should be taken to a page which records their details
Test 11	The user should be asked to enter their first name and last name
Test 12	The user should be asked to enter their username and password twice, to confirm these are the correct details
Test 13	Once the user has entered these details, they should be stored in a local database, alongside the email which the user used earlier.
Test 14	The password needs to be hashed to ensure that user details are not stored insecurely.
Test 15	If successful in creating an account, the user should be presented with the option to return to the welcome page.
Test 16	If the user clicks on the login option, they should be presented with the option to enter a username and password.
Test 17	If the username or password are incorrect, display an error message.
Test 18	If the username and password are correct, they should be taken to a screen which explains the simulation details to them.
Test 19	The user should be presented with the options of moving to the creature selection screen, or to view their saved graphs
Test 20	If the user chooses to view their saved graphs, take them to the graph downloads page
Test 21	If the user has any saved graphs, these should be downloaded to the user's downloads folder as a zip file containing all saved graphs.
Test 22	If there are errors, or if the user has no saved graphs, display an error message.
Test 23	The user should have the option to return to the main screen from the graph downloads page

Test 24	If the user chooses to move to the creature selection screen, a list of choices of species of creatures should be given to the user.
Test 25	The user should be able to select which species they would like to include via check buttons
Test 26	The next screen should display spin boxes for the speed of the simulation, the number of food and the number of animals, specifically only for the selected species.
Test 27	If the numbers input by the user are valid, the user should be taken to the simulation screen.
Test 28	The simulation should spawn the number of food and the number and types of creatures specified.
Test 29	The creatures should be able to move properly, according to their specified move logic.
Test 30	The prey should be able to eat when they land on food, removing the food from the grid.
Test 31	The predators should be able to eat when they land on prey, removing the prey from the grid.
Test 32	The predators/prey should reproduce when they reach the reproduction threshold.
Test 33	The predators/prey should die when their energy reaches 0 or below.
Test 34	The simulation should spawn food every 10 seconds
Test 35	A graph should be produced side by side with the simulation.
Test 36	This graph should record current population values of the predator and prey classes over time.
Test 37	This graph should display a curve fitted to the parameters found in the simulation.
Test 38	Using values from the simulation, the program should calculate the Lotka-Volterra differential equations and display the relevant parameters on the screen.
Test 39	The simulation should be able to be paused using the spacebar. This should stop animals from moving, the graph from being generated, and the timer should stop.
Test 40	The simulation should stop if the number of food or the number of prey becomes 0 or if the time runs out.

Test 41	The simulation should then be stopped, and a visual should be shown to indicate this.
Test 42	The user should be given the option to hide real values from the simulation, to just display the curve-fitted graph.
Test 43	The user should be able to save the graph, both downloading it to their downloads folder, in its current state of real values/no real values and saving it to their profile in the database.

Table 2: Test Plan Table

Chapter 2. NEA Design

2.1. Description

This project's design, at its heart, is motivated by ease of use and user-interactivity. For example, tkinter, a very popular GUI, was used to create the system in order to allow users to interact with the system familiarly, as opposed to using a CLI. This interface enables users to make accounts, log in, informs the users about the program's function, takes the user's parameters and overall allows for a pleasing human-computer interaction. Interactive controls such as tkinter's Checkbuttons and Spinboxes are familiar to users and allow for a smooth experience, without needing to adjust for text-based complications such as misspellings and missing capitalisation. The simulation is run in a separate Pygame window, which separates the input interaction layer from the simulation display layer. User control is still emphasised in the display through the ability to pause and end the simulation using keyboard inputs, and the user is presented with buttons they can interact with using the mouse.

An object-oriented approach was taken for security and organisation. This separates the program elements, which allowed for the maintenance of a highly organised codebase. It also allowed for unit testing, since classes were separated and therefore could be run individually for testing, as opposed to testing the system all at once. Inheritance was used for the benefit of code reusability to reduce redundancy, with polymorphism being used to change methods, whilst keeping the same common interface. Composition was used to share functionality across unrelated classes, where maintaining a high level of class encapsulation was paramount.

Databases were used to store user data in a secure location. This abstracts the process of data storage from the user, as they interact with data through the user interface and never directly with the database. All created tables were normalised to the 3rd form to eliminate data redundancy and ensure all created data is consistent. This style of data organisation led to efficient querying, as the number of queries required to access data was minimised.

2.2. File Structure

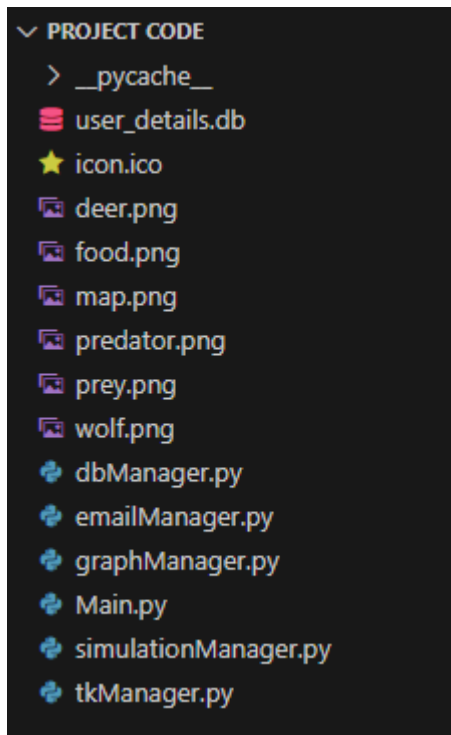
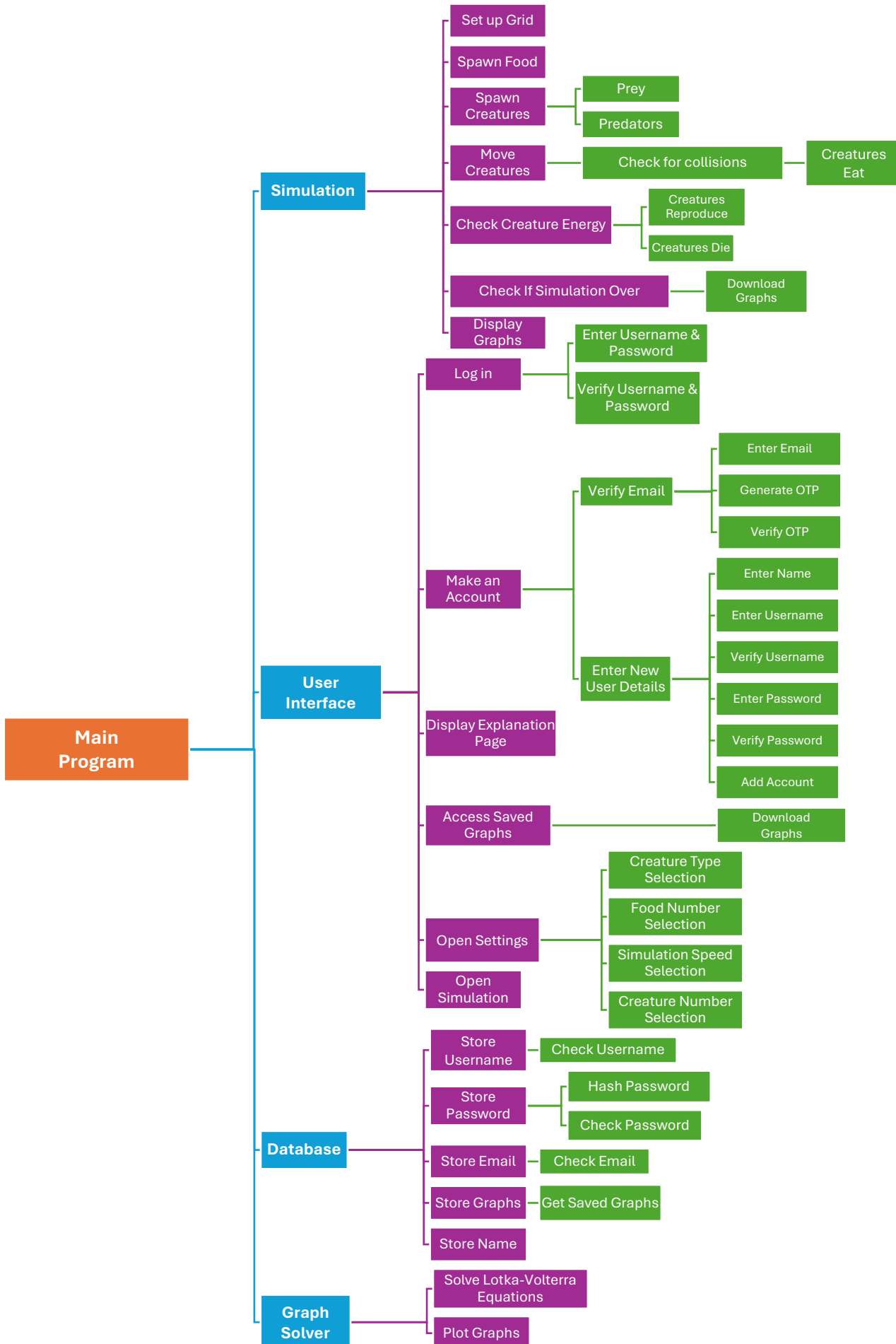


Figure 14: Project Code Folder

The project files were organised in a single folder, removing the need for complex path handling and preventing runtime errors from importing across directories. This decision was consciously made to future-proof the project, as keeping all the resources together ensures the project can be easily zipped and packaged for distribution. This increased the accessibility of the program and was more appropriate for the size of the NEA project. A modular design was followed with the files, with `Main.py` serving as the main script, and relevant module files and classes from modules being imported at runtime. Each module file was self-contained and only related to one specific aspect of the code.

2.3. Structure/Hierarchy Chart



This structure/hierarchy chart was created to model the separate parts of the solution. It highlights the main features of the program which are the database, graph solver, user interface and the simulation. This chart decomposed the main program into these sections and then provided insight into the required subroutines to be developed for the program.

2.4. Algorithms and Flowchart

A few algorithms have been evaluated to simulate animal behaviour in the program. These algorithms are going to control how the creatures will move and what stimuli are prioritised. The use of some of these algorithms may require additional resources compared to others, and thus, may require an energy cost for the more memory-intensive algorithms. Through this research, the best algorithm to represent animals in the wild will be determined. The following algorithms were assessed for this project.

2.4.1. Random Walk

Description: This is where the animal will randomly pick a direction every tick, regardless of any stimuli.

Pros: There is no extra memory cost, and it is simple to implement.

Cons: The animal is unlikely to survive if food is scarce and may not be able to compete with other species.

2.4.2. Biased Random Walk

Description: In a biased random walk, the animal will initially walk in a random direction, and then a bias will be applied based on where the food was located last. The animal will therefore be more likely to go in this direction but is not guaranteed.

Pros: There is very little memory cost, which makes the algorithm easy to implement.

Cons: Despite the improvement on the random walk, as food is generated randomly each time, the bias may not help the animal.

2.4.3. Greedy Algorithm

Description: The animal will move randomly until food enters its sense range. Once it detects food, it will move towards it.

Pros: This is more accurate to the behaviour of actual animals

Cons: If the animal has a small sense range and doesn't detect the food, this method effectively becomes a random walk. Furthermore, if the animal is a prey creature, this may lead them to predator creatures.

2.4.4. Spiral Search

Description: The animal will move in a spiral pattern from its starting position.

Pros: The animal will cover much of its local area.

Cons: If the animal spawns in a sparse region, it will take longer to find food, which is inefficient and it may run out of energy before finding any food.

2.4.5. Levy Flights

Description: These are a special type of random walk, which models the behaviour of animals which hunt over large, sparse environments, such as sharks/albatrosses. In this model, the animal travels in a random direction for a specified distance according to a random reading on a Levy distribution. A Levy distribution is distributed such that short walks are common, but there is a small chance for a very large walk in a direction. Thus, the animal will move in a series of short walks, with the occasional long walk, placing it in a unique location

Pros: This method is directly inspired by animal behaviour and will work very effectively when food is scarce in an environment. Additionally, as this is a type of random walk, no additional memory is required; therefore, this method has no energy cost.

Cons: Despite the increased probability for the animals to move to locations with more food, this is not guaranteed, and the animals may move to a very sparse location. It is still based on random probability. It is difficult to implement.

2.4.6. Breadth-First Searching

Description: This method of searching involves the animal sending out search trails at the start of the program to determine the locations of food. It will then rank the directions to go in and follow this set path

Pros: The animal is guaranteed to know the location of food and thus has a high likelihood of survival

Cons: The animal will have to expend energy and will remain vulnerable to being eaten while it is searching, thus making this a high-risk high-reward strategy.

2.4.7. Depth-First Searching

Description: Depth First Searching is defined by persistent exploration. An animal with this algorithm will explore the randomly chosen direction fully until the animal hits a wall, and then it will explore in another direction, aiming to explore the entire environment deeply

Pros: The animal will be able to explore each direction fully and will not turn away at each tick, increasing the probability of running into food

Cons: The animal is prone to running out of energy before it reaches its target

Simpler creature prototypes were made before implementing more complex algorithms, and the first implementation included creatures which used the random walk algorithm.

```
SIMPLE CREATURE PROTOTYPE:
-----
CREATE LIST(MOVES)

WHILE ALIVE:
    MOVE RANDOMLY FROM LIST(MOVES)
```

Figure 15: Random Walk Algorithm

A randomised greedy algorithm for simple prey and predators was implemented, as this would lead to more natural behaviour for the simulation, leading to results which were accurate to a real ecosystem.

```
SIMPLE PREY/PREDATOR LOGIC:
-----

GET LIST(FOOD LOCATIONS)

WHILE FOOD REMAINS:
    IF FOOD IN RANGE:
        ADD TO LIST(VALID LOCATIONS)

    IF FOOD IN LIST(VALID LOCATIONS):
        TARGET POSITION = CLOSEST FOOD

    CREATE LIST(MOVES)
    FILTER LIST(MOVES) < CREATURE SENSE RANGE

    IF LIST(MOVES) EXISTS:
        IF TARGET POSITION EXISTS:
            MOVE TOWARDS FOOD
        ELSE:
            MOVE RANDOMLY FROM LIST(MOVES)
    ELSE:
        STAY STILL
```

Figure 16: Randomised Greedy Algorithm

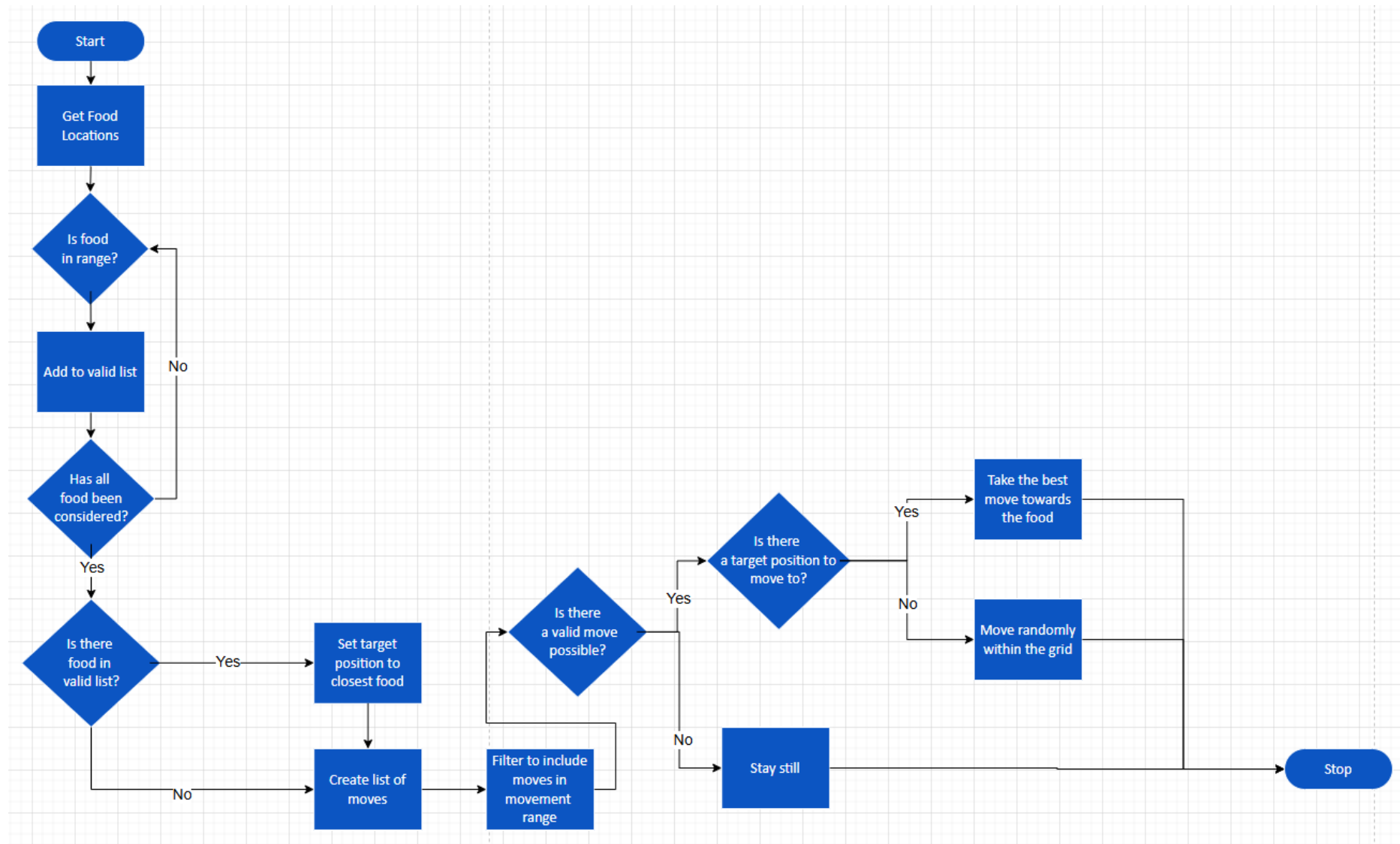


Figure 17: Randomised Greedy Algorithm Flowchart

For the wolves and deer, a randomised greedy algorithm was chosen to be implemented instead, but with communication between animals, which effectively creates a creature which will perform a randomised greedy algorithm on a local level, and if no food is found, it will do a biased random walk to the closest shared location, combining the best of both worlds.

```

WOLF/DEER LOGIC:
-----

GET LIST(COLLECTIVE KNOWN LOCATIONS)
GET LIST(FOOD LOCATIONS)

WHILE FOOD REMAINS:
    IF FOOD IN RANGE:
        ADD TO LIST(VALID LOCATIONS)
        ADD TO LIST(COLLECTIVE KNOWN LOCATIONS)

    IF FOOD IN LIST(VALID LOCATIONS):
        TARGET POSITION = CLOSEST FOOD

    CREATE LIST(MOVES)
    FILTER LIST(MOVES) < CREATURE SENSE RANGE

    IF LIST(MOVES) EXISTS:
        IF TARGET POSITION EXISTS:
            MOVE TOWARDS FOOD
        ELSE:
            MOVE TOWARDS CLOSEST FOOD IN LIST(COLLECTIVE KNOWN LOCATIONS)
    ELSE:
        STAY STILL

```

Figure 18: Randomised Greedy Algorithm with Biased Random Walk

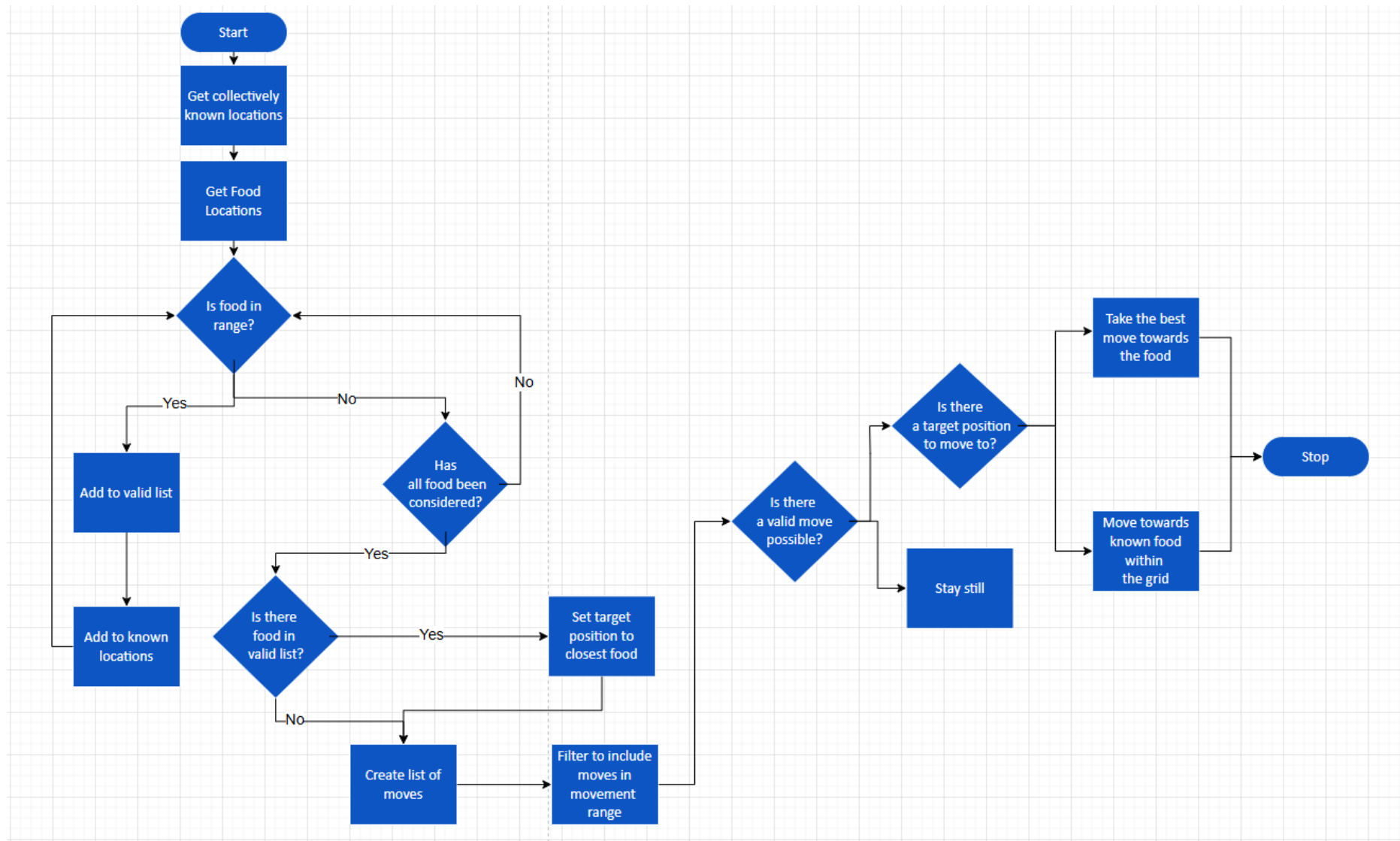


Figure 19: Randomised Greedy Algorithm with Biased Random Walk Flowchart

2.5. Entity Relationship Diagrams

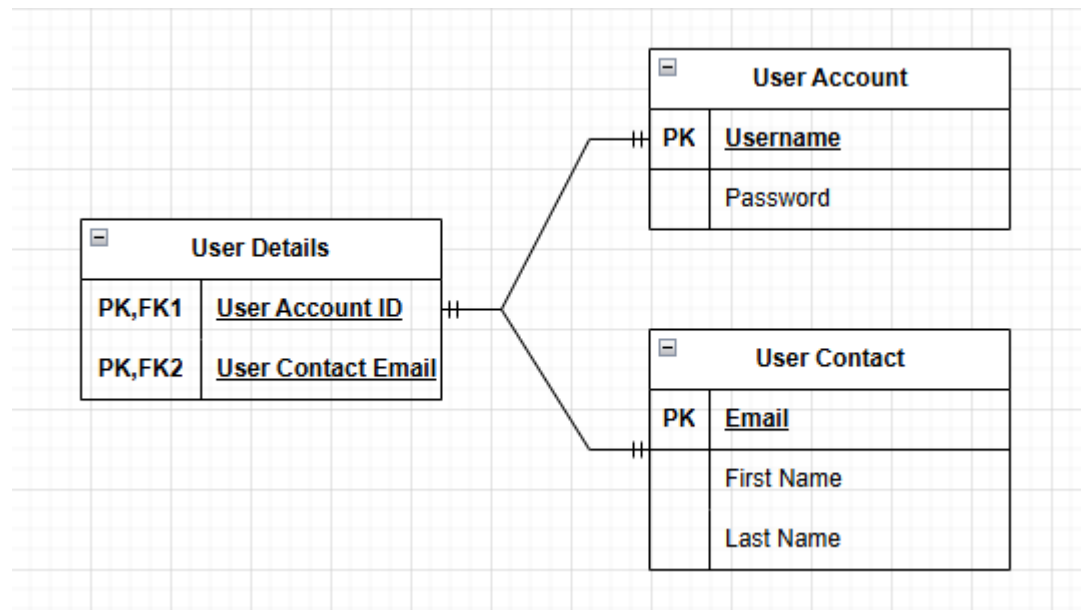


Figure 20: User Details ERD

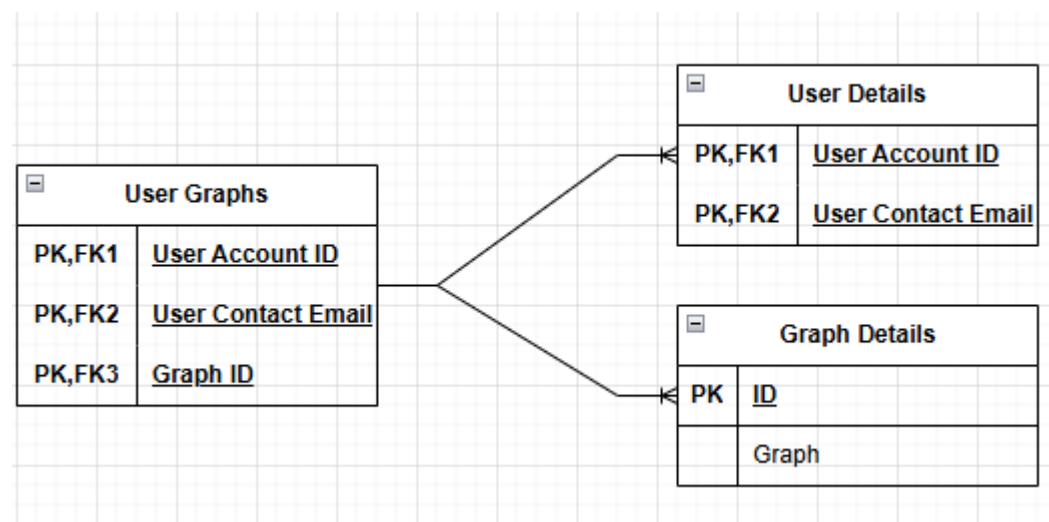


Figure 21: User Graphs ERD

These Entity-Relationship Diagrams expand on the ones created in the Analysis Section, as they represent the actual relationship present between the tables in the program. The first diagram shows that user details can only be linked to one set of account details and one set of contact details, which means that an account can only have one username and email address. Conversely, an email address and a username must be related to only one account. This means users are free to create multiple accounts, but they must use different details, which is more secure. The second diagram demonstrates that the user graphs table can hold many graphs and many users, but graphs can only be linked to one user. These follow the principles that the logical model set out earlier.

2.6. Class Diagrams

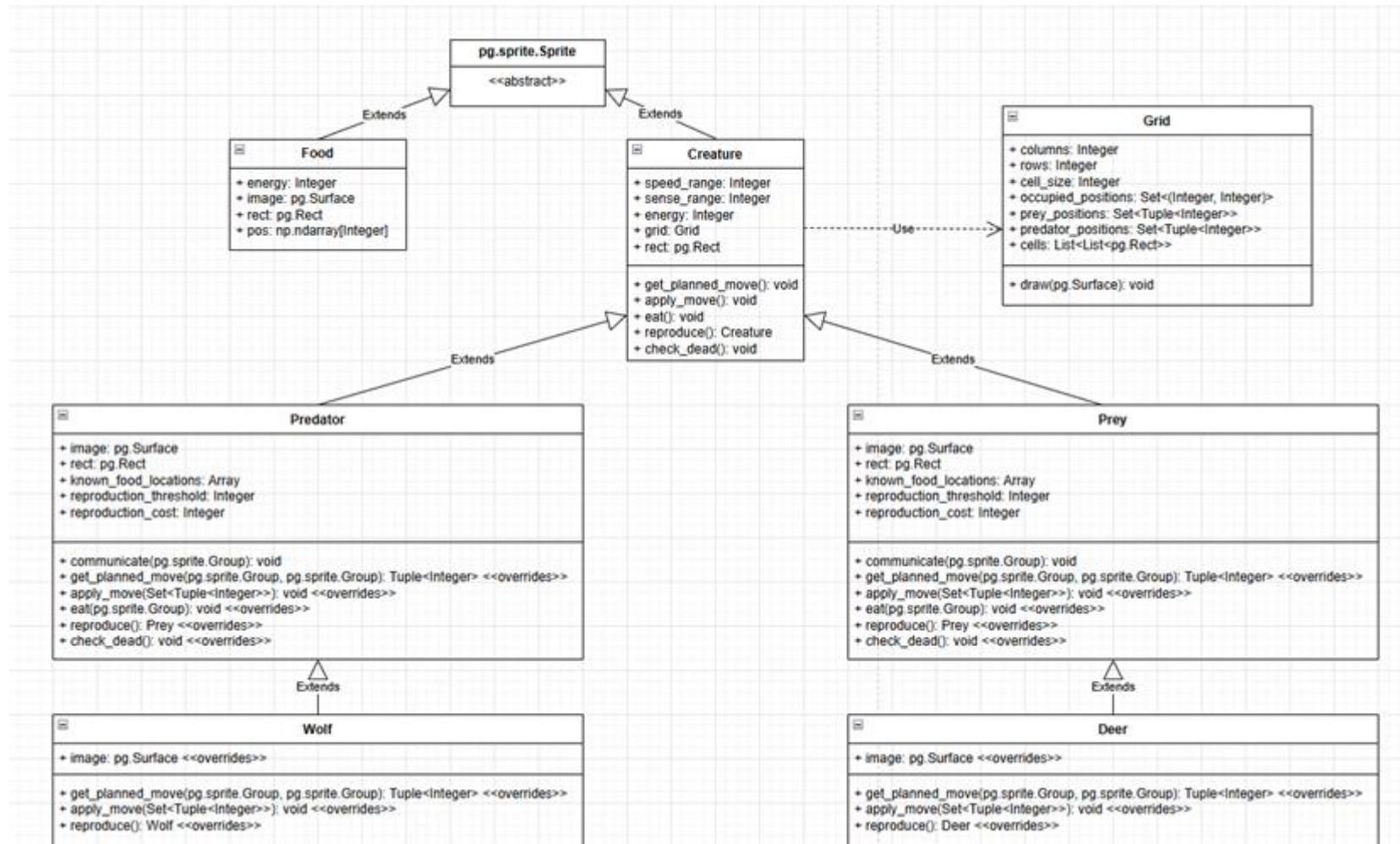


Figure 22: Simulation Class Diagrams

This diagram demonstrates the inheritance chain used by classes in the simulationManager file. It shows how creatures and food were modelled by the Pygame Sprite class. It also shows how the creatures have a dependency connection to the Grid. Prey and Predators are subtypes of Creature and inherit values such as energy, speed range, and sense range. However, they inherit and then override methods related to moving, eating, reproduction and checking if they are dead. They also have unique attributes, such as known food locations and images. Wolves and Deer then go on to inherit methods of eating and checking if they are dead explicitly but demonstrate polymorphism through their inheritance of the movement-related methods, redefining them to work differently whilst keeping the same name.

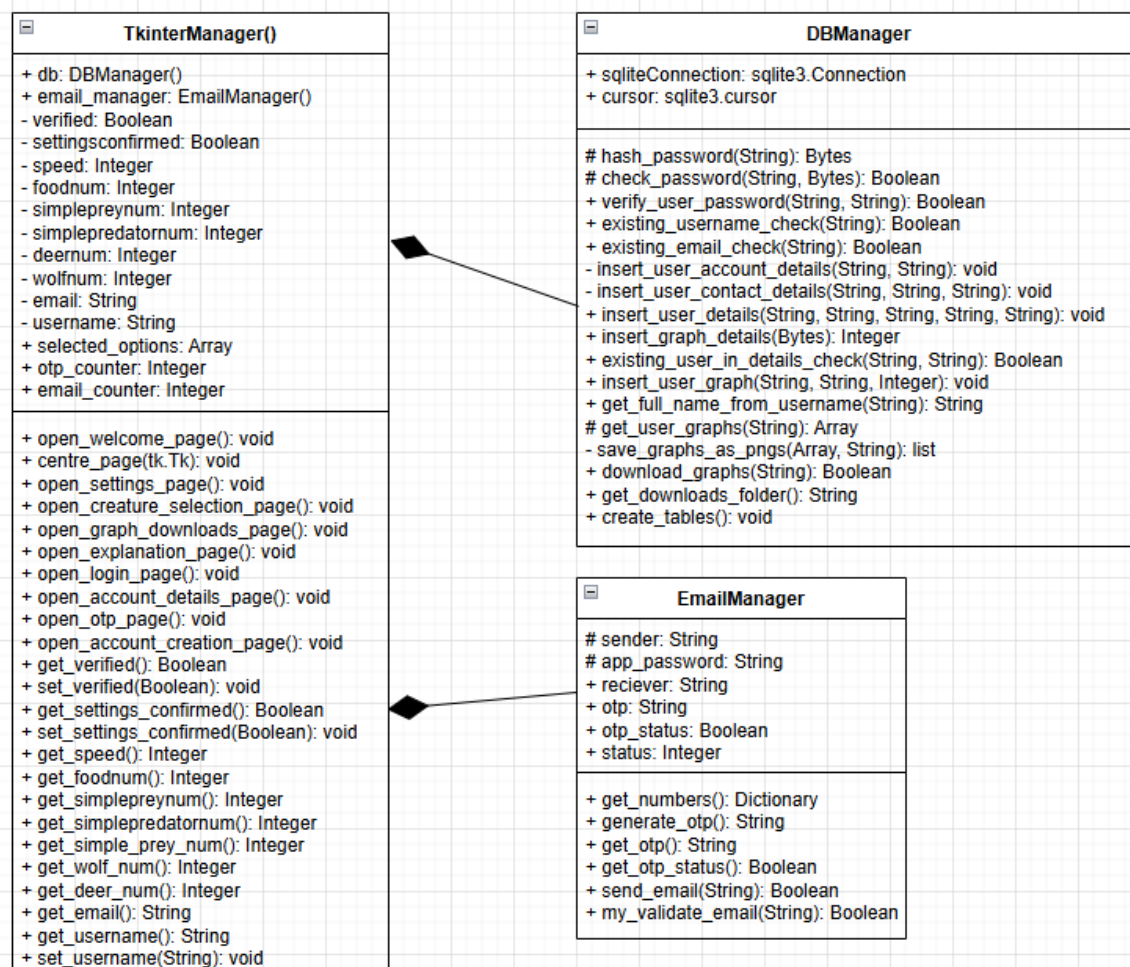


Figure 23: Client Server Class Diagrams

This diagram demonstrates how the TkinterManager, EmailManager and DBManager classes are related. There is a composition relationship between TkinterManager and the rest of the classes, as they are instantiated within the class TkinterManager. This means that if the TkinterManager class is destroyed, the EmailManager and DBManager classes will also be destroyed, not being able to exist outside of the TkinterManager class during runtime. The TkinterManager class interacts with these classes through their interfaces, using the getter and setter methods contained within them to obtain values. Similarly, the TkinterManager class is also interacted with through its interface in the main program.

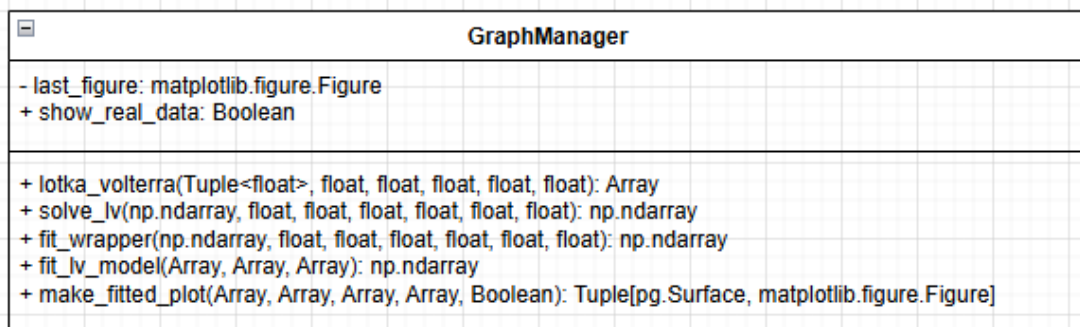


Figure 24: GraphManager Class Diagram

This diagram demonstrates how the GraphManager class is set up as a stand-alone class. It contains multiple methods and a few attributes related to its function of managing the graphing portion of the code, fully encapsulating this section of the program.

2.7. Data Tables

User Account					
Field	Key	Foreign Key	Data Type	Constraint	Notes
Username	Primary		String		
Password			Binary	NOT NULL	Stores hashed version of password

Table 3: User Account

The table above was used to store the user's account details, with the primary key being the username, which is unique, and a hashed password, stored as a binary BLOB object.

User Contact					
Field	Key	Foreign Key	Data Type	Constraint	Notes
Email	Primary		String		
First Name			String	NOT NULL	
Last Name			String	NOT NULL	

Table 4: User Contact

The table above was used to store the user's contact details, with the primary key being the email. The first and last name had to be populated.

User Details					
Field	Key	Foreign Key	Data Type	Constraint	Notes
User Account ID	Primary (Composite)	User Account(Username)	String		
User Contact Email	Primary (Composite)	User Contact(Email)	String		

Table 5: User Details

This table was used to combine the user's account and contact details into one table, where all user details were stored. This became the composite primary key under which the table was referenced.

Graph Details					
Field	Key	Foreign Key	Data Type	Constraint	Notes
ID	Primary		Integer		Auto-increments every time a graph is saved, even when graphs are deleted
Graph			Binary		Stores graph as binary data

Table 6: Graph Details

This table stores the graphs generated by users as binary BLOB objects, which can be remade into PNGs. The ID of the graphs was set to autoincrement, which creates unique IDs forever, something that was required as part of the user experience.

User Graphs					
Field	Key	Foreign Key	Data Type	Constraint	Notes
User Account ID	Primary (Composite)	User Details(User Account ID, User Contact Email)	String		
User Contact Email	Primary (Composite)	User Details(User Account ID, User Contact Email)	String		
Graph ID	Primary (Composite)	Graph Details(ID)	Integer		

Table 7: User Graphs

This table linked all the graphs generated to the users who made them, thus being the table which users queried when retrieving their graphs.

2.8. Human-Computer Interaction

This section outlines the graphical user interface of the developed solution.

Welcome Page

This is the welcome page that the user is greeted with when the program is run. It features options to log in and make a new account.

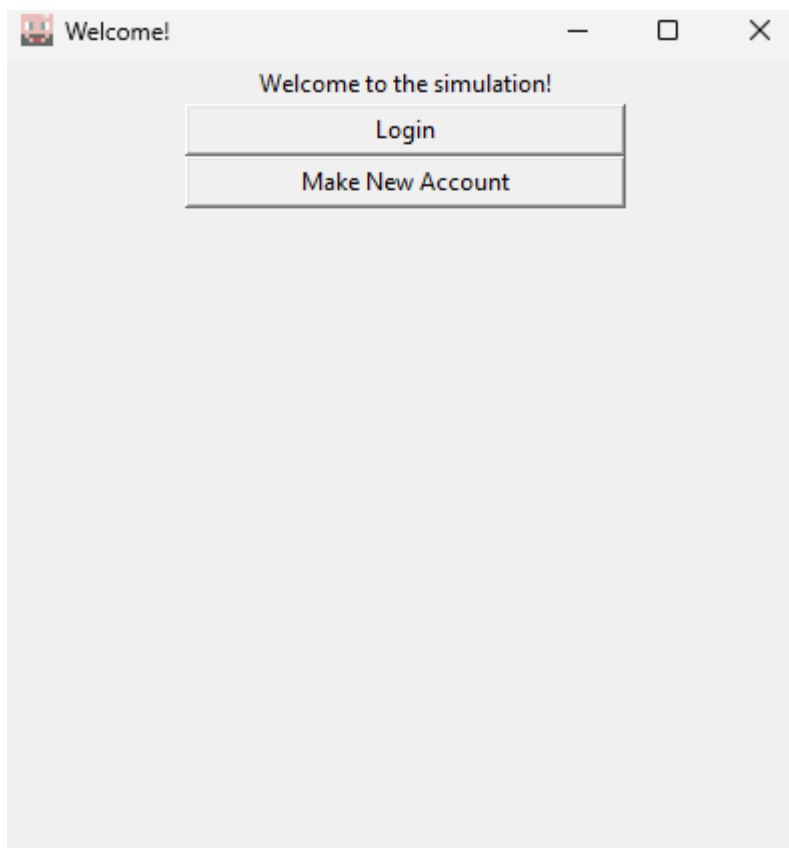


Figure 25: Welcome Page Screen

Account Creation Page

If the user chooses to make a new account, they are taken to this page where they are prompted to enter an email. This email entry is checked to ensure that the email is of the correct format and that the domain is registered. The email entry is also checked to see if the email has been used before. If these conditions are met, an email is sent containing a one-time password (OTP). If they are not met, an error message is displayed. If the user enters their email wrong 5 times, the page is destroyed, as a security measure.

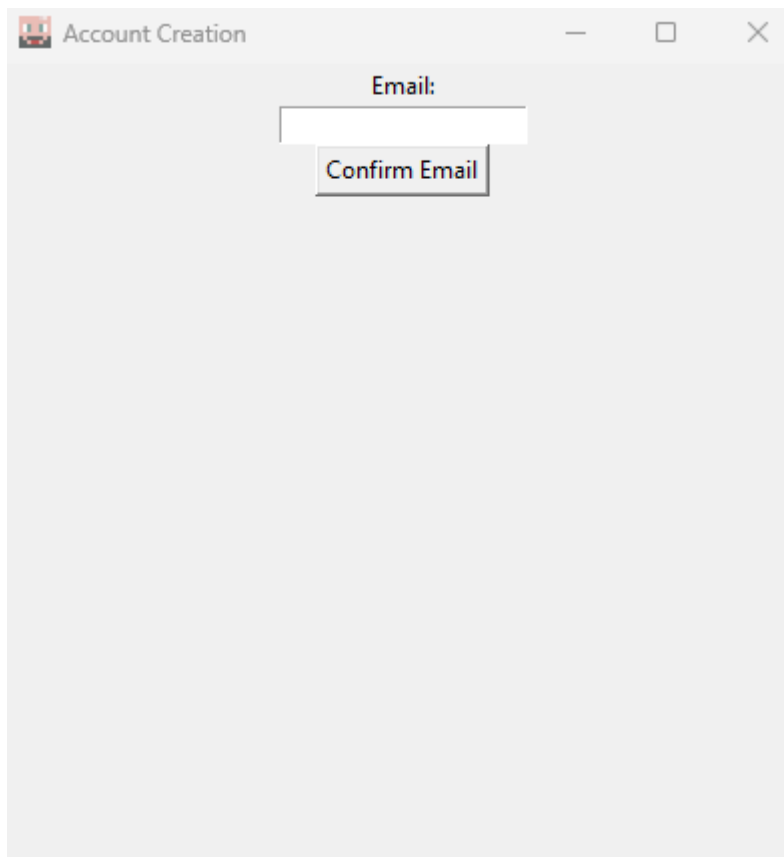


Figure 26: Account Creation Screen

Email

This is the email sent to new users, which welcomes them and includes the OTP, denoted by the code in between the asterisks. It also warns users about the window closing after 3 minutes.

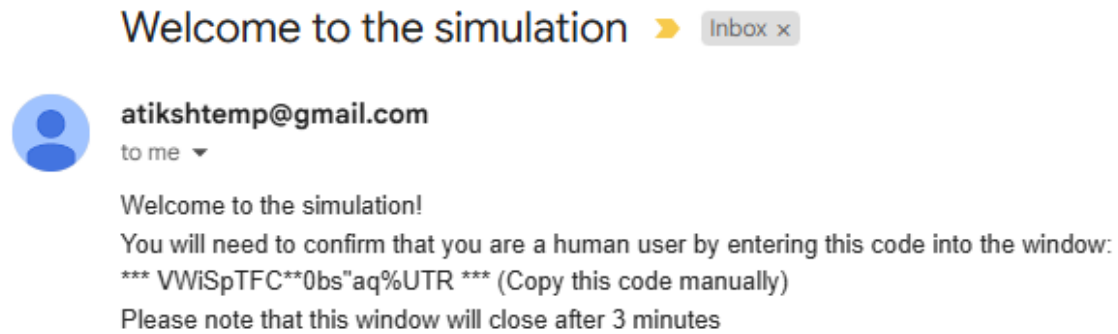


Figure 27: Email Screen

One -Time -Password (OTP) Page

The user is presented with this screen when the email is sent off. This screen is set to close in 3 minutes, which acts as a safety measure to avoid users trying to crack the OTP using a brute force bot. The OTP entry is checked to see if it matches the generated OTP. If the OTP matches, the user is taken to the account details entry form page. If it does not, an error message is displayed. If the user enters the OTP wrong 5 times, the page is destroyed, as a security measure.

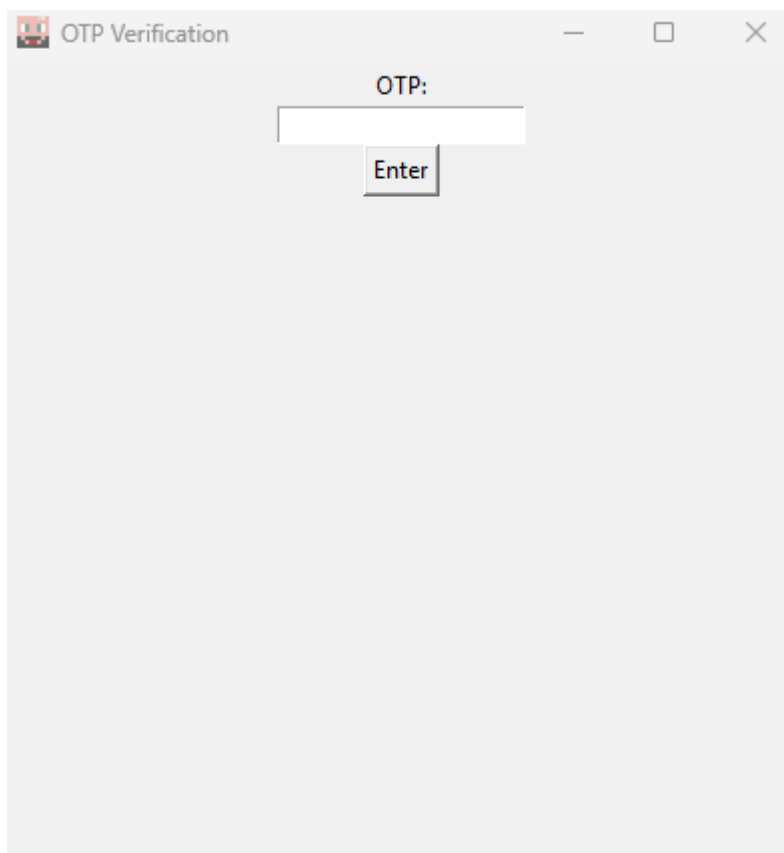


Figure 28: OTP Screen

Account Details Entry Form Page

Users are presented with a form to fill out to add their details so that their account can be made. The entries are checked to ensure that the entry boxes are filled and that the username and password match the “Confirm username” and “Confirm password” entry boxes. The password and username are also checked to ensure they are 8 or more characters. If the details entered are valid, the account is added to the database, and a button appears to take the user back to the welcome page.

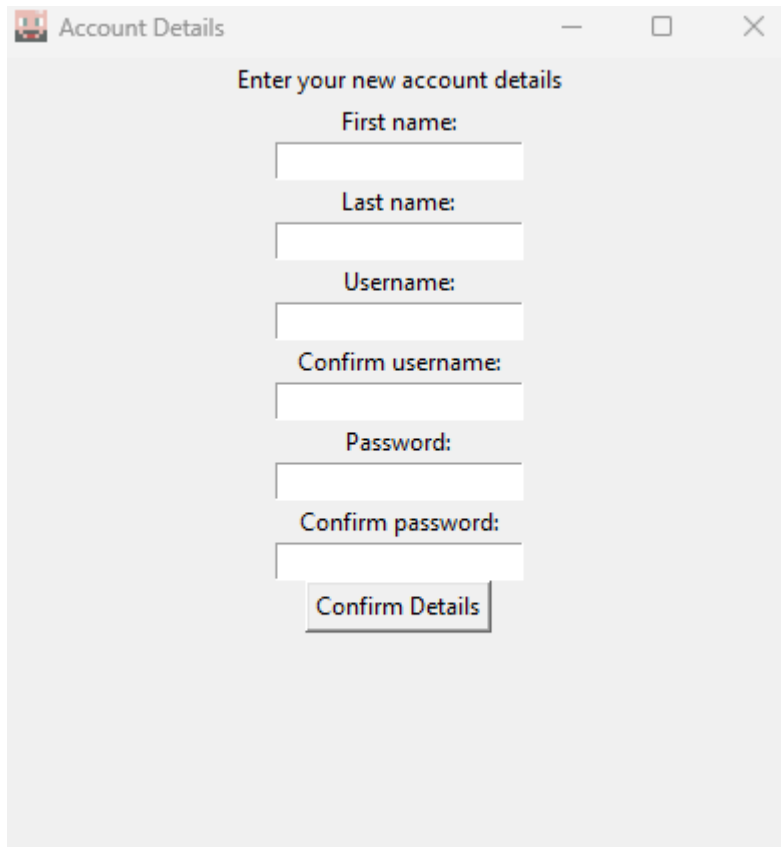
A screenshot of a web application window titled "Account Details". The window has a standard macOS-style title bar with a red, yellow, and green button on the left, and minimize, maximize, and close buttons on the right. The main content area has a light gray background. At the top, the text "Enter your new account details" is displayed in a bold, black font. Below this, there are seven input fields arranged vertically, each with a label above it: "First name:", "Last name:", "Username:", "Confirm username:", "Password:", "Confirm password:", and a "Confirm Details" button at the bottom. The input fields are white with a thin gray border. The "Confirm Details" button is a light gray rectangle with a thin black border and the text "Confirm Details" in a bold, black font.

Figure 29: Account Details Screen

Login Page

The user is prompted to enter a username and password. If the username and password entered is “Admin” and “Password1”, an admin account is opened, which doesn’t have graph saving capability. Else, the username and password are checked to see if they match any stored details in the database. If these details are wrong, an error message is displayed.

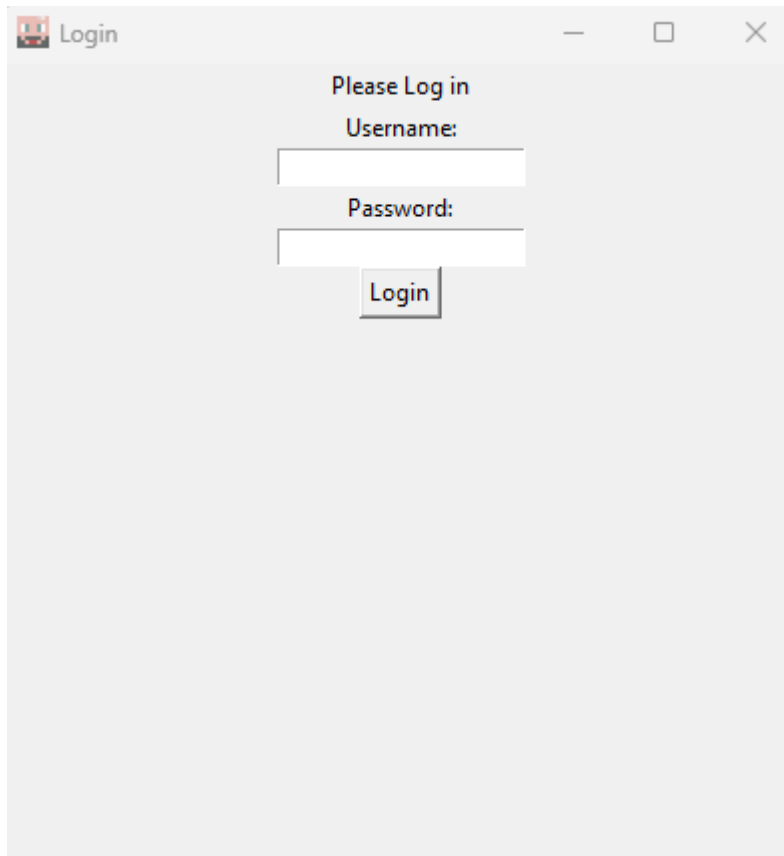


Figure 30: Login Page Screen

Explanation Page

This page serves to help users understand the purpose of the program, displaying a wide array of information about the program. It provides two options for users: either the opportunity to view the saved graphs associated with the user's account or to move to the creature selection page.

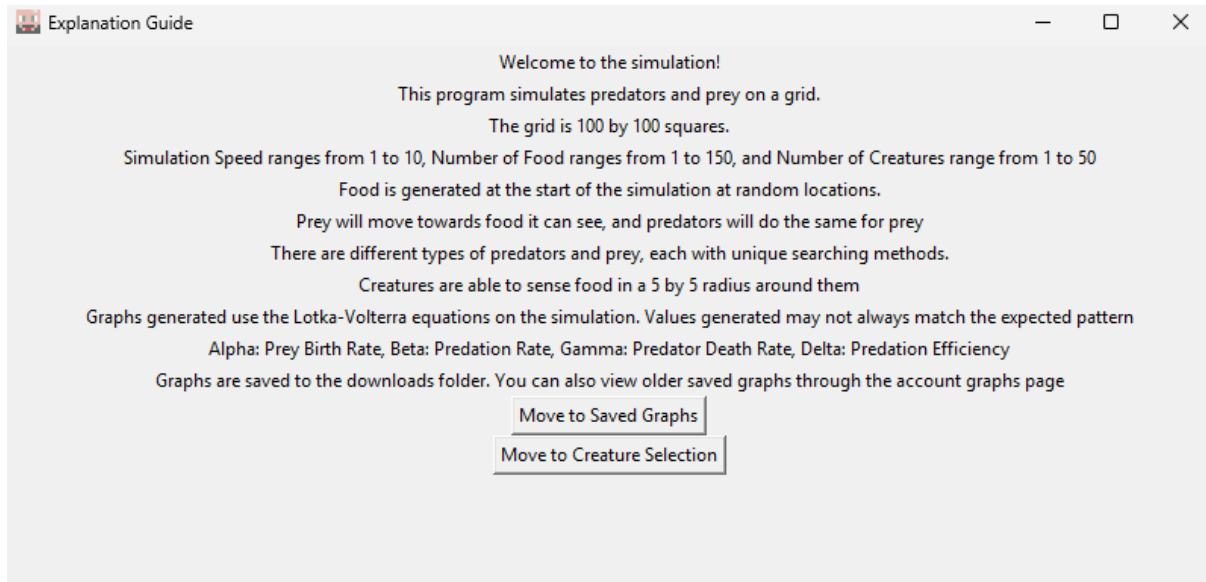


Figure 31: Explanation Page

Graph Downloads Page

This page allows users to download all their saved graphs, in the form of a zip file, saved to their downloads folder. Users are also presented with the option to return to the welcome page, in case they have not saved any graphs, or if they wish to return, to be able to run another simulation.

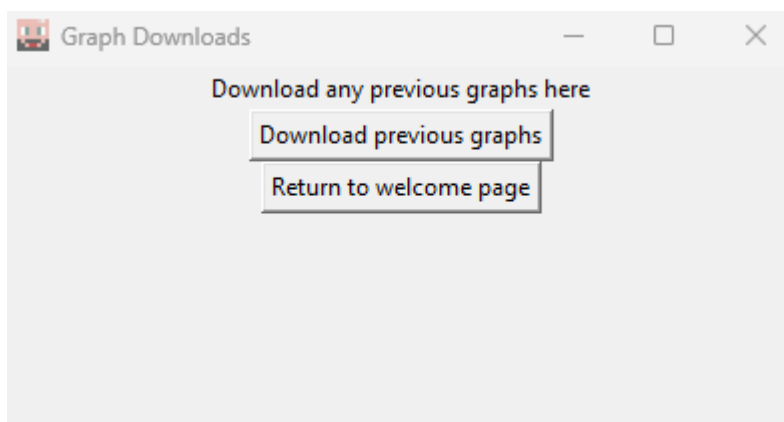


Figure 32: Graph Download Screen

Creature Selection Page

This page uses Checkbuttons to help users select their preferred combination of predators and prey for the simulation. These choices determine the settings displayed on the next page. A button is present to allow users to move to the next page.

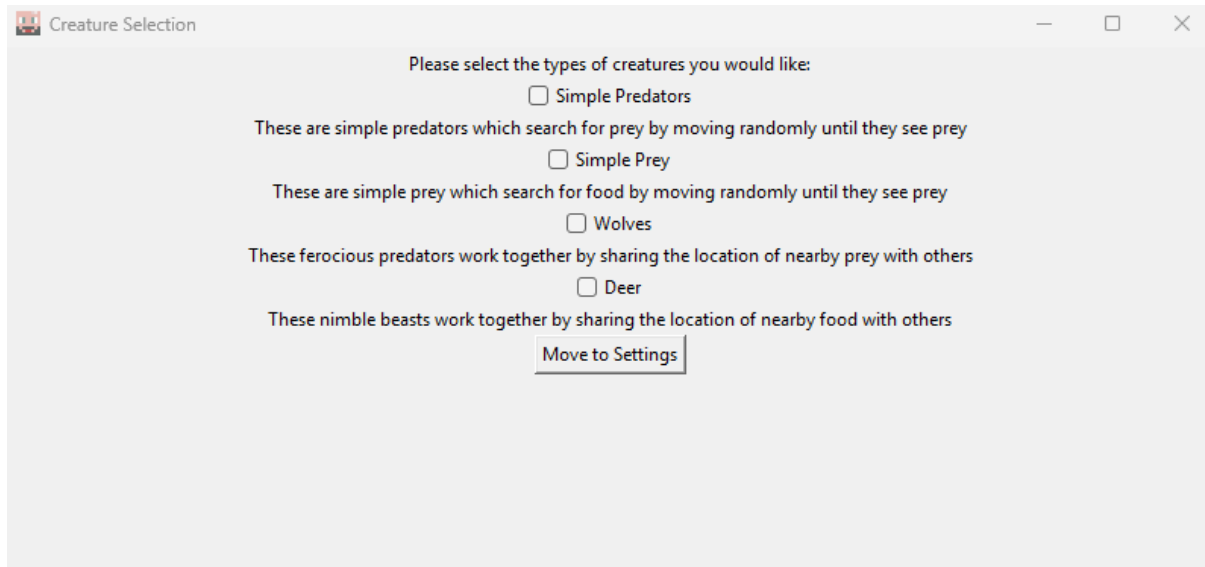


Figure 33: Creature Selection Screen

Settings Page

This page allows users to customise the simulation to their liking using Spinboxes. The simulation speed ranges from 1 to 10. The number of food ranges from 1 to 150. The creature numbers range from 1 to 50. If one of these settings is not met, an error message is displayed; otherwise, the simulation is opened with the user's specified settings.

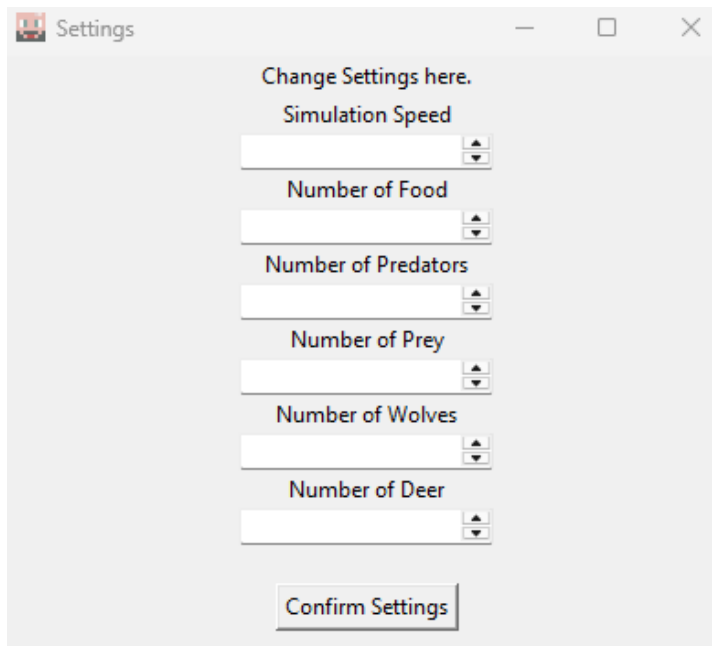


Figure 34: Settings Screen

Simulation Screen

This is the simulation screen. It includes the simulation on the left of the screen, a timer at the top left corner and a graph, which is dynamically generated on the right. Creatures move on the grid, in discrete movements, which the user sees in real time.

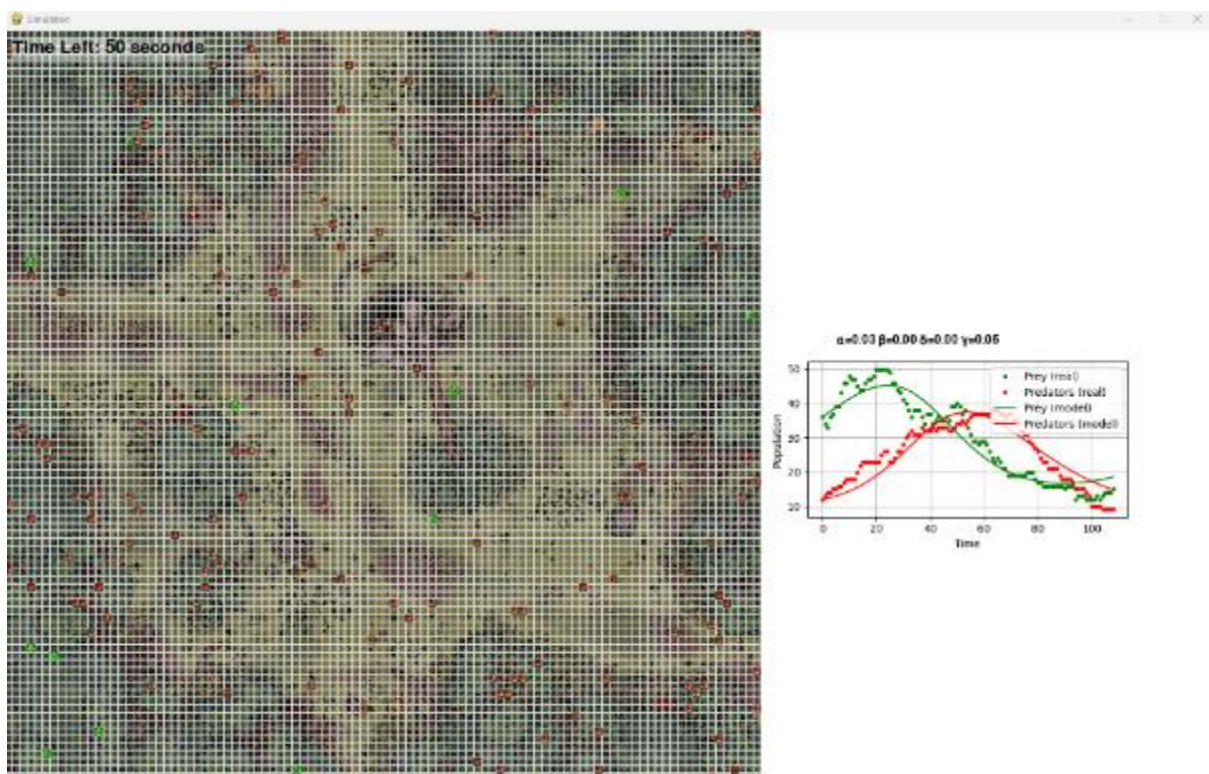


Figure 35: Simulation Screen

Paused Screen

This is the paused screen, which stops the simulation from running and displays a large red “Paused” text to indicate this.

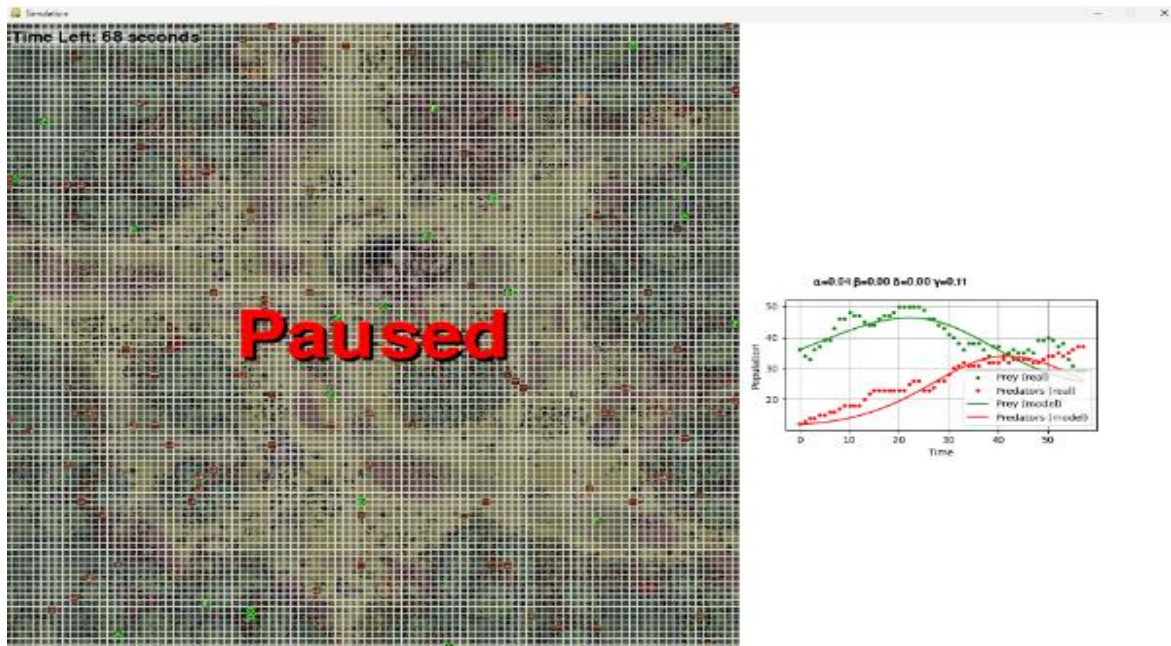


Figure 36: Paused Screen

Finished Screen

This is the finished screen, displayed when the timer runs out, or if the number of prey or food reaches 0.

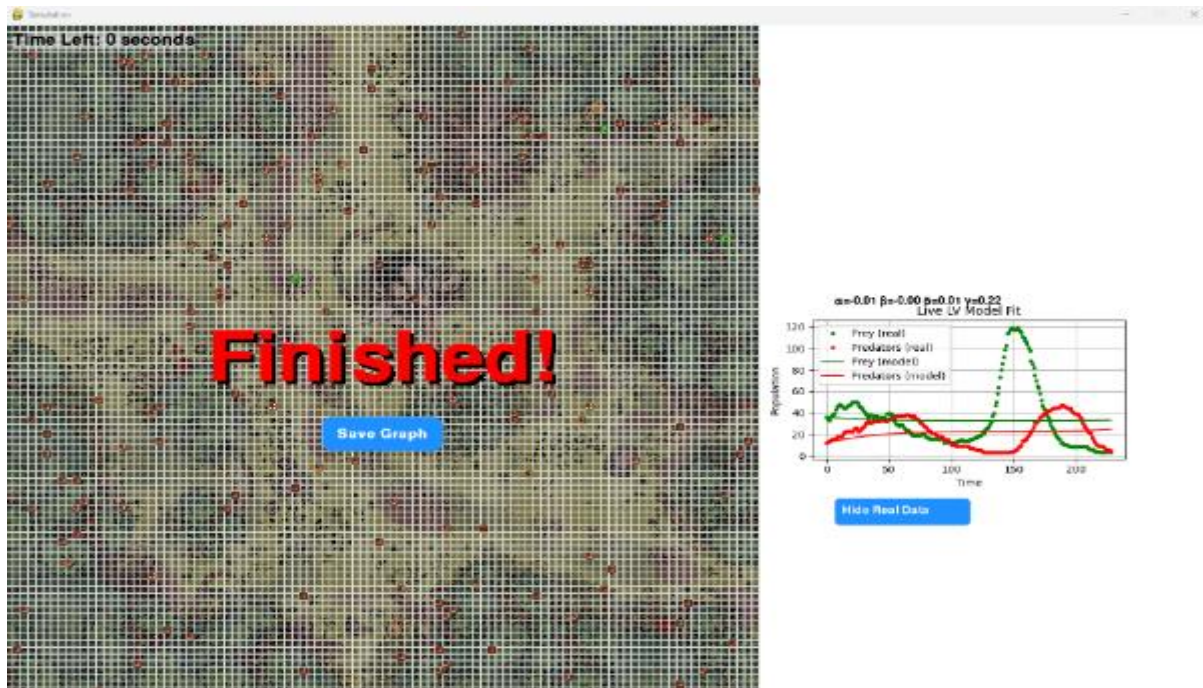


Figure 37: Finished Screen

Graph Saved Message

When a graph is saved, a message is displayed to indicate this.



Figure 38: Graph Saved Message Screen

Saved Graphs

These are the sample saved graphs produced from the simulation.

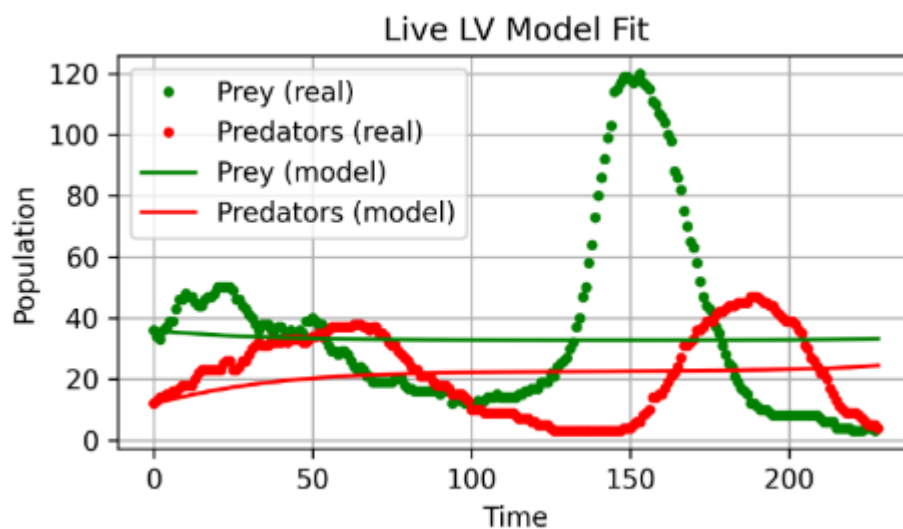


Figure 39: Sample Graph 1

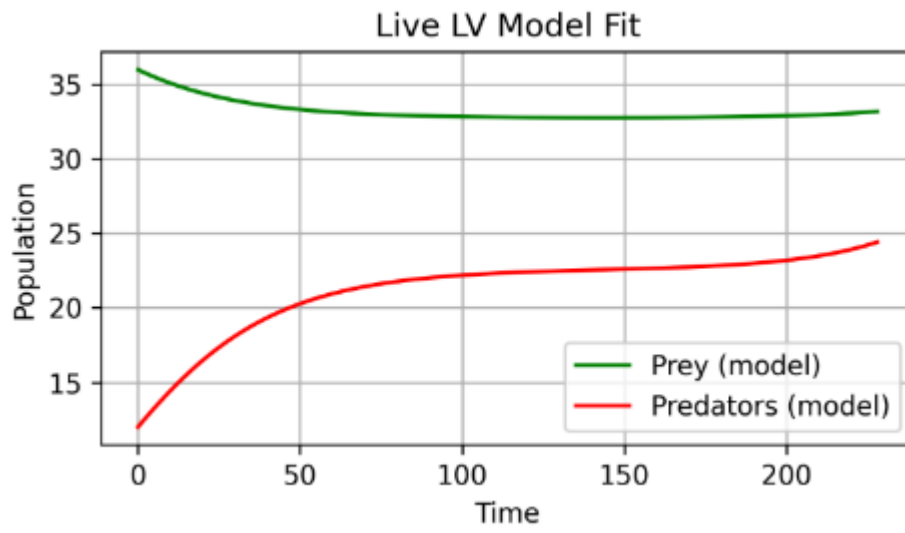


Figure 40: Sample Graph 2

Chapter 3. NEA Technical Solution

3.1. Code Listing

3.1.1. Main.py

```
from tkManager import TkinterManager
# Importing the user-defined TkinterManager to handle all human-computer interaction
from datetime import datetime, timedelta
# Importing datetime to handle features to do with time
from graphManager import GraphManager
# Importing the user-defined GraphManager to handle graphing and calculations
from simulationManager import *
# Importing all classes from simulationManager to handle simulation logic
import os
# Importing os to handle
from io import BytesIO
# Importing to wrap binary in file-like objects, so the binary can be treated as an image file in memory so it can be saved
import matplotlib.pyplot as plt
# Importing matplotlib to process and display image onto a plot figure

def main():
    # This is the main guard entry point

    tkinter_manager = TkinterManager()
    # Instantiates TkinterManager class
    graph_manager = GraphManager()
    # Instantiates the GraphManager class
```

```

if tkinter_manager.get_verified() == True and tkinter_manager.get_settings_confirmed() == True:
    # If the user has logged in and selected appropriate settings, open the simulation
    pg.init()
    # Initialises a pygame window
    display_surface = pg.display.set_mode((WIDTH, HEIGHT))
    # Sets the dimensions to the constants HEIGHT and WIDTH
    display_surface.fill("white")
    display_surface.blit(background_map, (0,0))
    # Blits the map png onto the background
    pg.display.set_caption('Simulation')
    # Sets the title to Simulation
    clock = pg.time.Clock()
    # Starts a pygame clock object to track time in ticks()

    grid = Grid(GRID_WIDTH, GRID_HEIGHT, CELL_SIZE)
    # Instantiates a Grid class

    prey_sprites = pg.sprite.Group()
    # Sprite groups are created to hold the creatures and food
    predator_sprites = pg.sprite.Group()
    simple_prey_sprites = pg.sprite.Group()
    simple_predator_sprites = pg.sprite.Group()
    wolf_sprites = pg.sprite.Group()
    deer_sprites = pg.sprite.Group()
    food_sprites = pg.sprite.Group()

    for _ in range(int(tkinter_manager.get_simplepreynum())):
        # Adds simple prey according to user specification
        prey_sprites.add(Prey(grid))

```

```

        simple_prey_sprites.add(Prey(grid))

    for _ in range(int(tkinter_manager.get_simplepredatornum())):
        # Adds simple predators according to user specification
        predator_sprites.add(Predator(grid))
        simple_predator_sprites.add(Predator(grid))

    for _ in range(int(tkinter_manager.get_deernum())):
        # Adds deer according to user specification
        prey_sprites.add(Deer(grid))
        deer_sprites.add(Predator(grid))

    for _ in range(int(tkinter_manager.get_wolfnum())):
        # Adds wolves according to user specification
        predator_sprites.add(Wolf(grid))
        wolf_sprites.add(Predator(grid))

    for _ in range(tkinter_manager.get_foodnum()):
        # Adds food according to user specification
        food_sprites.add(Food(grid))

    DISCRETE_STEP = 10
    # Sets the number of time steps to pass until paramaters are recalculated
    pop_time = [0]
    # Initialises the population time array
    pop_pre = [len(prey_sprites)]
    # Initialises the population prey array
    pop_pred = [len(predator_sprites)]
    # Initialises the predator_sprites array
    step_number = 0
    # Initialises the counter for each time value

```

```

fitted_params = None
# Initialises the parameters
show_real_data = True
# Sets the initial state of the graph to show the real population data

simulationOn = True
# Turns the simulation on
paused = False
# Initialises the paused checker
simulation_finished = False
# Initialises the finished checker
start_time = datetime.now()
# Records the time at which the simulation started in seconds
last_spawn_time = pg.time.get_ticks()
# Records when the last spawn occurred
spawn_interval = 1000
# Records the spawn interval in ticks for food which corresponds to 10 seconds

while simulationOn:
    # While loop to control logic when the simulation is active
    pg.display.flip()
    # Updates the pygame screen

    if not paused:
        # While the simulation is not paused
        now_time = datetime.now()
        # Gets the time at the instant when the simulation is not paused/unpaused
        elapsed_time = now_time - start_time
        # This records the time between the simulation being unpaused and paused
        current_ticks = pg.time.get_ticks()

```

```

# Gets the time in ticks at the instant when the simulation is not paused/unpaused

for event in pg.event.get():
    # For each pygame event
    if event.type == pg.QUIT:
        # If the X button is pressed, close the simulation
        simulationOn = False
        # Turns off the simulation

    elif event.type == pg.KEYDOWN:
        # Handles key press events
        if event.key == pg.K_ESCAPE or event.key == pg.K_BACKSPACE:
            # If the Escape Key or Backspace key are pressed, terminate the simulation
            simulationOn = False
            # Turns off the simulation

        elif event.key == pg.K_SPACE and not simulation_finished:
            # If the simulation is still running and the space bar is pressed
            paused = not paused
            # Flips the paused state

            if paused:
                # If the program is paused
                pause_start = datetime.now()
                # Records the time at which the pause state started
                font_paused = pg.font.Font(None, 150)
                # Sets the font of the paused message
                text_paused = font_paused.render("Paused", True, (255, 0, 0))
                # Creates a red Paused message

                shadow_text = font_paused.render("Paused", True, (0, 0, 0))

```

```

# Creates a black shadow effect on the Paused message
text_rect = text_paused.get_rect(center=(GRID_WIDTH // 2, GRID_HEIGHT // 2))
# Centres the text
shadow_rect = shadow_text.get_rect(center=(GRID_WIDTH // 2 + 5, GRID_HEIGHT // 2 + 5))
# The shadow is slightly off center to create the shadow effect

display_surface.blit(shadow_text, shadow_rect)
# Draws the shadow
display_surface.blit(text_paused, text_rect)
# Draws the Paused message

pg.display.flip()
# Updates the pygame screen
continue
# Skips the loop, so that the message is only blitted once
else:
    start_time += datetime.now() - pause_start
    # Shifts the start_time by the amount of time for which the simulation was paused, to
offset this time

elif simulation_finished:
    # If the simulation is finished
    if event.type == pg.MOUSEBUTTONDOWN:
        # If the mouse button is clicked
        if save_button_rect.collidepoint(event.pos):
            # If the mouse click is on the save button

            timestamp = str(int(datetime.utcnow().timestamp() * 1000))
            # Gets the current UTC time and then converts it to Unix time, which gives a unique
identifier

```

```

        save_path = os.path.join(tkinter_manager.db.get_downloads_folder(),
f"predator_preym_graph_{timestamp}.png")
        #Sets the save path to the downloads folder

    try:
        latest_figure.savefig(save_path, dpi=300)
        # Saves the figure to the given save path
        print(f"Graph saved to {save_path}")

    except Exception as e:
        print("Failed to save graph to downloads:", e)
        pass

    try:
        image_png = BytesIO()
        # Instantiates a IO object
        latest_figure.savefig(image_png, format="png")
        # Saves the figure as a png in memory, by using the BytesIO object
        image_blob = image_png.getvalue()
        # Extracts the binary data from the BytesIO object
        image_png.close()
        # Closes the BytesIO object
        tkinter_manager.db.insert_graph_details(image_blob)
        # Inserts the binary data of the graph into the Graph Details table
        graph_id = tkinter_manager.db.cursor.lastrowid
        # Gets the graph ID of the recently inserted graph
        user_account_id = tkinter_manager.get_username()
        # Gets the user's username
        user_contact_id = tkinter_manager.get_email()
        # Gets the user's email
        tkinter_manager.db.insert_user_graph(user_account_id, user_contact_id, graph_id)

```

```

        # Creates an entry for the graph in the User Graph table
    except Exception as e:
        print("Failed to save graph to database:", e)
        pass

    font_saved = pg.font.Font(None,
80)

    # Sets the font for the saved text
    text_saved = font_saved.render("Graph Saved!", True, (0, 200,
0))

    # Creates a green Graph Saved message
    shadow_saved = font_saved.render("Graph Saved!", True, (0, 0,
0))

    # Creates a black shadow effect on the Graph Saved message
    save_text_rect = text_saved.get_rect(center=(GRID_WIDTH // 2, GRID_HEIGHT // 2 +
200))

    # Centres the text and shifts it slightly down
    shadow_save_rect = shadow_saved.get_rect(center=(GRID_WIDTH // 2 + 3, GRID_HEIGHT // 2 +
203))

    # The shadow is slightly off center to create the shadow effect
    display_surface.blit(shadow_saved,
shadow_save_rect)

    # Draws the shadow
    display_surface.blit(text_saved,
save_text_rect)

    # Draws the Graph Saved message
    pg.display.flip()

    # Updates the pygame display
    plt.close(latest_figure)

```

```

# Closes the figure to save memory space
latest_figure =

None

# Resets the latest figure

elif toggle_button_rect.collidepoint(event.pos):
    show_real_data = not show_real_data
    # Flips the show_real_data so only the curve is shown, or the real data is displayed again
    graph_surface, latest_figure = graph_manager.make_fitted_plot(pop_time, pop_pre,
pop_pred, fitted_params, show_real_data=show_real_data)
    # Gets the graph
    display_surface.blit(graph_surface, (1000, 400))
    # Draws the graph again
    font_parameters = pg.font.SysFont(None, 24)
    # Sets the font for the parameters
    text = font_parameters.render(f" $\alpha$ ={alpha_fit:.2f}  $\beta$ ={beta_fit:.2f}  $\delta$ ={delta_fit:.2f}
 $\gamma$ ={gamma_fit:.2f}", True, (0, 0, 0))
    # Displays the parameters onscreen
    display_surface.blit(text, (1100, 400))
    # Draws the parameters
    pg.display.update()
    # Updates the pygame display

if simulation_finished:
    # If the simulation has finished

    font_finished = pg.font.Font(None, 150)
    # Sets the font for the finished text
    text_finished = font_finished.render("Finished!", True, (255, 0, 0))
    # Creates a Finished message
    shadow_text_finished = font_finished.render("Finished!", True, (0, 0, 0))

```

255)

```

# Creates a black shadow effect on the Finished message
text_rect = text_finished.get_rect(center=(GRID_WIDTH // 2, GRID_HEIGHT // 2))
# Centres the Finished message
shadow_rect = shadow_text_finished.get_rect(center=(GRID_WIDTH // 2 + 5, GRID_HEIGHT // 2 + 5))
# The shadow is slightly off center to create the shadow effect
display_surface.blit(shadow_text_finished, shadow_rect)
# Draws the shadow
display_surface.blit(text_finished, text_rect)
# Draws the Finished message

save_button_rect = pg.Rect((GRID_WIDTH - 160) // 2, GRID_HEIGHT // 2 + 80, 160, 50)
# Centres and then moves the button slightly down
save_button_color = (65, 105, 225) if save_button_rect.collidepoint(pg.mouse.get_pos()) else (30, 144,
255)

# Changes the colour if the mouse hovers over the button

pg.draw.rect(display_surface, save_button_color, save_button_rect, border_radius=12)
# Draws the Rect of the button
font_button = pg.font.SysFont(None, 28, bold=True)
# Sets the font for the Save Button text
save_button_text = font_button.render("Save Graph", True, (255, 255, 255))
# Creates the Save Button text for the button
text_button_rect = save_button_text.get_rect(center=save_button_rect.center)
# Centres the text inside the Save Button Rect
display_surface.blit(save_button_text, text_button_rect)
# Draws the button and text

toggle_button_rect = pg.Rect(1100, 700, 180, 40)
# Defines the dimensions and size of the toggle button

```

```

toggle_button_color = (65, 105, 225) if toggle_button_rect.collidepoint(pg.mouse.get_pos()) else (30,
144, 255)

# Changes the colour if the mouse hovers over the button
toggle_text = "Hide Real Data" if show_real_data else "Show Real Data"
# Displays the button text required depending on show_real_data
font_toggle_button = pg.font.SysFont(None, 24)
# Sets the font for the Toggle Button text
toggle_button_text = font_toggle_button.render(toggle_text, True, (255, 255, 255))
# Creates the Toggle Button text
pg.draw.rect(display_surface, toggle_button_color, toggle_button_rect, border_radius=6)
# Draws the button
display_surface.blit(toggle_button_text, (toggle_button_rect.x + 10, toggle_button_rect.y + 10))
# Draws the text inside the button

continue

# Stops the loop, so the buttons are only drawn once

if not paused and not simulation_finished:
    # While the simulation is running

    step_number += 1
    # Increases the time step by 1

    display_surface.blit(background_map, (0, 0))
    # Blits the background map onto the pygame Surface
    grid.draw(display_surface)
    # Draws the grid object
    food_sprites.draw(display_surface)
    # Draws the food object

    font_time = pg.font.Font(None, 36)

```

```

# Sets the font for the Time text
time_text = font_time.render(f"Time Left: {90 - elapsed_time.seconds} seconds", True, (0, 0, 0))
# Creates a countdown timer

time_text_bg = pg.Surface(time_text.get_size())
# Creates a new Surface object which is the size of the Time text
time_text_bg.fill((255, 255, 255))
# Fills the Surface as white
time_text_bg.set_alpha(128)
# Makes the Surface partially transparent
display_surface.blit(time_text_bg, (10, 10))
# Blits the new Surface onto the main Surface
display_surface.blit(time_text, (10, 10))
# Blits the Time text

reserved_positions = set()
# Creates a set to hold all the reserved positions
planned_moves = {}
# Creates a dictionary to hold all the planned moves

for prey in prey_sprites:
    # For all prey alive
    planned_pos = prey.get_planned_move(food_sprites, prey_sprites)
    # Gets the planned moves for each prey
    planned_moves[prey] = planned_pos
    # Creates a key-value pair for each prey in planned_moves

for prey in prey_sprites:
    # For all prey alive
    prey.apply_move(reserved_positions)
    # Apply the moves

```

```

prey.eat(food_sprites.sprites())
# Allow prey which collide with food to eat
prey.check_dead()
# Checks if any prey are dead

if prey.energy >= prey.reproduction_threshold:
    # Checks if any prey have reached the reproduction threshold
    prey.energy -= prey.reproduction_cost
    # Applies the reproduction cost
    new_pre = prey.reproduce()
    # Prey reproduce
    prey_sprites.add(new_pre)
    # Adds new prey to the prey sprites Group

prey_sprites.draw(display_surface)
# Draws the prey

for predator in predator_sprites:
    # For all predators alive
    planned_pos = predator.get_planned_move(prey_sprites, predator_sprites)
    # Gets the planned moves for each predator
    planned_moves[predator] = planned_pos
    # Creates a key-value pair for each predator in planned_moves

for predator in predator_sprites:
    # For all predators alive
    predator.apply_move(reserved_positions)
    # Apply the moves
    predator.eat(prey_sprites.sprites())
    # Allow predators which collide with prey to eat
    predator.check_dead()

```

```

# Checks if any predators are dead

if predator.energy >= predator.reproduction_threshold:
    # Checks if any predators have reached the reproduction threshold
    predator.energy -= predator.reproduction_cost
    # Applies the reproduction cost
    new_pred = predator.reproduce()
    # Predators reproduce
    predator_sprites.add(new_pred)
    # Adds new predators to the predator sprites Group

predator_sprites.draw(display_surface)
# Draws the predators

for prey in list(pre_sprites):
    # For all prey
    if prey.energy <= 0:
        # If prey are dead
        prey_sprites.remove(prey)
        # Remove dead prey

for predator in list(predator_sprites):
    # For all predators
    if predator.energy <= 0:
        # If predators are dead
        predator_sprites.remove(predator)
        # Remove dead predators

pop_pre.append(len(list(pre_sprites)))
# Adds the number of prey to pop_pre
pop_pred.append(len(list(predator_sprites)))

```

```

# Adds the number of predators to pop_pred
pop_time.append(step_number)
# Adds the time step number to pop_time

if step_number % DISCRETE_STEP == 0 and len(pop_time) > 20:
    # If there are more than 20 time data points and 10 steps have passed
    fitted_params = graph_manager.fit_lv_model(pop_time, pop_pre, pop_pred)
    # Fits the Lotka Volterra model to an estimate and gets parameters

if len(pop_time) > 20:
    # If there are more than 20 time data points
    graph_surface, latest_figure = graph_manager.make_fitted_plot(pop_time, pop_pre, pop_pred,
fitted_params)

    # Plots the results
    display_surface.blit(graph_surface, (1000, 400))
    # Draws the graph

if fitted_params is not None:
    # If the parameters have been initialised
    alpha_fit, beta_fit, delta_fit, gamma_fit = fitted_params
    # Updates the parameters

    text_bg_rect = pg.Rect(1100, 400, 300, 30)
    # Creates the background Rect for the parameters
    pg.draw.rect(display_surface, (255, 255, 255), text_bg_rect)
    # Draws the background Rect

    font_parameters = pg.font.SysFont(None,
24)

    # Sets the font for the parameters

```

```

        text = font_parameters.render(f" $\alpha$ ={alpha_fit:.2f}  $\beta$ ={beta_fit:.2f}  $\delta$ ={delta_fit:.2f}
 $\gamma$ ={gamma_fit:.2f}", True, (0, 0, 0))
        # Displaus the paramaters on screen
        display_surface.blit(text, (1100,
400))

        # Draws the parameters
    else:
        text_bg_rect = pg.Rect(1200, 300, 300, 30)
        # Draws the background Rect regardless
        pg.draw.rect(display_surface, (255, 255, 255), text_bg_rect)

    if current_ticks - last_spawn_time > spawn_interval:
        # If it has been 10 seconds
        for _ in range(10):
            food_sprites.add(Food(grid))
            # Adds 10 food to the grid

        last_spawn_time = current_ticks
        # Resets the last spawn time

    if elapsed_time >= timedelta(seconds=90) or len(food_sprites) == 0 or len(preying_sprites) == 0:
        # If the time has run out, or if the number of food/prey reaches zero
        simulation_finished = True
        # Finishes the simulation
        continue

    pg.display.flip()
    # Updates the pygame display
    clock.tick(tkinter_manager.get_speed())
    # Sets the speed of the simulation to what the user specified

```

```

pg.QUIT
# Closes the pygame window

if __name__ == "__main__":
    # Instead of running it directly, I use this main guard
    main()

```

3.1.2. simulationManager.py

```

import numpy as np
# Importing numpy for vector features
import pygame as pg
# Importing pygame for simulation features
from pygame import *
# Making sure all of pygame is imported
import random
# Importing the random library

HEIGHT = 1000
# pygame Window Height
WIDTH = 1600
# pygame Window Width

GRID_HEIGHT = 1000
# Grid Height

```

```

GRID_WIDTH = 1000
# Grid Width
CELL_SIZE = 10
# Grid cell size

SPEED_RANGE_MODIFIER = 3
# Determines how far creatures can move in one step
SENSE_RANGE_MODIFIER = 5
# Determines how far creatures can detect food/prey

def load_image(image_name, X, Y):
    # This functions takes the image file name, desired width and desired height of the image
    image = pg.image.load(image_name)
    # This loads the image file into the variable image
    image = pg.transform.scale(image, (X,Y))
    # This command scales the image according to the input width and height
    return image
# The scaled image is returned

background_map = load_image("map.png", GRID_WIDTH, GRID_HEIGHT)
# This block loads all the required images and stores them in variables to be referenced
food_image = load_image("food.png", CELL_SIZE, CELL_SIZE)
simple_preys_image = load_image("prey.png", CELL_SIZE, CELL_SIZE)
simple_predator_image = load_image("predator.png", CELL_SIZE, CELL_SIZE)
wolf_image = load_image("wolf.png", CELL_SIZE, CELL_SIZE)
deer_image = load_image("deer.png", CELL_SIZE, CELL_SIZE)

class Grid():
    def __init__(self, width, height, cell_size):
        #This method initialises the Grid according to the width, height and cell_size passed in

```

```

self.columns = width // cell_size
self.rows = height // cell_size
self.cell_size = cell_size
self.occupied_positions = set()
# Stores all creature and food positions and forces them to be unique
self.prey_positions = set()
# Stores all prey positions
self.predator_positions = set()
# Stores all predator positions
self.cells = [[pg.Rect(col * cell_size, row * cell_size, cell_size, cell_size) for col in range(self.columns)]
for row in range(self.rows)]
# Stores every cell in the Grid

def draw(self, surface):
# This method draws the grid onto a specified pygame Surface
grid_colour = (220,220,220)
# Light grey colour
for row in self.cells:
    for cell in row:
        pg.draw.rect(surface, grid_colour, cell, 1)

class Food(pg.sprite.Sprite):
def __init__(self,grid):
# Initialises the food
pg.sprite.Sprite.__init__(self)
self.energy = 20
# Each piece of food will restore prey with 20 energy
self.image = food_image
# Passing in the image for food
self.rect= self.image.get_rect()
# This creates a Rect object, so the coordinates of the food are able to be tracked

```

```

while True:
    # This snippet runs an infinite loop until a unique position is found for a food object to avoid stacking
    row = np.random.randint(0, grid.rows)
    # This gets the grid coordinate for the row the food occupies
    col = np.random.randint(0, grid.columns)
    # This gets the grid coordinate for the column the food occupies
    if (row, col) not in grid.occupied_positions:
        break

self.pos = np.array([row,col], dtype = int)
# This creates a numpy array to track the food as a position vector
self.rect.topleft = (col * grid.cell_size, row * grid.cell_size)
# Sets the position of the food sprite to the grid using pixel coordinates on the actual pygame Surface

class Creature(pg.sprite.Sprite):
    def __init__(self, grid):
        pg.sprite.Sprite.__init__(self)
        self.speed_range = CELL_SIZE * SPEED_RANGE_MODIFIER
        # This ensures the creatures can only move in discrete steps which match the cell size
        self.sense_range = CELL_SIZE * SENSE_RANGE_MODIFIER
        # This ensures the creatures can only sense in discrete steps which match the cell size
        self.energy = 100
        # This sets the starting energy of all creatures
        self.grid = grid

    while True:
        # This snippet runs an infinite loop until a unique position is found for a creature object to avoid
        stacking
        row = np.random.randint(0, grid.rows)
        # This gets the grid coordinate for the row the creature occupies

```

```

col = np.random.randint(0, grid.columns)
# This gets the grid coordinate for the column the creature occupies
pos = (row * CELL_SIZE, col * CELL_SIZE)

if pos not in grid.occupied_positions:
    self.pos = np.array(pos, dtype=float)
    # This creates a numpy array to track the creature as a position vector
    self.grid.occupied_positions.add(pos)
    break

self.rect = pg.Rect(self.pos[0], self.pos[1], CELL_SIZE, CELL_SIZE)
# This creates a Rect object, so the coordinates of the creature are able to be tracked

def get_planned_move(self):
    #This method is initialised but will be overridden. It returns the move the creature is going to do.
    pass

def apply_move(self):
    #This method is initialised but will be overridden. It applies the planned move
    pass

def eat(self):
    #This method is initialised but will be overridden. It allows the creature to eat
    pass

def reproduce(self):
    #This method is initialised but will be overridden. It creates a new instance of Creature
    new_creature = Creature(self.grid)
    new_creature.rect.center = self.rect.center
    return new_creature

```

```
def check_dead(self):
    #This method is initialised but will be overridden. It checks if creatures are dead.
    if self.energy == 0:
        self.grid.occupied_positions.discard(tuple(self.pos.astype(int)))
        self.kill()
```

```
class Prey(Creature):
```

```
def __init__(self, grid):
    super().__init__(grid)
    self.image = simple_preimage
    # Passing in the image for simple prey
    self.rect = self.image.get_rect()
    self.rect.topleft = self.pos.copy()
    self.known_food_locations = []
    # Creates a list of known food locations for prey
    self.reproduction_threshold = 120
    # Sets the energy required to reproduce
    self.reproduction_cost = 20
    # Sets the energy lost during reproduction

def communicate(self, all_pre):
    # Creates a shared memory between the prey objects of food, so that all prey knows when food is discovered
    all_food_locations = set(self.known_food_locations)
    # Creates a set of food locations the collective prey know about
    for prey in all_pre:
        # For each individual prey in the list
        all_food_locations.update(prey.known_food_locations)
```

```

        # Adds the individual prey's known food locations to the set of all food locations the prey know about
self.known_food_locations = list(all_food_locations)
# Updates each prey's known food locations to include every known food location to the collective

def get_planned_move(self, food_list, all_pre):
    # Function to obtain the planned move of the prey

    # self.communicate(all_pre)
    # Simple prey do not communicate

    target_pos = None
    # Initialises the target position to be returned

    if food_list:
        # If there is food in the food list,
        target_positions = np.array([food.rect.topleft for food in food_list])
        # Initialise a numpy array of positions vectors of each food in the food list
        distances = np.sum((target_positions - self.pos) ** 2, axis=1)
        # Creates a numpy array of squared distances from the target food
        valid_indices = np.where(distances <= self.sense_range ** 2)[0]
        # Filters so that only the food in the sense_range is considered

        if valid_indices.size > 0:
            # Checks if the array of visible food is not empty
            distances = distances[valid_indices]
            # Redefines distances to only include visible food
            target_positions = target_positions[valid_indices]
            # Redefines the target positions to only include visible food positions
            closest_target_index = np.argmin(distances)
            # Finds the index of the food at the smallest distance from the creature
            target_pos = target_positions[closest_target_index]

```

```

        # Narrows down the target positions to just the closest food index
        self.known_food_locations.append(tuple(target_pos))
        # Adds the target position of the closest food to known food_locations

move_options = []
# Initialises a list of move options
for i in range(1, (self.speed_range // CELL_SIZE) + 1):
    # This snippet of code creates the options for moving according to the creature's speed range
    step = i * CELL_SIZE
    # Each step is in discrete multiples of the cell size
    move_options.extend([
        self.pos + np.array([-step, 0]),
        # Left
        self.pos + np.array([step, 0]),
        # Right
        self.pos + np.array([0, -step]),
        # Up
        self.pos + np.array([0, step]),
        # Down
        self.pos + np.array([-step, -step]),
        # Left and Up
        self.pos + np.array([-step, step]),
        # Left and Down
        self.pos + np.array([step, -step]),
        # Right and Up
        self.pos + np.array([step, step])
        # Right and Down
    ])

valid_moves = [
    move for move in move_options

```

```

        if 0 <= move[0] < self.grid.rows * CELL_SIZE and
            0 <= move[1] < self.grid.columns * CELL_SIZE
    ]
    # Creates an array of valid move options
    # Makes sure that the move stays within the borders of the grid

    if valid_moves:
        # If the array valid moves is not empty
        if target_pos is not None:
            # If the creature has a target food
            best_move = min(valid_moves, key=lambda move: (np.linalg.norm(target_pos - move) +
random.uniform(0.01, 0.1)))
            # Returns the move with the minimum distance, with a random element to break any tie
        else:
            best_move = random.choice(valid_moves)
            # If there is no target food, choose a random valid move
        self.planned_pos = best_move
        # Set the planned position to the best ascertained move
    else:
        self.planned_pos = self.pos
        # If there are no valid moves, stay still

    return tuple(self.planned_pos.astype(int))
# Returns an integer tuple of the planned position to move to

def apply_move(self, reserved_positions):
    # This method applies planned moves if they are not already taken
    old_pos = tuple(self.pos.astype(int))
    # This creates an integer tuple of the prey's current position
    new_pos = tuple(self.planned_pos.astype(int))
    # This creates an integer tuple of the planned position to move to

```

```

if new_pos not in reserved_positions:
    # This snippet updates the position of the prey if the move is not already taken
    reserved_positions.add(new_pos)
    self.grid.occupied_positions.discard(old_pos)
    self.grid.prey_positions.discard(old_pos)
    self.grid.occupied_positions.add(new_pos)
    self.grid.prey_positions.add(new_pos)
    self.pos = np.array(new_pos)
    self.rect.topleft = new_pos
else:
    reserved_positions.add(old_pos)
    # If the move is taken, stay still to avoid collisions

self.energy -= 2
# Reduces the prey's energy by 2 for every movement

def eat(self, food_list):
    # This method allows prey to eat
    for food in food_list:
        # This snippet allows prey to eat if they land on a position with food
        food_pos = np.array(food.rect.topleft)
        food_pos = np.round(food_pos / CELL_SIZE) * CELL_SIZE
        if np.linalg.norm(self.pos - food_pos) < CELL_SIZE:
            self.energy += food.energy
            # The prey gains energy from food
            food.kill()
            # This removes the food sprite once eaten
            pg.display.update()
            break
    # Only one food can be ate at a time

```

```

def reproduce(self):
    # This method allows prey to reproduce
    new_pre = Prey(self.grid)
    # Creates a new instance of Prey
    new_pre.rect.center = self.rect.center
    # Sets the position close to itself
    return new_pre

def check_dead(self):
    # This method checks if a prey is dead
    if self.energy <= 0:
        # If the energy of prey is 0 or lower
        self.grid.occupied_positions.discard(tuple(self.pos.astype(int)))
        # The prey is removed from the lists of positions, it belonged in
        self.grid.prey_positions.discard(tuple(self.pos.astype(int)))
        self.kill()
        # The prey sprite is killed

```

```

class Predator(Creature):
    def __init__(self, grid):
        super().__init__(grid)
        self.image = simple_predator_image
        # Passing in the image for simple predators
        self.rect = self.image.get_rect()
        self.rect.topleft = self.pos.copy()
        self.known_pre_locations = []
        # Creates a list of known prey locations for predators
        self.reproduction_threshold = 190
        # Sets the energy required to reproduce

```

```

self.reproduction_cost = 80
# Sets the energy lost during reproduction

def communicate(self, predators):
    # Creates a shared memory between the predator objects of prey, so that all predators know when prey is
    discovered
    all_pre_y_locations = set(self.known_pre_y_locations)
    # Creates a set of prey locations the collective predators know about
    for predator in predators:
        # For each individual predator in the list
        all_pre_y_locations.update(predator.known_pre_y_locations)
        # Adds the individual predator's known food locations to the set of all food locations the predators know
        about
    self.known_pre_y_locations = list(all_pre_y_locations)
    # Updates each predator's known food locations to include every known food location to the collective

def get_planned_move(self, prey_list, all_predators):
    # Function to obtain the planned move of the predator

    # self.communicate(all_predators)
    # Simple predators do not communicate

    target_pos = None

    if prey_list:
        target_positions = np.array([prey.rect.topleft for prey in prey_list])
        distances = np.sum((target_positions - self.pos) ** 2, axis=1)
        valid_indices = np.where(distances <= self.sense_range ** 2)[0]

        if valid_indices.size > 0:
            distances = distances[valid_indices]

```

```

        target_positions = target_positions[valid_indices]
        closest_target_index = np.argmin(distances)
        target_pos = target_positions[closest_target_index]
        self.known_preys_locations.append(tuple(target_pos))

    move_options = []
    for i in range(1, (self.speed_range // CELL_SIZE) + 1):
        step = i * CELL_SIZE
        move_options.extend([
            self.pos + np.array([-step, 0]),
            self.pos + np.array([step, 0]),
            self.pos + np.array([0, -step]),
            self.pos + np.array([0, step]),
            self.pos + np.array([-step, -step]),
            self.pos + np.array([-step, step]),
            self.pos + np.array([step, -step]),
            self.pos + np.array([step, step])
        ])

    valid_moves = [
        move for move in move_options
        if 0 <= move[0] < self.grid.rows * CELL_SIZE and
        0 <= move[1] < self.grid.columns * CELL_SIZE
    ]

    if valid_moves:
        if target_pos is not None:
            best_move = min(valid_moves, key=lambda move: (np.linalg.norm(target_pos - move) +
random.uniform(0.01, 0.1)))
        else:
            best_move = random.choice(valid_moves)

```

```

        self.planned_pos = best_move
    else:
        self.planned_pos = self.pos

    return tuple(self.planned_pos.astype(int))

def apply_move(self, reserved_positions):
    old_pos = tuple(self.pos.astype(int))
    new_pos = tuple(self.planned_pos.astype(int))

    if new_pos not in reserved_positions:
        reserved_positions.add(new_pos)
        self.grid.occupied_positions.discard(old_pos)
        self.grid.predator_positions.discard(old_pos)
        self.grid.occupied_positions.add(new_pos)
        self.grid.predator_positions.add(new_pos)
        self.pos = np.array(new_pos)
        self.rect.topleft = new_pos
    else:
        reserved_positions.add(old_pos)

    self.energy -= 4
    # Reduces the predator's energy by 4 for every movement

def eat(self, prey_list):
    # This method allows predators to eat
    for prey in prey_list:
        prey_pos = np.array(prey.rect.topleft)
        prey_pos = np.round(prey_pos / CELL_SIZE) * CELL_SIZE
        if np.linalg.norm(self.pos - prey_pos) <= CELL_SIZE:
            self.energy += 0.88 * prey.energy

```

```

        # Predators gain a percentage of the energy of prey
        prey.kill()
        break

def reproduce(self):
    new_predator = Predator(self.grid)
    # Creates a new instance of Predator
    new_predator.rect.center = self.rect.center
    return new_predator

def check_dead(self):
    if self.energy <= 0:
        self.grid.occupied_positions.discard(tuple(self.pos.astype(int)))
        self.grid.predator_positions.discard(tuple(self.pos.astype(int)))
        self.kill()

class Deer(Prey):
    def __init__(self, grid):
        super().__init__(grid)
        self.image = deer_image
        # Passing in the image for deer

    def get_planned_move(self, food_list, all_preys):

        self.communicate(all_preys)
        # Deers can communicate

        target_pos = None

        if food_list:

```

```

target_positions = np.array([food.rect.topleft for food in food_list])
distances = np.sum((target_positions - self.pos) ** 2, axis=1)
valid_indices = np.where(distances <= self.sense_range ** 2)[0]

if valid_indices.size > 0:
    distances = distances[valid_indices]
    target_positions = target_positions[valid_indices]
    closest_target_index = np.argmin(distances)
    target_pos = target_positions[closest_target_index]
    self.known_food_locations.append(tuple(target_pos))

move_options = []
for i in range(1, (self.speed_range // CELL_SIZE) + 1):
    step = i * CELL_SIZE
    move_options.extend([
        self.pos + np.array([-step, 0]),
        self.pos + np.array([step, 0]),
        self.pos + np.array([0, -step]),
        self.pos + np.array([0, step]),
        self.pos + np.array([-step, -step]),
        self.pos + np.array([-step, step]),
        self.pos + np.array([step, -step]),
        self.pos + np.array([step, step])
    ])

valid_moves = [
    move for move in move_options
    if 0 <= move[0] < self.grid.rows * CELL_SIZE and
        0 <= move[1] < self.grid.columns * CELL_SIZE
]

```

```

    if valid_moves:
        if target_pos is not None:
            best_move = min(valid_moves, key=lambda move: (np.linalg.norm(target_pos - move) +
random.uniform(0.01, 0.1)))
        else:
            best_move = random.choice(valid_moves)
            self.planned_pos = best_move
    else:
        self.planned_pos = self.pos # Stay still

    return tuple(self.planned_pos.astype(int))

def apply_move(self, reserved_positions):
    old_pos = tuple(self.pos.astype(int))
    new_pos = tuple(self.planned_pos.astype(int))

    if new_pos not in reserved_positions:
        reserved_positions.add(new_pos)
        self.grid.occupied_positions.discard(old_pos)
        self.grid.prey_positions.discard(old_pos)
        self.grid.occupied_positions.add(new_pos)
        self.grid.prey_positions.add(new_pos)
        self.pos = np.array(new_pos)
        self.rect.topleft = new_pos
    else:
        reserved_positions.add(old_pos)

    self.energy -= 2

def reproduce(self):
    new_prey = Deer(self.grid)

```

```

        # Creates a new instance of Deer
        new_prej.rect.center = self.rect.center
        return new_prej

class Wolf(Predator):
    def __init__(self, grid):
        super().__init__(grid)
        self.image = wolf_image
        # Passing in the image for wolves

    def get_planned_move(self, prey_list, all_predators):

        self.communicate(all_predators)
        # Wolves can communicate

        target_pos = None

        if prey_list:
            target_positions = np.array([prej.rect.topleft for prej in prey_list])
            distances = np.sum((target_positions - self.pos) ** 2, axis=1)
            valid_indices = np.where(distances <= self.sense_range ** 2)[0]

            if valid_indices.size > 0:
                distances = distances[valid_indices]
                target_positions = target_positions[valid_indices]
                closest_target_index = np.argmin(distances)
                target_pos = target_positions[closest_target_index]
                self.known_prej_locations.append(tuple(target_pos))

        move_options = []
        for i in range(1, (self.speed_range // CELL_SIZE) + 1):

```

```

        step = i * CELL_SIZE
        move_options.extend([
            self.pos + np.array([-step, 0]),
            self.pos + np.array([step, 0]),
            self.pos + np.array([0, -step]),
            self.pos + np.array([0, step]),
            self.pos + np.array([-step, -step]),
            self.pos + np.array([-step, step]),
            self.pos + np.array([step, -step]),
            self.pos + np.array([step, step])
        ])

    valid_moves = [
        move for move in move_options
        if 0 <= move[0] < self.grid.rows * CELL_SIZE and
        0 <= move[1] < self.grid.columns * CELL_SIZE
    ]

    if valid_moves:
        if target_pos is not None:
            best_move = min(valid_moves, key=lambda move: (np.linalg.norm(target_pos - move) +
random.uniform(0.01, 0.1)))
        else:
            best_move = random.choice(valid_moves)
        self.planned_pos = best_move
    else:
        self.planned_pos = self.pos

    return tuple(self.planned_pos.astype(int))

def apply_move(self, reserved_positions):

```

```

old_pos = tuple(self.pos.astype(int))
new_pos = tuple(self.planned_pos.astype(int))

if new_pos not in reserved_positions:
    reserved_positions.add(new_pos)
    self.grid.occupied_positions.discard(old_pos)
    self.grid.predator_positions.discard(old_pos)
    self.grid.occupied_positions.add(new_pos)
    self.grid.predator_positions.add(new_pos)
    self.pos = np.array(new_pos)
    self.rect.topleft = new_pos
else:
    reserved_positions.add(old_pos)

self.energy -= 4

def reproduce(self):
    new_predator = Wolf(self.grid)
    # Creates a new instance of Wolf
    new_predator.rect.center = self.rect.center
    return new_predator

```

3.1.3. tkManager.py

```

import tkinter as tk
# Importing tkinter for GUI features
from tkinter import ttk
# Importing extra tkinter features

```

```

import time
# Importing the time library
from emailManager import EmailManager
# Importing the user-defined EmailManager class to handle email-related features
from dbManager import DBManager
# Importing the user-defined DBManager class to handle database-related features

class TkinterManager():

    def __init__(self):
        self.db = DBManager()
        # Creates an instance of DBManager
        self.email_manager = EmailManager()
        # Creates an instance of EmailManager
        self._verified = False
        # Checker to make sure user has logged in properly
        self._settingsconfirmed = False
        # Checker to make sure settings were confirmed before running simulation
        self._speed = 0
        self._foodnum = 0
        self._simplepreynum = 0
        self._simplepredatornum = 0
        self._wolfnum = 0
        self._deernum = 0
        self._email = ""
        self._username = ""
        self.selected_options = []
        self.otp_counter = 4
        # Sets number of tries for user to get the OTP right
        self.email_counter = 4
        # Sets number of tries for user to get a valid email

```

```

self.db.create_tables()
# Creates database tables on device locally
self.open_welcome_page()
# Opens welcome page

def open_welcome_page(self):
    # Welcomes user to the simulation, giving them the option to log in or make a new account

    self.welcome_page = tk.Tk()
    self.welcome_page.title("Welcome!")
    self.welcome_page.geometry("400x400")
    self.centre_page(self.welcome_page)
    self.welcome_page.iconbitmap("Icon.ico")
    # Loads in the icon file for the window
    welcome_message = tk.Label(self.welcome_page, text = "Welcome to the simulation!")
    welcome_message.pack()
    login_choice = tk.Button(self.welcome_page, text = "Login", width = 30, command = self.open_login_page)
    login_choice.pack()
    new_account_choice = tk.Button(self.welcome_page, text = "Make New Account", width = 30, command =
self.open_account_creation_page)
    new_account_choice.pack()
    self.welcome_page.mainloop()

def centre_page(self,window):
    # This function centres the window on the user's screen
    window.update_idletasks()
    window_width = window.winfo_width()
    window_height = window.winfo_height()
    userscreen_width = window.winfo_screenwidth()
    userscreen_height = window.winfo_screenheight()

```

```

x = (userscreen_width - window_width) // 2
y = (userscreen_height - window_height) // 2
window.geometry(f"{window_width}x{window_height}+{x}+{y}")

def open_settings_page(self):
    # Opens the settings page where users can select the speed, number of food and number of creatures

    self.creature_selection_page.destroy()

    self.settings_page = tk.Tk()
    self.settings_page.title("Settings")
    self.settings_page.geometry("400x400")
    self.centre_page(self.settings_page)
    self.settings_page.iconbitmap("Icon.ico")
    settings_message = tk.Label(self.settings_page, text = "Change Settings here.")
    settings_message.pack()
    simulation_speed_label = tk.Label(self.settings_page, text = "Simulation Speed")
    simulation_speed_label.pack()
    simulation_speed_choice = ttk.Spinbox(self.settings_page, from_=1, to=10)
    # Ensures the choice is only between 1 and 10
    simulation_speed_choice.pack()
    foodnum_label = tk.Label(self.settings_page, text = "Number of Food")
    foodnum_label.pack()
    foodnum_choice = ttk.Spinbox(self.settings_page, from_=1, to=100)
    # Ensures the choice is only between 1 and 100
    foodnum_choice.pack()

    if "1" in self.selected_options:
        # If the user has selected simple predators, add this option
        simple_predatornum_label = tk.Label(self.settings_page, text = "Number of Predators")
        simple_predatornum_label.pack()

```

```

simple_predatornum_choice = ttk.Spinbox(self.settings_page, from_=1, to=25)
# Ensures the choice is only between 1 and 25
simple_predatornum_choice.pack()

if "2" in self.selected_options:
    # If the user has selected simple prey, add this option
    simple_preynum_label = tk.Label(self.settings_page, text = "Number of Prey")
    simple_preynum_label.pack()
    simple_preynum_choice = ttk.Spinbox(self.settings_page, from_=1, to=25)
    # Ensures the choice is only between 1 and 25
    simple_preynum_choice.pack()

if "3" in self.selected_options:
    wolfnum_label = tk.Label(self.settings_page, text = "Number of Wolves")
    # If the user has selected wolves, add this option
    wolfnum_label.pack()
    wolfnum_choice = ttk.Spinbox(self.settings_page, from_=1, to=25)
    # Ensures the choice is only between 1 and 25
    wolfnum_choice.pack()

if "4" in self.selected_options:
    deernum_label = tk.Label(self.settings_page, text = "Number of Deer")
    # If the user has selected deer, add this option
    deernum_label.pack()
    deernum_choice = ttk.Spinbox(self.settings_page, from_=1, to=25)
    # Ensures the choice is only between 1 and 25
    deernum_choice.pack()

def confirm_settings():
    # Confirms settings are valid

```

```

boolean_queue_list_checker = [False for _ in range(len(self.selected_options) + 2)]
# Populates a boolean queue list with False

try:
    if int(simulation_speed_choice.get()) <= 10 and int(simulation_speed_choice.get()) >= 1:
        self._speed = float(simulation_speed_choice.get())
        boolean_queue_list_checker.pop(0)
        # If the choice is valid, dequeues a False from the list
        boolean_queue_list_checker.append(True)
        # Enqueues a True

    elif int(simulation_speed_choice.get()) > 10 or int(simulation_speed_choice.get()) < 1:
        settings_status.configure(text = "Settings are invalid.", fg = "red")
        boolean_queue_list_checker = [False for _ in range(len(self.selected_options) + 2)]
        # If the choice is invalid, the queue is reset

except:
    pass

try:
    if int(foodnum_choice.get()) <= 150 and int(foodnum_choice.get()) >= 1:
        self._foodnum = int(foodnum_choice.get())
        boolean_queue_list_checker.pop(0)
        # If the choice is valid, dequeues a False from the list
        boolean_queue_list_checker.append(True)
        # Enqueues a True

    elif int(foodnum_choice.get()) > 150 or int(foodnum_choice.get()) < 1:
        settings_status.configure(text = "Settings are invalid.", fg = "red")
        boolean_queue_list_checker = [False for _ in range(len(self.selected_options) + 2)]
        # If the choice is invalid, the queue is reset

```

```

except:
    pass

try:
    if int(simple_predatornum_choice.get()) <= 50 and int(simple_predatornum_choice.get()) >= 1:
        self._simplepredatornum = int(simple_predatornum_choice.get())
        boolean_queue_list_checker.pop(0)
        # If the choice is valid, dequeues a False from the list
        boolean_queue_list_checker.append(True)
        # Enqueues a True

    elif int(simple_predatornum_choice.get()) > 50 or int(simple_predatornum_choice.get()) < 1:
        settings_status.configure(text = "Settings are invalid.", fg = "red")
        boolean_queue_list_checker = [False for _ in range(len(self.selected_options) + 2)]
        # If the choice is invalid, the queue is reset

except:
    pass

try:
    if int(simple_preynum_choice.get()) <= 50 and int(simple_preynum_choice.get()) >= 1:
        self._simplepreynum = int(simple_preynum_choice.get())
        boolean_queue_list_checker.pop(0)
        # If the choice is valid, dequeues a False from the list
        boolean_queue_list_checker.append(True)
        # Enqueues a True

    elif int(simple_preynum_choice.get()) > 50 or int(simple_preynum_choice.get()) < 1:
        settings_status.configure(text = "Settings are invalid.", fg = "red")
        boolean_queue_list_checker = [False for _ in range(len(self.selected_options) + 2)]

```

```

        # If the choice is invalid, the queue is reset

except:
    pass

try:
    if int(wolfnum_choice.get()) <= 50 and int(wolfnum_choice.get()) >= 1:
        self._wolfnum = int(wolfnum_choice.get())
        boolean_queue_list_checker.pop(0)
        # If the choice is valid, dequeues a False from the list
        boolean_queue_list_checker.append(True)
        # Enqueues a True

    elif int(wolfnum_choice.get()) > 50 or int(wolfnum_choice.get()) < 1:
        settings_status.configure(text = "Settings are invalid.", fg = "red")
        boolean_queue_list_checker = [False for _ in range(len(self.selected_options) + 2)]
        # If the choice is invalid, the queue is reset

except:
    pass

try:
    if int(deernum_choice.get()) <= 50 and int(deernum_choice.get()) >= 1:
        self._deernum = int(deernum_choice.get())
        boolean_queue_list_checker.pop(0)
        # If the choice is valid, dequeues a False from the list
        boolean_queue_list_checker.append(True)
        # Enqueues a True

    elif int(deernum_choice.get()) > 50 or int(deernum_choice.get()) < 1:
        settings_status.configure(text = "Settings are invalid.", fg = "red")

```

```

        boolean_queue_list_checker = [False for _ in range(len(self.selected_options) + 2)]
        # If the choice is invalid, the queue is reset

    except:
        pass

    if boolean_queue_list_checker == [True for _ in range(len(self.selected_options) + 2)]:
        # If all Falses have been dequeued and replaced with True values, settings are confirmed
        self.set_settings_confirmed(True)
        self.settings_page.destroy()

    settings_status = tk.Label(self.settings_page, text = "")
    settings_status.pack()
    confirm_settings_button = tk.Button(self.settings_page, text = "Confirm Settings", command = confirm_settings)
    confirm_settings_button.pack()

    self.settings_page.mainloop()

def open_creature_selection_page(self):
    # Opens the creature selection page, where users select the type of creatures included in the simulation

    self.explanation_page.destroy()

    self.creature_selection_page = tk.Tk()
    self.creature_selection_page.title("Creature Selection")
    self.creature_selection_page.geometry("800x400")
    self.centre_page(self.creature_selection_page)
    self.creature_selection_page.iconbitmap("Icon.ico")

    simple_predator_option = tk.BooleanVar()

```

```

simple_prey_option = tk.BooleanVar()
wolf_option = tk.BooleanVar()
deer_option = tk.BooleanVar()

def check_selections():
    # Creates a list of the selected choices
    self.selected_options = []
    if simple_predator_option.get() == True:
        self.selected_options.append("1")
    if simple_prey_option.get() == True:
        self.selected_options.append("2")
    if wolf_option.get() == True:
        self.selected_options.append("3")
    if deer_option.get() == True:
        self.selected_options.append("4")

    selection_message = tk.Label(self.creature_selection_page, text = "Please select the types of creatures you
would like:")
    selection_message.pack()

    simple_predator_choice = ttk.Checkbutton(self.creature_selection_page, text = "Simple Predators", variable=
simple_predator_option, command= check_selections)
    simple_predator_choice.pack()
    simple_predator_label = tk.Label(self.creature_selection_page, text="These are simple predators which search
for prey by moving randomly until they see prey")
    simple_predator_label.pack()

    simple_prey_choice = ttk.Checkbutton(self.creature_selection_page, text= "Simple Prey", variable=
simple_prey_option, command= check_selections)
    simple_prey_choice.pack()

```

```

        simple_preys_label = tk.Label(self.creature_selection_page, text="These are simple prey which search for food
by moving randomly until they see prey")
        simple_preys_label.pack()

        wolf_choice = ttk.Checkbutton(self.creature_selection_page, text= "Wolves", variable= wolf_option, command=
check_selections)
        wolf_choice.pack()
        wolf_label = tk.Label(self.creature_selection_page, text="These ferocious predators work together by sharing
the location of nearby prey with others")
        wolf_label.pack()

        deer_choice = ttk.Checkbutton(self.creature_selection_page, text= "Deer", variable= deer_option, command=
check_selections)
        deer_choice.pack()
        deer_label = tk.Label(self.creature_selection_page, text="These nimble beasts work together by sharing the
location of nearby food with others")
        deer_label.pack()

        close_button = tk.Button(self.creature_selection_page, text = "Move to Settings", command =
self.open_settings_page)
        close_button.pack()

        self.creature_selection_page.mainloop()

def open_graph_downloads_page(self):
    # Opens the graph download page for users to download zip files of their saved graphs

    self.explanation_page.destroy()

    self.graphs_downloads_page = tk.Tk()
    self.graphs_downloads_page.title("Graph Downloads")

```

```

self.graphs_downloads_page.geometry("400x400")
self.centre_page(self.graphs_downloads_page)
self.graphs_downloads_page.iconbitmap("Icon.ico")

welcome_message = tk.Label(self.graphs_downloads_page, text = "Download any previous graphs here")
welcome_message.pack()

def download_saved_graphs():
    # Downloads the graphs associated with the username
    username = self.get_username()
    if self.db.download_graphs(username):
        self.download_status.configure(text = "Success! Graphs downloaded!", fg = "green" )
    else:
        self.download_status.configure(text = "Error! Try again later", fg = "red" )

def return_to_welcome_page():
    # Returns the user to the welcome page
    self.graphs_downloads_page.destroy()
    self.open_welcome_page()

graph_download_button = tk.Button(self.graphs_downloads_page, text = "Download previous graphs", command =
download_saved_graphs)
graph_download_button.pack()
return_button = tk.Button(self.graphs_downloads_page, text = "Return to welcome page", command =
return_to_welcome_page)
return_button.pack()
self.download_status = tk.Label(self.graphs_downloads_page, text = "")
self.download_status.pack()

self.graphs_downloads_page.mainloop()

```

```

def open_explanation_page(self):
    # Opens a page to explain the simulation to users

    self.login_page.destroy()

    self.explanation_page = tk.Tk()
    self.explanation_page.title("Explanation Guide")
    self.explanation_page.geometry("800x400")
    self.centre_page(self.explanation_page)
    self.explanation_page.iconbitmap("Icon.ico")

    welcome_message = tk.Label(self.explanation_page, text = "Welcome to the simulation!")
    welcome_message.pack()

    message_line1 = tk.Label(self.explanation_page, text = "This program simulates predators and prey on a grid.")
    message_line1.pack()

    message_line2 = tk.Label(self.explanation_page, text = "The grid is 100 by 100 squares.")
    message_line2.pack()

    message_line3 = tk.Label(self.explanation_page, text = "Simulation Speed ranges from 1 to 10, Number of Food
ranges from 1 to 150, and Number of Creatures range from 1 to 50")
    message_line3.pack()

    message_line4 = tk.Label(self.explanation_page, text = "Food is generated at the start of the simulation at
random locations.")
    message_line4.pack()

    message_line5 = tk.Label(self.explanation_page, text = "Prey will move towards food it can see, and predators
will do the same for prey")

```

```

message_line5.pack()

message_line6 = tk.Label(self.explanation_page, text = "There are different types of predators and prey, each
with unique searching methods.")
message_line6.pack()

message_line7 = tk.Label(self.explanation_page, text = "Creatures are able to sense food in a 5 by 5 radius
around them")
message_line7.pack()

message_line8= tk.Label(self.explanation_page, text = "Graphs generated use the Lotka-Volterra equations on
the simulation. Values generated may not always match the expected pattern")
message_line8.pack()

message_line9 = tk.Label(self.explanation_page, text = "Alpha: Prey Birth Rate, Beta: Predation Rate, Gamma:
Predator Death Rate, Delta: Predation Efficiency")
message_line9.pack()

message_line10 = tk.Label(self.explanation_page, text = "Graphs are saved to the downloads folder. You can
also view older saved graphs through the account graphs page")
message_line10.pack()

graphs_button = tk.Button(self.explanation_page, text = "Move to Saved Graphs", command =
self.open_graph_downloads_page)
graphs_button.pack()
close_button = tk.Button(self.explanation_page, text = "Move to Creature Selection", command =
self.open_creature_selection_page)
close_button.pack()

self.explanation_page.mainloop()

```

```
def open_login_page(self):
    # Opens the log in page, requiring a username and password

    self.welcome_page.destroy()

    self.login_page = tk.Tk()
    self.login_page.title("Login")
    self.login_page.geometry("400x400")
    self.centre_page(self.login_page)
    self.login_page.iconbitmap("Icon.ico")

    login_message = tk.Label(self.login_page, text = "Please Log in")
    login_message.pack()

    username_label = tk.Label(self.login_page, text = "Username:")
    username_label.pack()
    username_entry = tk.Entry(self.login_page)
    username_entry.pack()

    password_label = tk.Label(self.login_page, text = "Password:")
    password_label.pack()
    password_entry = tk.Entry(self.login_page, show = "*")
    # Hides the password by showing only stars when typing
    password_entry.pack()

    def verify_login():
        # Verifies log in details
        username = username_entry.get()
        password = password_entry.get()
        self._verified = False
```

```

        if username == "admin" and password == "Password1":
            # Built in admin account details so users don't have to make an account (However, graphs can't be
            saved to this account)
            self.login_status.configure(text = "Success! Welcome Admin.", fg = "green")
            self.login_page.update_idletasks()
            time.sleep(10)
            self.set_verified(True)
            self.set_username(username)
            self.open_explanation_page()

        elif self.db.existing_username_check(username) and self.db.verify_user_password(username ,password):
            # If the details entered match a username and the correct password is entered, log in
            full_name = self.db.get_full_name_from_username(username)
            self.login_status.configure(text = f"Success! Welcome {full_name}.", fg = "green")
            # Welcomes user with their full name
            self.login_page.update_idletasks()
            time.sleep(10)
            self.set_verified(True)
            self.set_username(username)
            self.open_explanation_page()

        else:
            self.login_status.configure(text = "Login Failed.. Try Again.", fg = "red")
            # If the details entered are wrong, return an error message

login_button = tk.Button(self.login_page, text = "Login", command = verify_login)
login_button.pack()
self.login_status = tk.Label(self.login_page, text = "")
self.login_status.pack()

```

```

self.login_page.mainloop()

def open_account_details_page(self):
    # Opens the new account details entry form

    self.otp_page.destroy()

    self.account_details_page = tk.Tk()
    self.account_details_page.title("Account Details")
    self.account_details_page.geometry("400x400")
    self.centre_page(self.account_details_page)
    self.account_details_page.iconbitmap("Icon.ico")

    login_message = tk.Label(self.account_details_page, text = "Enter your new account details")
    login_message.pack()

    first_name_label = tk.Label(self.account_details_page, text = "First name:")
    first_name_label.pack()
    first_name_entry = tk.Entry(self.account_details_page)
    first_name_entry.pack()

    last_name_label = tk.Label(self.account_details_page, text = "Last name:")
    last_name_label.pack()
    last_name_entry = tk.Entry(self.account_details_page)
    last_name_entry.pack()

    new_username_label = tk.Label(self.account_details_page, text = "Username:")
    new_username_label.pack()
    new_username_entry = tk.Entry(self.account_details_page)
    new_username_entry.pack()

```

```

new_username_confirmation_label = tk.Label(self.account_details_page, text = "Confirm username:")
new_username_confirmation_label.pack()
new_username_confirmation_entry = tk.Entry(self.account_details_page)
new_username_confirmation_entry.pack()

new_password_label = tk.Label(self.account_details_page, text = "Password:")
new_password_label.pack()
new_password_entry = tk.Entry(self.account_details_page, show = "*")
new_password_entry.pack()

new_password_confirmation_label = tk.Label(self.account_details_page, text = "Confirm password:")
new_password_confirmation_label.pack()
new_password_confirmation_entry = tk.Entry(self.account_details_page, show = "*")
new_password_confirmation_entry.pack()

def return_to_menu():
    # Returns to the welcome page
    self.account_details_page.destroy()
    self.open_welcome_page()

def confirm_details():
    # Checks if the details are valid
    self._username = new_username_entry.get()
    conf_username = new_username_confirmation_entry.get()
    password = new_password_entry.get()
    conf_password = new_password_confirmation_entry.get()
    first_name = first_name_entry.get()
    last_name = last_name_entry.get()
    valid = True
    new_user = True

```

```

        if self._username != "" and conf_username != "" and password != "" and conf_password != "" and first_name
        != "" and last_name != "": # If the Entry blocks are not empty
            if len(self._username) < 8:
                # If the username length is too short, fail
                self.confirm_details_message.configure(text = "Username too short. Try Again.", fg = "red")
                valid = False
            if len(password) < 8:
                # If the password length is too short, fail
                self.confirm_details_message.configure(text = "Password too short. Try Again.", fg = "red")
                valid = False
            if self.db.existing_username_check(self._username):
                # If the username already exists, fail
                self.confirm_details_message.configure(text = "Username is already taken. Try Again.", fg = "red")
                new_user = False
            if new_username_entry.get() == new_username_confirmation_entry.get() and new_password_entry.get() ==
            new_password_confirmation_entry.get() and valid and new_user:
                # If the username and password match the confirm username and confirm password entries, succeed
                if self.db.insert_user_details(self._username, password, self.get_email(), first_name, last_name):
                    # Attempts to insert the user details into the database
                    self.confirm_details_message.configure(text = "Details match! Account added to database", fg =
"green")
                    return_to_menu_button = tk.Button(self.account_details_page, text = "Return to Menu",
command=lambda: return_to_menu())
                    return_to_menu_button.pack()
                else:
                    self.confirm_details_message.configure(text = "Database Error! Try again later", fg = "red")
                    self.account_details_page.destroy()
            else:
                self.confirm_details_message.configure(text = "Details don't match. Try Again.", fg = "red")
        else:
            self.confirm_details_message.configure(text = "Details missing. Try Again", fg = "red")

```

```

        confirm_details_button = tk.Button(self.account_details_page, text = "Confirm Details", command=lambda:
confirm_details())
        confirm_details_button.pack()
        self.confirm_details_message = tk.Label(self.account_details_page, text = "")
        self.confirm_details_message.pack()
        self.account_details_page.mainloop()

def open_otp_page(self):
    # Opens the OTP entry page

    self.account_creation_page.destroy()

    self.otp_page = tk.Tk()
    self.otp_page.title("OTP Verification")
    self.otp_page.geometry("400x400")
    self.centre_page(self.otp_page)
    self.otp_page.iconbitmap("Icon.ico")
    otp_label = tk.Label(self.otp_page, text = "OTP:")
    otp_label.pack()
    otp_entry = tk.Entry(self.otp_page)
    otp_entry.pack()
    self._validated = False

def confirm_otp():
    # Confirms the user entered the right OTP
    if otp_entry.get() != "" and self.otp_counter > 0:
        if self.email_manager.get_otp() == otp_entry.get():
            self._validated = True
            self.otp_message.configure(text = "OTP is valid!", fg = "green")

```

```

        self.otp_page.update_idletasks()
        time.sleep(10)
        self.open_account_details_page()
    else:
        self.otp_message.configure(text = f"OTP was invalid. Try Again. {self.otp_counter} tries left", fg
= "red")

        self.otp_page.update_idletasks()
        self.otp_counter -= 1
    elif self.otp_counter == 0:
        self.otp_message.configure(text = "Too many tries. The application will close now.", fg = "red")
        self.otp_page.update_idletasks()
        time.sleep(10)
        self.otp_page.destroy()

confirm_otp_button = tk.Button(self.otp_page, text = "Enter", command=lambda: confirm_otp())
confirm_otp_button.pack()
self.otp_message = tk.Label(self.otp_page, text = "")
self.otp_message.pack()

self.otp_page.mainloop()

def open_account_creation_page(self):
    # Opens the account creation page, which asks for a valid email

    self.welcome_page.destroy()

    self.account_creation_page = tk.Tk()
    self.account_creation_page.title("Account Creation")
    self.account_creation_page.geometry("400x400")
    self.centre_page(self.account_creation_page)

```

```

self.account_creation_page.iconbitmap("Icon.ico")
email_label = tk.Label(self.account_creation_page, text = "Email:")
email_label.pack()
email_entry = tk.Entry(self.account_creation_page)
email_entry.pack()

def confirm_email():
    # Confirms the email sent is correct and then sends an email containing the OTP
    if email_entry.get() != "" and self.email_counter > 0:
        self._email = str(email_entry.get())
        if not(self.db.existing_email_check(self._email)):
            if self.email_manager.my_validate_email(self.get_email()):
                self.email_message.configure(text = "Email is valid!", fg = "green")
                self.account_creation_page.update_idletasks()
                time.sleep(10)
                if self.email_manager.send_email(self.get_email()):
                    self.open_otp_page()
            else:
                self.email_message.configure(text = "An OTP code could not be generated. Please try again
later. The application will close now.", fg = "red")
                self.account_creation_page.update_idletasks()
                time.sleep(10)
                self.account_creation_page.destroy()
        else:
            self.email_message.configure(text = f"Email was invalid. Try Again. {self.email_counter} tries
left", fg = "red")
            self.account_creation_page.update_idletasks()
            self.email_counter -= 1
    else:
        self.email_message.configure(text = f"This email is already in use. Try another?", fg = "red")
        self.account_creation_page.update_idletasks()

```

```

        elif self.email_counter == 0:
            self.email_message.configure(text = "Too many tries. The application will close now.", fg = "red")
            self.account_creation_page.update_idletasks()
            time.sleep(10)
            self.account_creation_page.destroy()

confirm_email_button = tk.Button(self.account_creation_page, text = "Confirm Email", command = confirm_email)
confirm_email_button.pack()
self.email_message = tk.Label(self.account_creation_page, text = "")
self.email_message.pack()

self.account_creation_page.mainloop()

def get_verified(self):
    # Returns verified checker
    return self._verified

def set_verified(self, status):
    # Sets the verified status
    self._verified = status

def get_settings_confirmed(self):
    # Returns the settings confirmed checker
    return self._settingsconfirmed

def set_settings_confirmed(self, status):
    # Sets the settings confirmed status
    self._settingsconfirmed = status

def get_speed(self):

```

```
    # Returns simulation speed
    return self._speed

def get_foodnum(self):
    # Returns number of food
    return self._foodnum

def get_simplepreynum(self):
    # Returns number of simple prey
    return self._simplepreynum

def get_simplepredatornum(self):
    # Returns number of simple predators
    return self._simplepredatornum

def get_wolfnum(self):
    # Returns number of wolves
    return self._wolfnum

def get_deernum(self):
    # Returns number of deer
    return self._deernum

def get_email(self):
    # Returns the email
    return self._email

def get_username(self):
    # Returns the username
    return self._username
```

```
def set_username(self, username):  
    # Sets the username  
    self._username = username
```

3.1.4. dbManager.py

```
import sqlite3  
# Importing sqlite3 for database related features  
import bcrypt  
# Importing bcrypt for industry standard hashing features  
import os  
# Importing os for files and file directory related features  
import zipfile  
# Importing zipfile for zip file saving feature  
import matplotlib.pyplot as plt  
# Importing matplotlib to process and display image onto a plot figure  
from io import BytesIO  
# Importing BytesIO for binary and file-like object features  
import platform  
# Importing platform to deal with platform-specific saving  
  
class DBManager():  
  
    def __init__(self):  
        self.sqliteConnection = sqlite3.connect("user_details.db")  
        # Initialises a connection with the database named user_details  
        self.cursor = self.sqliteConnection.cursor()  
        # Creates a cursor object to interact with the database
```

```

def __hash_password(self, password):
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt())
# Hashes a password with an industry-standard salt

def __check_password(self, plain_password, hashed_password):
    return bcrypt.checkpw(plain_password.encode(), hashed_password)
# Checks if an entered password is the same as a stored hashed password

def verify_user_password(self, username, entered_password):
    # Checks if an entered password is correct
    try:

        self.cursor.execute("SELECT PASSWORD FROM USER_ACCOUNT WHERE USERNAME = ?", (username,))
        result = self.cursor.fetchone()

        if not result:
            return False

        stored_hashed_password = result[0]

        verified_state = self.__check_password(entered_password, stored_hashed_password)
        # Accesses protected method locally

        return verified_state

    except sqlite3.Error as e:
        print(f"Database error in verify_user_password: {e}")
        return False

def existing_username_check(self, username):

```

```

# Checks if a username already exists in the table
self.cursor.execute("SELECT * FROM USER_ACCOUNT WHERE USERNAME = ?", (username,))
return self.cursor.fetchone() is not None

def existing_email_check(self, email):
    # Checks if an email already exists in the table
    self.cursor.execute("SELECT * FROM USER_CONTACT WHERE EMAIL = ?", (email,))
    return self.cursor.fetchone() is not None

def _insert_user_account_details(self, username, password):
    # Inserts the user's entered username and password into the user account table
    hashed_password = self.__hash_password(password)
    # Accesses protected method locally
    self.cursor.execute(
        "INSERT INTO USER_ACCOUNT (USERNAME, PASSWORD) VALUES (?, ?)",
        (username, hashed_password)
    )
    self.sqliteConnection.commit()

def _insert_user_contact_details(self, email, first_name, last_name):
    # Inserts the user's email, first name and last name into the user contact table
    self.cursor.execute(
        "INSERT INTO USER_CONTACT (EMAIL, FIRST_NAME, LAST_NAME) VALUES (?, ?, ?)",
        (email, first_name, last_name)
    )
    self.sqliteConnection.commit()

def insert_user_details(self, username, password, email, first_name, last_name):
    # Inserts the complete user details into the program
    try:
        self._insert_user_contact_details(email, first_name, last_name)

```

```

        # Accesses above private method
        self._insert_user_account_details(username, password)
        # Accesses above private method

        self.cursor.execute(
            "INSERT INTO USER_DETAILS (USER_ACCOUNT_ID, USER_CONTACT_EMAIL) VALUES (?, ?)",
            (username, email)
        )
        self.sqliteConnection.commit()
        return True
    except sqlite3.Error as e:
        print(f"Insert error: {e}")
        return False

def insert_graph_details(self, graph_blob):
    # Inserts binary data of graph into the table
    self.cursor.execute("INSERT INTO GRAPH_DETAILS (GRAPH) VALUES (?)", (graph_blob,))
    self.sqliteConnection.commit()
    return self.cursor.lastrowid

def existing_user_in_details_check(self, username, email):
    # Checks if the username and email are already in use
    self.cursor.execute(
        "SELECT * FROM USER_DETAILS WHERE USER_ACCOUNT_ID = ? AND USER_CONTACT_EMAIL = ?",
        (username, email)
    )
    return self.cursor.fetchone() is not None

def insert_user_graph(self, username, email, graph_id):
    # Adds the graph to the user graph table
    if self.existing_user_in_details_check(username, email):

```

```

        # Checks if the user exists, as above
        self.cursor.execute(
            "INSERT INTO USER_GRAPHS (USER_ACCOUNT_ID, USER_CONTACT_EMAIL, GRAPH_ID) VALUES (?, ?, ?)",
            (username, email, graph_id)
        )
        self.sqliteConnection.commit()

def get_full_name_from_username(self, username):
    # Method to extract the full name of the user from the tables by using username
    try:
        self.cursor.execute("SELECT USER_CONTACT_EMAIL FROM USER_DETAILS WHERE USER_ACCOUNT_ID = ?", (username,))
        result = self.cursor.fetchone()
        if not result:
            return None

        email = result[0]

        self.cursor.execute("SELECT FIRST_NAME, LAST_NAME FROM USER_CONTACT WHERE EMAIL = ?", (email,))
        name_result = self.cursor.fetchone()
        if not name_result:
            return None

        first_name, last_name = name_result
        return f"{first_name} {last_name}"

    except sqlite3.Error as e:
        print(f"Database error in get_full_name_from_username: {e}")
        return None

def __get_user_graphs(self, username):
    # Gets the IDs of every graph stored under a person's account

```

```

try:
    self.cursor.execute("SELECT USER_CONTACT_EMAIL FROM USER_DETAILS WHERE USER_ACCOUNT_ID = ?", (username,))
    result = self.cursor.fetchone()

    if not result:
        print("No USER_DETAILS found for username:", username)
        return []

    email = result[0]
    print(f"Found email for {username}: {email}")

    self.cursor.execute(
        "SELECT GRAPH_ID FROM USER_GRAPHS WHERE USER_ACCOUNT_ID = ? AND USER_CONTACT_EMAIL = ?",
        (username, email)
    )
    graph_ids = self.cursor.fetchall()
    print(f"Graph IDs found for {username} + {email}: {graph_ids}")

    if not graph_ids:
        print("No graphs found in USER_GRAPHS table.")
        return []

    graphs = []
    for graph_id in graph_ids:
        self.cursor.execute("SELECT GRAPH FROM GRAPH_DETAILS WHERE ID = ?", (graph_id[0],))
        row = self.cursor.fetchone()
        if row and row[0]:
            graphs.append(row[0])
        else:
            print(f"No graph found for GRAPH_ID: {graph_id[0]}")

```

```

        return graphs
    except sqlite3.Error as e:
        print(f"Error fetching graphs: {e}")
        return []

def _save_graphs_as_pngs(self, graphs, temp_folder):
    png_files = []

    for i, graph_blob in enumerate(graphs):
        image_png = BytesIO(graph_blob)
        # Binary data is wrapped in a BytesIO object
        img = plt.imread(image_png, format='png')
        # Read the image from the binary and converts to png
        fig, ax = plt.subplots()
        ax.imshow(img)
        ax.axis('off')

        file_path = os.path.join(temp_folder, f"graph_{i+1}.png")
        # Adds a graph to the temp_folder
        fig.savefig(file_path, format='png')
        plt.close(fig)
        png_files.append(file_path)
        # Adds the file path to an array, to be zipped

    return png_files

def download_graphs(self, username):
    download_folder = self.get_downloads_folder()
    # Gets the download folder on the system
    temp_folder = "temp_graphs"
    os.makedirs(temp_folder, exist_ok=True)

```

```

# Creates the directory for the temporary folde

graphs = self.__get_user_graphs(username)
# Accesses protected method locally

if not graphs:
    print("No graphs found for this user.")
    return False

pngs = self._save_graphs_as_pngs(graphs, temp_folder)
# Saves the png list

zip_file_path = os.path.join(download_folder, f"{username}_graphs.zip")
# Constructs a file path for the zip folder
with zipfile.ZipFile(zip_file_path, 'w') as zipf:
    # Opens the file path in write mode
    for file in pngs:
        zipf.write(file, os.path.basename(file))
        # Writes the files to the zip file, stripping them from the temp folder

for file in pngs:
    # This snippet cleans up the files and folder used as per best practices
    os.remove(file)
os.rmdir(temp_folder)

print(f"Graphs downloaded successfully! ZIP file saved at: {zip_file_path}")

return True

def get_downloads_folder(self):
    if platform.system() == "Windows":

```

```

        # If the user is on a Windows OS device
        import winreg
        with winreg.OpenKey(winreg.HKEY_CURRENT_USER, r"SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\Shell
Folders") as key:
            # Opens a key in the Windows Registry to access the shell files
            return winreg.QueryValueEx(key, '{374DE290-123F-4565-9164-39C4925E467B}')[0]
        # Uses the specific registry for the Downloads folder to return the file path
    elif platform.system() == "Darwin":
        # If the user is on a Mac
        return os.path.expanduser("~/Downloads")
    # Returns the download directory on Mac devices

def create_tables(self):
    # Creates all the tables
    try:
        table_user_account_details = """CREATE TABLE IF NOT EXISTS USER_ACCOUNT(
            USERNAME VARCHAR PRIMARY KEY,
            PASSWORD BLOB NOT NULL
        );"""

        table_user_contact_details = """CREATE TABLE IF NOT EXISTS USER_CONTACT(
            EMAIL VARCHAR PRIMARY KEY,
            FIRST_NAME VARCHAR NOT NULL,
            LAST_NAME VARCHAR NOT NULL
        );"""

        table_all_user_details = """CREATE TABLE IF NOT EXISTS USER_DETAILS(
            USER_ACCOUNT_ID VARCHAR,
            USER_CONTACT_EMAIL VARCHAR,
            PRIMARY KEY (USER_ACCOUNT_ID, USER_CONTACT_EMAIL),
            FOREIGN KEY (USER_ACCOUNT_ID) REFERENCES USER_ACCOUNT(USERNAME),

```

```

        FOREIGN KEY (USER_CONTACT_EMAIL) REFERENCES USER_CONTACT(EMAIL)
    );"""

table_graph_details = """CREATE TABLE IF NOT EXISTS GRAPH_DETAILS(
    ID INTEGER PRIMARY KEY AUTOINCREMENT,
    GRAPH BLOB
);"""

table_user_graphs = """CREATE TABLE IF NOT EXISTS USER_GRAPHS(
    USER_ACCOUNT_ID VARCHAR,
    USER_CONTACT_EMAIL VARCHAR,
    GRAPH_ID INTEGER,
    PRIMARY KEY (USER_ACCOUNT_ID, USER_CONTACT_EMAIL, GRAPH_ID),
    FOREIGN KEY (USER_ACCOUNT_ID, USER_CONTACT_EMAIL) REFERENCES USER_DETAILS(USER_ACCOUNT_ID,
USER_CONTACT_EMAIL),
    FOREIGN KEY (GRAPH_ID) REFERENCES GRAPH_DETAILS(ID)
);"""

self.cursor.execute(table_user_account_details)
self.cursor.execute(table_user_contact_details)
self.cursor.execute(table_all_user_details)
self.cursor.execute(table_graph_details)
self.cursor.execute(table_user_graphs)
self.sqliteConnection.commit()

except sqlite3.Error as error:
    print(f"Error occurred: {error}")

```

3.1.5. EmailManager.py

```
import email
# Importing to specify email content
import smtplib
# Importing to send emails using the smtp protocol
from email_validator import *
# Importing the email_validator library to make sure emails are registered and real
import requests
# Importing to handle API requests
API_URL = "https://qrng.anu.edu.au/API/jsonI.php?length=20&type=uint16"
# API for the Australian National University's quantum number data set

class EmailManager():

    def __init__(self):
        self.__sender = 'atikshtemp@gmail.com'
        # Protected attribute containing the email address used to send the OTP
        self.__app_password = "ehci qwaf xsbj biwa"
        # Protected attribute containing the app password used to verify the sender email
        self.receiver = ""
        # Initialises the reciever
        self.otp = ""
        # Initialises the otp
        self.otp_status = False
        # Initialises the otp checker
        self.status = 0
        # Initialises the response status code

    def get_numbers(self):
```

```

# Calls the API to get true random numbers
try:
    response = requests.get(API_URL, timeout= 6)
    self.status = response.status_code
    data = response.json()

    if data["success"] == True:
        return data
    else:
        print("Request unsuccessful")
except:
    print(f"Response failed with error code: {self.status}")

def generate_otp(self):
    # Generates the OTP code to be sent to the user
    numbers = self.get_numbers()
    otp_list = []
    if type(numbers) is dict:
        for i in list(numbers["data"]):
            # This snippet takes the numbers from the API and gets valid ASCII characters for the OTP
            i = i % (126-33)
            i = chr(i+33)
            otp_list.append(i)
        self.otp = "".join(otp_list)
        self.otp_status = True
        return self.otp
    # Returns the OTP
    else:
        self.otp_status = False

def get_otp(self):

```

```

        return self.otp

def get_otp_status(self):
    return self.otp_status

def send_email(self, receiver_email):
    # This method sends the welcome email to the new user
    self.my_validate_email(receiver_email)
    # Calls the validate email method to ensure email exists
    self.generate_otp()
    if self.get_otp_status() == False:
        return False
    else:
        message = email.message_from_string(f"Welcome to the simulation!\n"
                                             f"You will need to confirm that you are a human user by entering this
code into the window:\n"
                                             f"*** {self.otp} *** (Copy this code manually)\n"
                                             f"Please note that this window will close after 3 minutes")

        message['From'] = self.__sender
        # Puts the sender email address in the From section of the email
        message['To'] = receiver_email
        # Puts the receiver email address in the To section of the email
        message['Subject'] = "Welcome to the simulation"
        # Makes the given string the email subject

        smtp = smtplib.SMTP("smtp.gmail.com",587)
        # Connects with the smtp server address for Gmail, and initialises to send via TLS (encrypted
communication)
        smtp.ehlo()
        # Sends a hello request to the receiver
        smtp.starttls()

```

```

        # Starts encrypted emailing
        smtp.ehlo()
        smtp.login(self.__sender, self.__app_password)
        # Logs the sender email in
        smtp.sendmail(self.__sender, receiver_email, message.as_string())
        # Sends the specified email message as a string
        return True

def my_validate_email(self, email):
    # Function to validate email addresses, ensuring they are real and registered.
    try:
        if validate_email(email):
            self.receiver = email
            return True
    except EmailNotValidError as e:
        print("Invalid email:", e)

```

3.1.6. graphManager.py

```

import pygame as pg
# Importing pygame to convert the graph into a pygame Surface to display
import numpy as np
# Importing numpy for array features
from scipy.integrate import odeint
# Importing scipy for the differential equation solver
from scipy.optimize import curve_fit
# Importing scipy for the curve fit

```

```

import matplotlib.pyplot as plt
# Importing pygame to deal with creating/closing matplotlib figures
from matplotlib.backends.backend_agg import FigureCanvasAgg as FigureCanvas
# Importing to wrap matplotlib figures so that they can be displayed onto pygame Surfaces

class GraphManager():
    def __init__(self):
        self._last_figure = None
        # Initialises a variable to hold the last matplotlib figure
        self.show_real_data = True
        # Initialises a Boolean to show the real data

    def lotka_volterra(self, populations, t, alpha, beta, delta, gamma):
        # Sets up the Lotka-Volterra system using passed in parameters
        x, y = populations
        dxdt = alpha * x - beta * x * y
        dydt = delta * x * y - gamma * y
        return [dxdt, dydt]

    def solve_lv(self, t, alpha, beta, delta, gamma, x0, y0):
        # Uses differential equation solver to return values of the Lotka-Volterra systems
        return odeint(self.lotka_volterra, [x0, y0], t, args=(alpha, beta, delta, gamma)).T
    # Transposes results to be able to plot them on the axis

    def my_concatenate_results(self, t, alpha, beta, delta, gamma, x0, y0):
        x, y = self.solve_lv(t, alpha, beta, delta, gamma, x0, y0)
        return np.concatenate([x, y])
    # Concatenates the results into a single numpy array to plot

    def fit_lv_model(self, time, prey_data, pred_data):
        # Fits the Lotka Volterra model to an estimate

```

```

t = np.array(time)
# Converts all passed in parameters to numpy arrays
x_data = np.array(pre_data)
y_data = np.array(pred_data)
x0, y0 = x_data[0], y_data[0]

def fit_function(t, alpha, beta, delta, gamma):
    return self.my_concatenate_results(t, alpha, beta, delta, gamma, x0, y0)
# Solves and concatenates the results with specified parameters p0

combined_data = np.concatenate([x_data, y_data])
try:
    params, _ = curve_fit(fit_function, t, combined_data, p0=[0.05, 0.02, 0.02, 0.1], maxfev=10000)
    # Returns the parameters for the next iteration
    return params
except RuntimeError:
    return None

def make_fitted_plot(self, pop_time, pop_pre, pop_pred, params, show_real_data=True):
    # Plots the results

    if self._last_figure is not None:
        # Closes the last figure to avoid overuse of memory
        plt.close(self._last_figure)
        self._last_figure = None

    fig, ax = plt.subplots(figsize=(5, 3), dpi=100)
    # Defines the size and image quality of graph
    self._last_figure = fig

    canvas = FigureCanvas(fig)

```

```

# Wraps the graph in a FigureCanvas object so it can be drawn directly onto pygame

t = np.array(pop_time)
x_data = np.array(pop_pre)
y_data = np.array(pop_pred)
x0, y0 = x_data[0], y_data[0]

if show_real_data:
    # Plots real data only if it has been selected
    ax.plot(t, x_data, 'g.', label='Prey (real)')
    ax.plot(t, y_data, 'r.', label='Predators (real)')

if params is not None:
    # If there are parameters to plot, plot the lotka-volterra model
    x_fit, y_fit = self.solve_lv(t, *params, x0, y0)
    ax.plot(t, x_fit, 'g-', label='Prey (model)')
    ax.plot(t, y_fit, 'r-', label='Predators (model)')

ax.legend()
ax.set_title("Live LV Model Fit")
ax.set_xlabel("Time")
ax.set_ylabel("Population")
ax.grid(True)

fig.tight_layout()
canvas.draw()
# Passes the figure into memory
renderer = canvas.get_renderer()
# Gets a renderer object to extract raw data
raw_data = np.frombuffer(renderer.buffer_rgba(), dtype=np.uint8)
# Extracts raw binary data and turns it into a numpy array of 8 bit integers

```

```
size = canvas.get_width_height()
# Gets the width and height of the now rendered figure
surf = pg.image.frombuffer(raw_data, size, "RGBA")
# Converts the figure into a pygame Surface to display

return surf,fig
```

3.2. Libraries

This is a list of the external libraries used in the development of this program.

External libraries	Used for
pygame	Creating games and managing graphics, events and user input
numpy	Numerical computing with multi-dimensional arrays and matrix operations
tkinter	Creating graphical user interfaces
matplotlib	Plotting graphs and charts, both statically and dynamically
scipy	Scientific computing and advanced mathematical calculations
sqlite3	Providing an interface to interact directly with relational databases
bcrypt	Hashing passwords for storage and user authentication
email	Constructing and parsing emails
email_validator	Checks if emails are syntactically correct and verifies that domains are registered
smtplib	Sending emails using the Simple Mail Transfer Protocol
requests	Sending HTTP requests and handling responses
datetime	Manipulating date and time objects and time-based calculations
time	Time-related tasks
random	Generating pseudorandom numbers and making pseudorandom selections
io	Handling Input/Output operations and working with in-memory files
os	Managing files and directories
Platform	Accessing information about the user's platform
Zipfile	Handles everything to do with .zip archive files

Table 8: External Libraries Table

Pygame was used to build the simulation-specific aspects of the project, specifically using the Sprite classes to create the creatures and food. Pygame was also valuable in user interaction, as it allowed the detection of keyboard and mouse inputs.

NumPy was used to handle the vector movements of creatures and also to plot the graphs which were produced during the simulation

Tkinter was used to handle user interaction, taking inputs for the simulation, managing user accounts and contact details and providing a way to access user's saved graphs.

Matplotlib was used to plot and display graphs of the simulation in real time.

SciPy was used to solve the set of differential equations for the Lotka-Volterra model and fit the results to a set of starting parameters.

SQLite3 was used to create and query the databases created by the program

Bcrypt was used to hash passwords before storing them in the database, and to check if entered passwords matched stored hashed passwords

Email was used to construct the email message

Email_validator was used to check if the user's email was correct and that the domain was registered

SMTP was used to log into the sender's email using an app password and then to send the email to the specified receiver securely

Requests was used to access the Australian National University Quantum Numbers dataset to request true random numbers

Datetime was used to manage the time the simulation ran for, and also for generating unique file names using UTC time

Time was used to pause the user interface, so that error messages could be displayed without the program shutting down immediately.

Random was used to allocate creature locations and resolve creature move clashes using pseudorandom numbers

io was used for the Bytesio object, which allowed graphs to be saved as PNG files in memory

os was used to create a temporary folder and manage file paths to save PNG files to

platform was used to get the user's operating system to determine the parameterised file path for the downloads folder.

Zipfile was used to zip the graph PNG files so that teachers can access all their previously created files.

3.3. Coding Style

A sophisticated coding style was attempted throughout this project. Code was produced almost entirely in modules for easier refactoring and improved security. The program was separated into files containing overall classes to be inherited and composed in other files. The files and classes were named with the convention [function]manager to demonstrate the separation of modules due to their function. Classes encapsulate all subroutines relevant to the overall class function. In cases where a subtype of a class was created, methods were inherited, and some of these were overridden to change the details of their implementation, whilst keeping the same general purpose.

The main file interacted with the module files via their class interfaces, and the tkinterManager module was composed of smaller modules, which interacted with the larger module via methods and attribute getter/setter methods, rather than direct access. Certain methods and attributes were also made private and protected to preserve their security within the program. Defensive programming was applied through the use of the main guard to ensure the code was only executed when directly run and not when imported. Input validation, type checking and exception

handling were also used as part of this. Constants were used to set properties, such as the grid size, which did not need to be changed during runtime. Rather than fixed file paths, parameterised file paths were used to ensure that all types of devices were able to use the download feature.

The use of global variables is minimised, as only the image files are stored globally. Local variables are used in functions to minimise memory use whilst the program is running. White space and comments were used effectively to improve the understanding and readability of the code.

3.4. Technical Skill Tables

Model	Description	Group	Page Number
Complex data model in database	A complete 3rd normalised form database was constructed and used to store user details and graphs	A	114
Queues	A Boolean array was used as a queue to verify the entered settings were appropriate. The queue was initially populated with False values. If a setting was valid, a False value would be dequeued and a True value enqueued. If the queue at the end contained all True values, the entered settings were valid, otherwise, an error message would be thrown up.	A	87
Vectors	NumPy arrays were used to model the creature positions. Vector calculations, such as the modulus and vector sums, were used.	A	69
Sets	Sets were used to store vector locations of creatures without allowing two of the same locations to be stored.	A	58
Files organised for direct access	All files were made accessible in a single directory for direct access by the program	A	Section 2.2
Complex scientific/mathematical model	The Lotka-Volterra differential equations were used to model the simulation dynamics. Differential equation solvers and curve-fitting functions were used to plot graphs.	A	120
Complex user-defined use of object-orientated programming (OOP) model	A complete object-oriented solution is achieved through extensive use of classes. Classes were used to encapsulate different aspects of the code and interacted with each other via inheritance, composition and polymorphism.	A	Section 2.6

Model	Description	Group	Page Number
Complex client-server model	A complex client-server model is achieved through the account creation system, which sends users an email to verify their existence and prompts them to enter details to make an account. These details are then stored on a local database, and when logging in, this database is checked to make sure details match. Users interact with the system to save and access their graphs through this account.	A	See Figure 23: Client Server Class Diagrams
Multi-dimensional arrays	Multi-dimensional arrays were used to store predator population, prey population and time values to be plotted using matplotlib	B	122
Dictionaries	Dictionaries were used in parsing JSON data from API calls for true random numbers. Furthermore, dictionaries were used to store the planned moves of predators and prey in key-value pairs.	B	117,58

Table 9: Technical Skills - Model

Algorithm	Description	Group	Page Number
Cross-table parameterised SQL	All table interaction was done using parameterised SQL to avoid security risks with SQL injection. Cross-table SQL was used at various points as well, due to the normalised nature of the tables.	A	109,110
User DDL script	SQL CREATE TABLE statements were designed and written to create multiple data tables. Tables are to the third normalised form, to avoid redundancies and improve query efficiency.	A	114
List operations	Several list operations are included in the program. Append and extend commands are used to add individual items and other whole lists to lists. Remove operations are used to remove specific items from lists. Advanced techniques such as list comprehensions with filters are also used.	A	75, 78, 113
Queue Operations	Python doesn't have native support for queue data structures, but the pop and append list operations are used to model enqueue and dequeue operations	A	87-89
Hashing	Passwords were hashed before being stored into the database. This was done using the bcrypt library to ensure maximal security	A	107
Advanced matrix operations	A matrix transpose was used on results from the differential equation solver in order to plot values on the axis of the graphs.	A	120
Complex user-defined algorithms	Many complex algorithms were used throughout the program such as a randomised greedy algorithm, randomised greedy algorithm with biased random walk, a boolean queue checker, database query methods and so on.	A	67-73, 77-90 ,86-90, 110

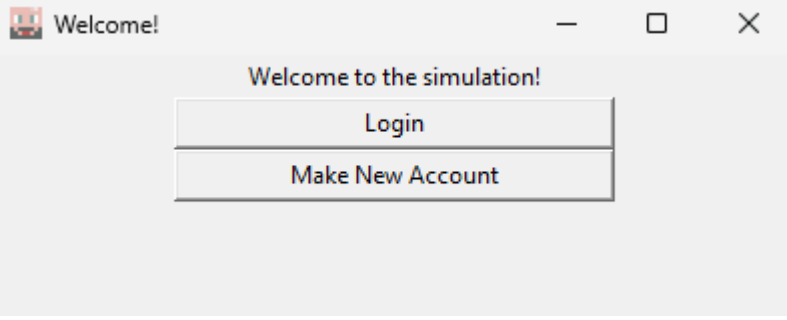
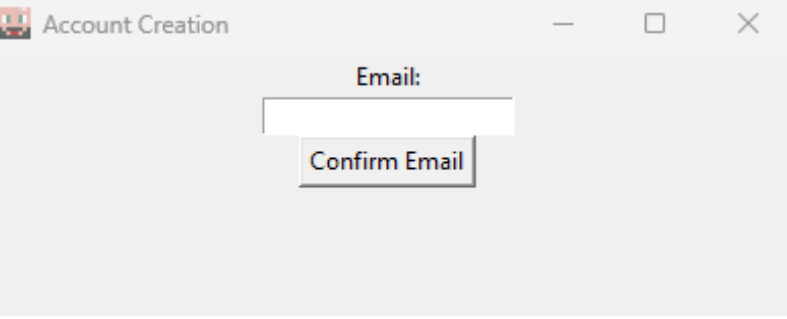
Algorithm	Description	Group	Page Number
Dynamic generation of objects based on complex user-defined use of OOP model	Objects were dynamically instantiated and destroyed during runtime. Creature classes also had methods to instantiate an instance of their own class using the reproduce method during runtime.	A	73, 83
Server-side scripting using request and response objects and server-side extensions for a complex client-server model	A request is sent to an API for a list of random numbers, which is then received and processed to produce a random string of ASCII characters for an OTP, sent as part of a welcome email to the user. The program then waits for a correct response from the user to verify them.	A	116-119
Calling parameterised Web service APIs and parsing JSON to service a complex client-server model	The Australian National University's Quantum Random Number API is called to generate true random numbers for a 20-character-long OTP	A	116,117
Writing and reading from files	Graphs were written to files in a binary format. Image files were read in to represent the creature sprites. Files were zipped to be downloaded in a compressed format, so that image files did not take up too much space.	B	110-114

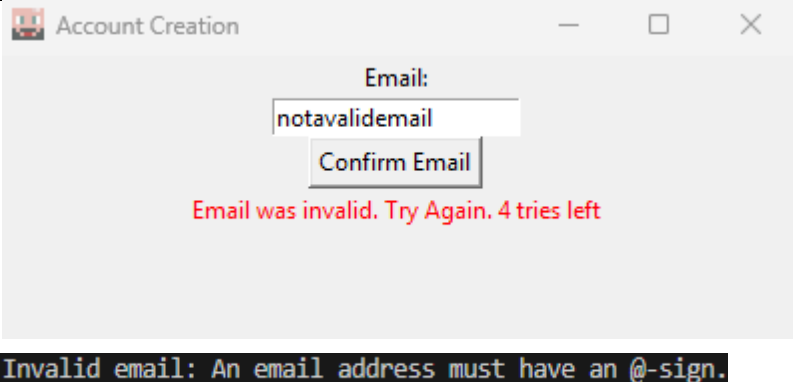
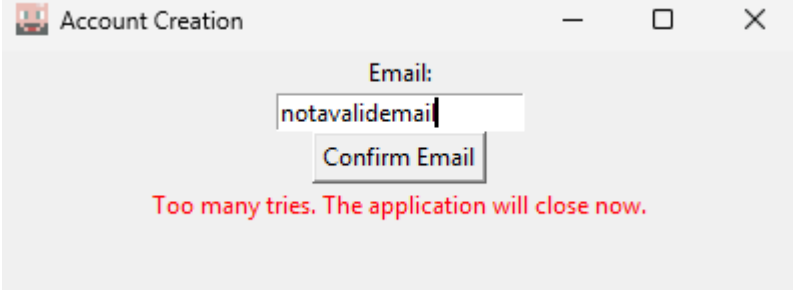
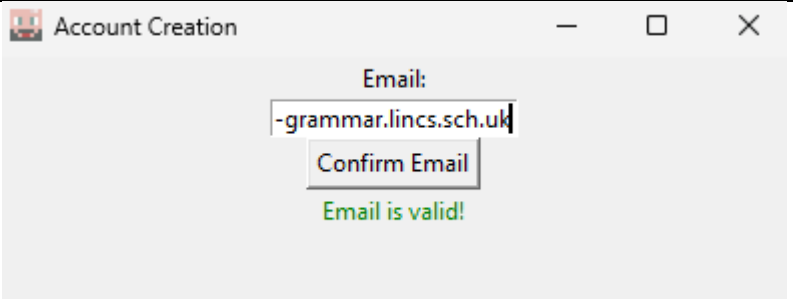
Table 10: Technical Skills - Algorithms

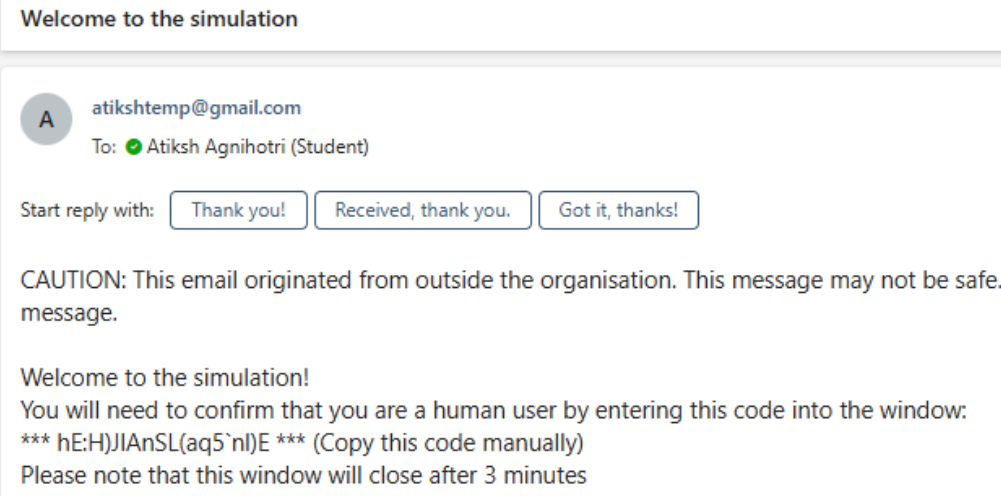
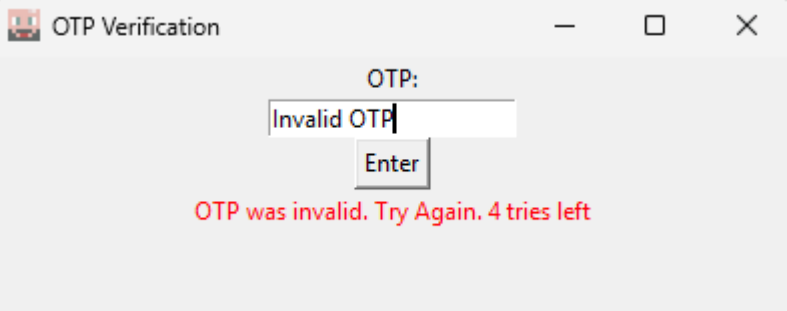
Chapter 4. NEA Testing:

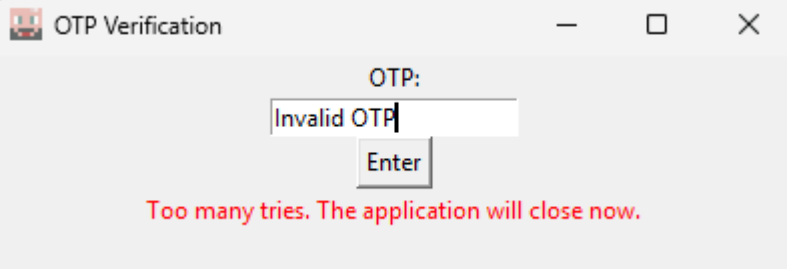
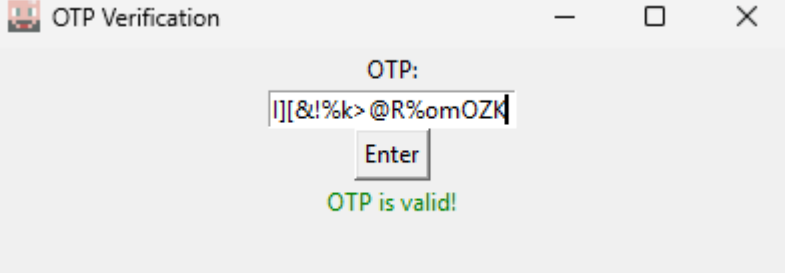
This section shows my testing objectives evaluated against the solution produced. Screenshots of the user interface, as well as test calls via the Python print function where necessary, are shown. A RAG (Red, Amber, Green) rating is provided, indicating full success, partial success and failure, respectively.

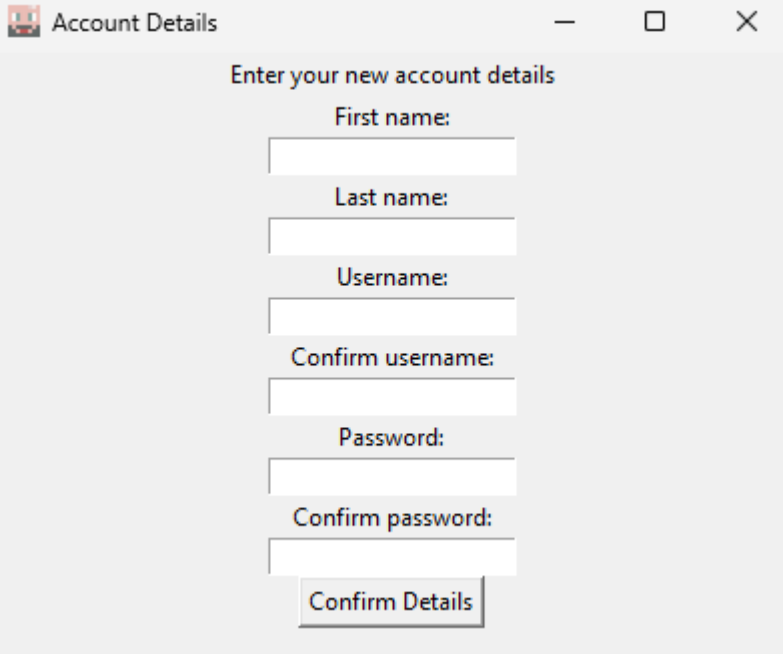
4.1. Test Table

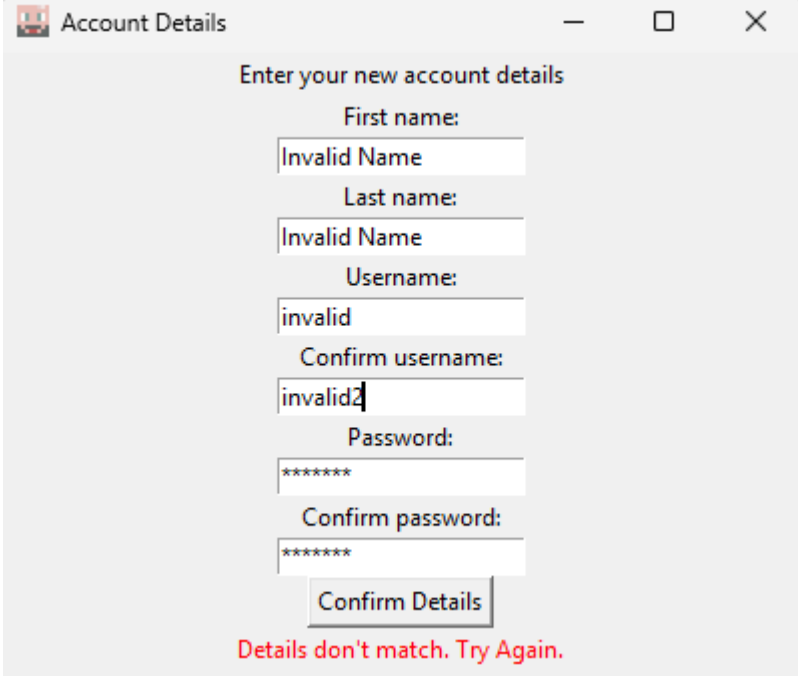
Test Number	Description	Evidence	RAG Rating
Test 1	The user should first be presented with the option to log in or make a new account.	 <pre>Welcome Page Opened! Users may log in or create a new account</pre>	
Test 2	If the user clicks on the new account screen, they should be taken to a screen which asks them to input their email.	 <pre>Make new account pressed! Account creation page opened!</pre>	

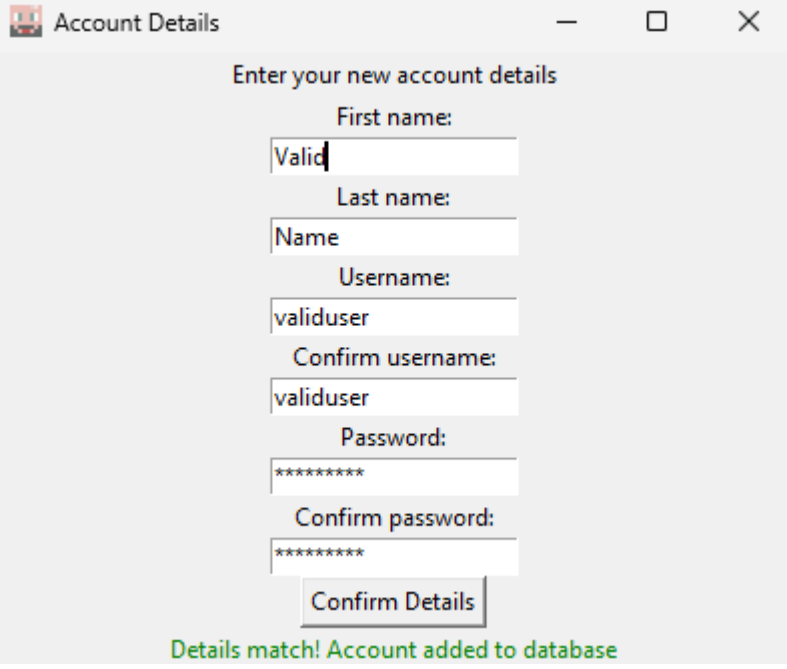
Test Number	Description	Evidence	RAG Rating
Test 3	If the user inputs an invalid email, display an error message.	 The screenshot shows a window titled "Account Creation" with a text input field labeled "Email:" containing the text "notavalidemail". Below the input field is a button labeled "Confirm Email". A red error message states: "Email was invalid. Try Again. 4 tries left". Below the window, a black command prompt window displays the text: "Invalid email: An email address must have an @-sign."	
Test 4	If the user keeps inputting an invalid email, close the program	 The screenshot shows the same "Account Creation" window with "notavalidemail" in the input field and the "Confirm Email" button. The red error message now reads: "Too many tries. The application will close now."	
Test 5	Once the user inputs a valid email, an email should be sent to the user.	 The screenshot shows the "Account Creation" window with the input field containing a valid email address: "-grammar.lincs.sch.uk". The "Confirm Email" button is visible, and a green message states: "Email is valid!"	

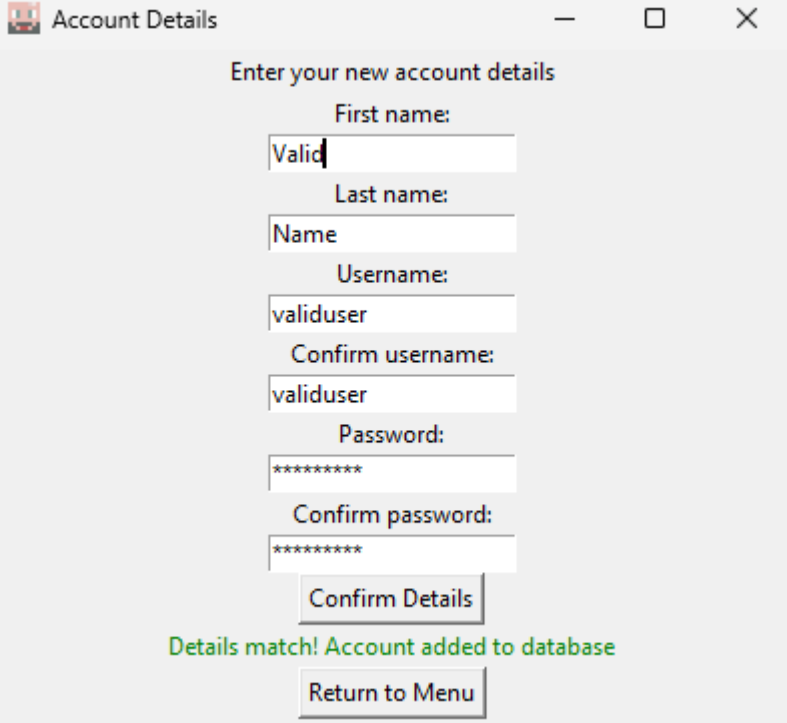
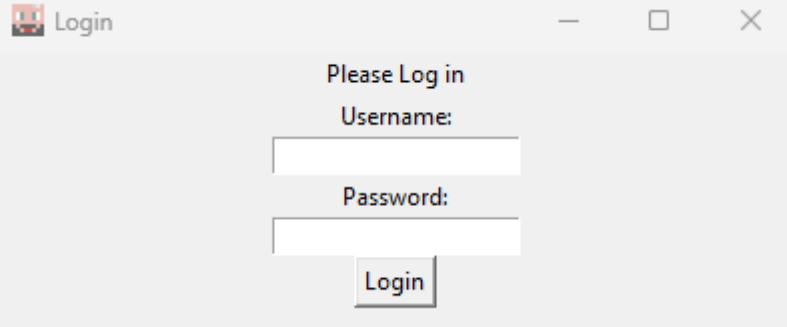
Test Number	Description	Evidence	RAG Rating
Test 6	A completely random code should be generated to verify that the user is real and has access to the email account input	OTP Generated!	
Test 7	The email should contain a unique code that the user must enter into the program to verify they are a real person and have access to the email account that was input.	 Welcome to the simulation A atikshtemp@gmail.com To: Atiksh Agnihotri (Student) Start reply with: Thank you! Received, thank you. Got it, thanks! CAUTION: This email originated from outside the organisation. This message may not be safe. message. Welcome to the simulation! You will need to confirm that you are a human user by entering this code into the window: *** hE:H)JIAnSL(aq5`n)E *** (Copy this code manually) Please note that this window will close after 3 minutes	
Test 8	If the code entered is invalid, an error message should be displayed	 OTP Verification OTP: Invalid OTP Enter OTP was invalid. Try Again. 4 tries left	

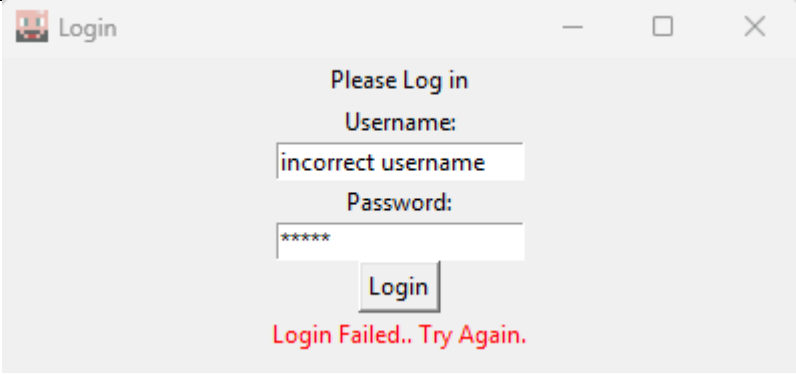
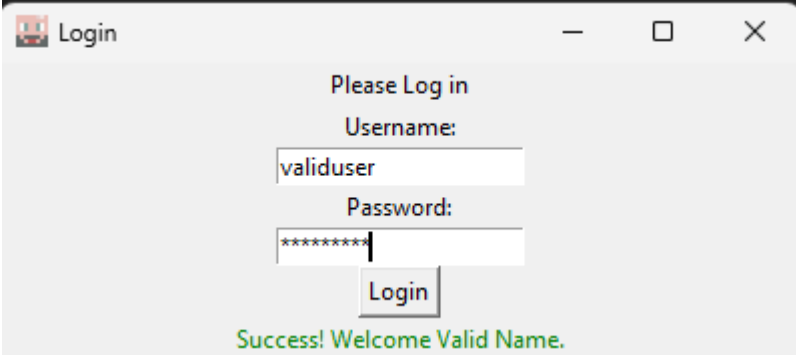
Test Number	Description	Evidence	RAG Rating
Test 9	If the user keeps inputting an invalid code, close the program	 OTP Verification OTP: Invalid OTP Enter Too many tries. The application will close now.	
Test 10	Once the code is validated, a message should be displayed to indicate this.	 OTP Verification OTP: 11[&!%k>@R%omOZK Enter OTP is valid!	

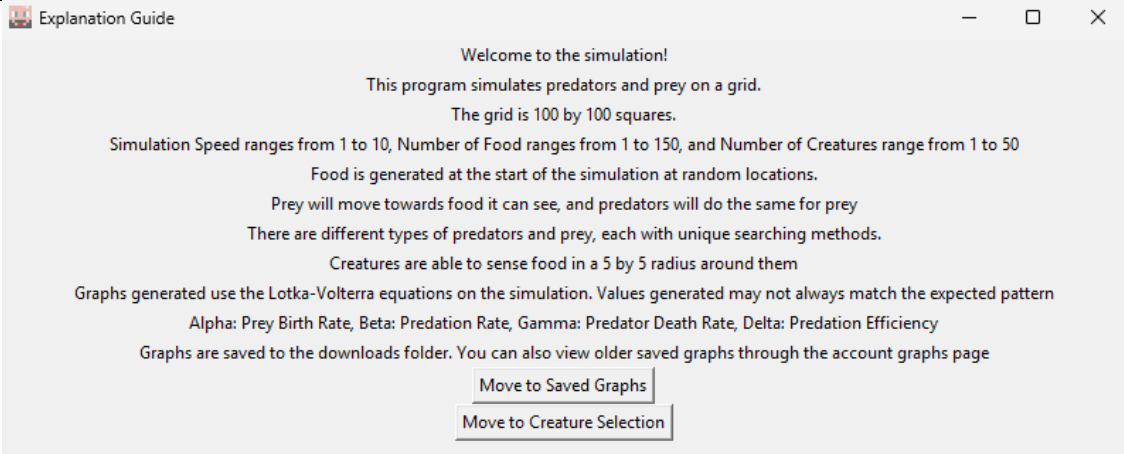
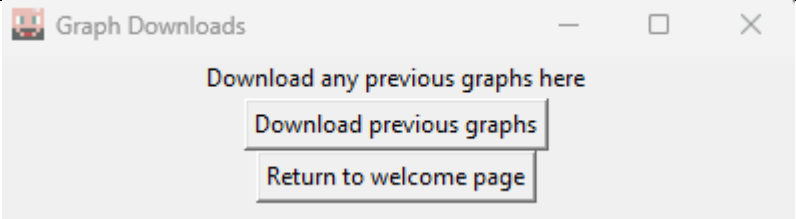
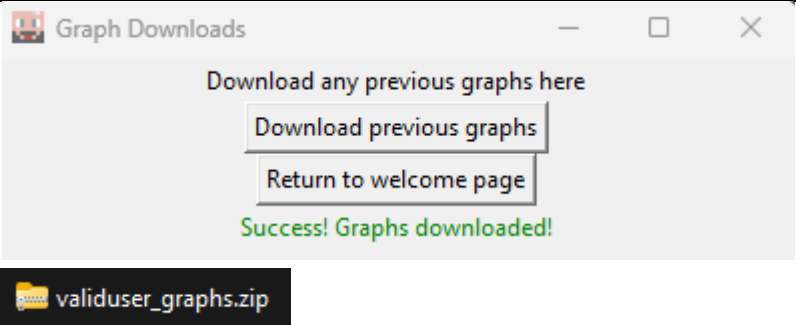
Test Number	Description	Evidence	RAG Rating
Test 11	The user should be taken to a page which records their details. The user should be asked to enter their first name and last name, and to enter their username and password twice, to confirm these are the correct details	 Account Details Entry Form Page opened!	

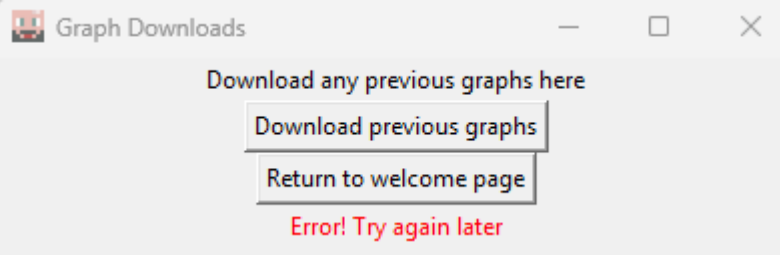
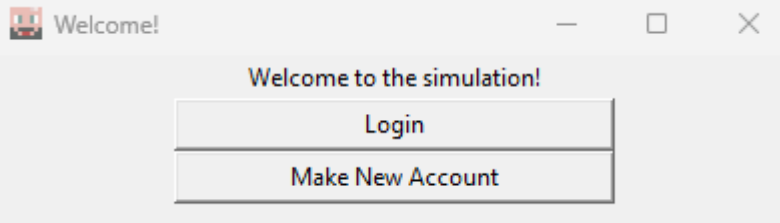
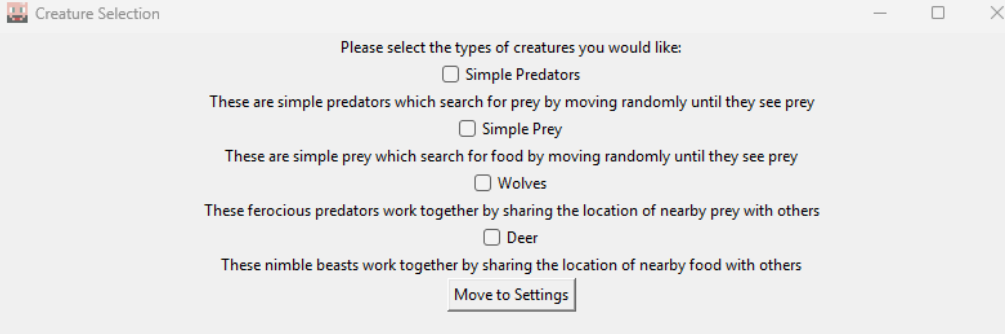
Test Number	Description	Evidence	RAG Rating
Test 12	If the details entered are invalid or already in use, an error message should be displayed	 The screenshot shows a window titled 'Account Details' with the heading 'Enter your new account details'. It contains four input fields with error messages: 'First name:' with 'Invalid Name', 'Last name:' with 'Invalid Name', 'Username:' with 'invalid', and 'Confirm username:' with 'invalid2'. Below these are 'Password:' and 'Confirm password:' fields, both containing '*****'. A 'Confirm Details' button is at the bottom. A red error message 'Details don't match. Try Again.' is displayed at the bottom of the window. Whenever this occurred, an error message was displayed, but it was always "Details don't match", rather than being more specific and telling users if they had an invalid username/password length or if their username was in use. Partial Success.	

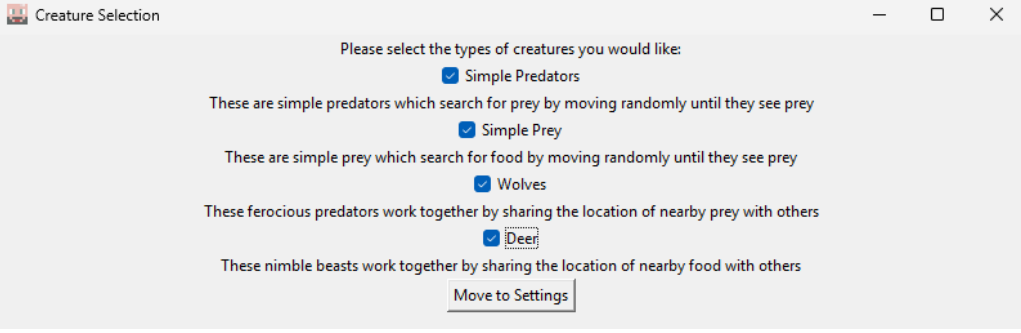
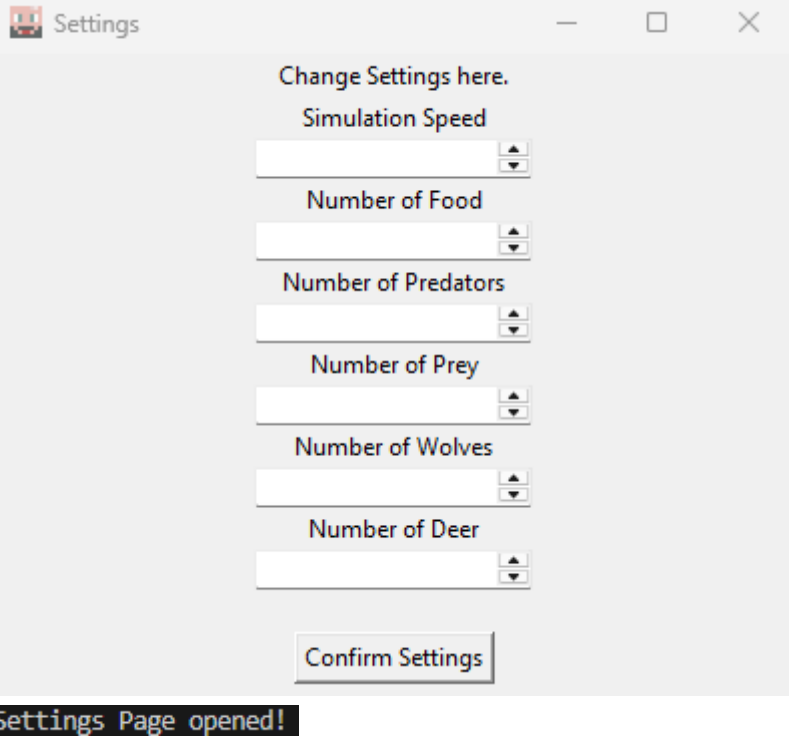
Test Number	Description	Evidence	RAG Rating
Test 13	Once the user has entered these details correctly, they should be stored in a local database, alongside the email which the user used earlier.	 9 06aa11@bourne-grammar.lincs.sch.uk Valid Name Database entry with email, first name and last name 9 validuser 06aa11@bourne-grammar.lincs.sch.uk Database entry with username and email	
Test 14	The password needs to be hashed to ensure that user details are not stored insecurely.	9 validuser 24326224313224674d6a315158497677575350715... Database entry with username and hashed password	

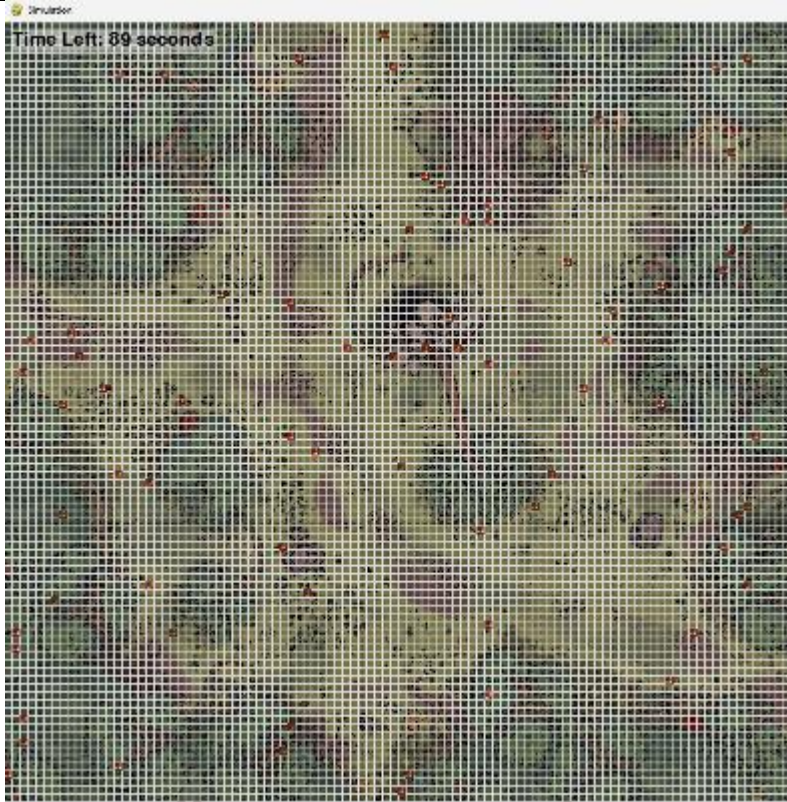
Test Number	Description	Evidence	RAG Rating
Test 15	If successful in creating an account, the user should be presented with the option to return to the welcome page.	 The screenshot shows a window titled 'Account Details' with the heading 'Enter your new account details'. It contains several input fields: 'First name:' with 'Valid' entered, 'Last name:' with 'Name' entered, 'Username:' with 'validuser' entered, 'Confirm username:' with 'validuser' entered, 'Password:' with '*****' entered, and 'Confirm password:' with '*****' entered. Below the fields are two buttons: 'Confirm Details' and 'Return to Menu'. A green message at the bottom states 'Details match! Account added to database'.	
Test 16	If the user clicks on the login option, they should be presented with the option to enter a username and password.	 The screenshot shows a window titled 'Login' with the heading 'Please Log in'. It contains two input fields: 'Username:' and 'Password:'. Below the fields is a 'Login' button. A status bar at the bottom of the window displays the message 'Login Page opened!'.	

Test Number	Description	Evidence	RAG Rating
Test 17	If the username or password are incorrect, display an error message.		
Test 18	If the username and password are correct, a message should be displayed to indicate this.		

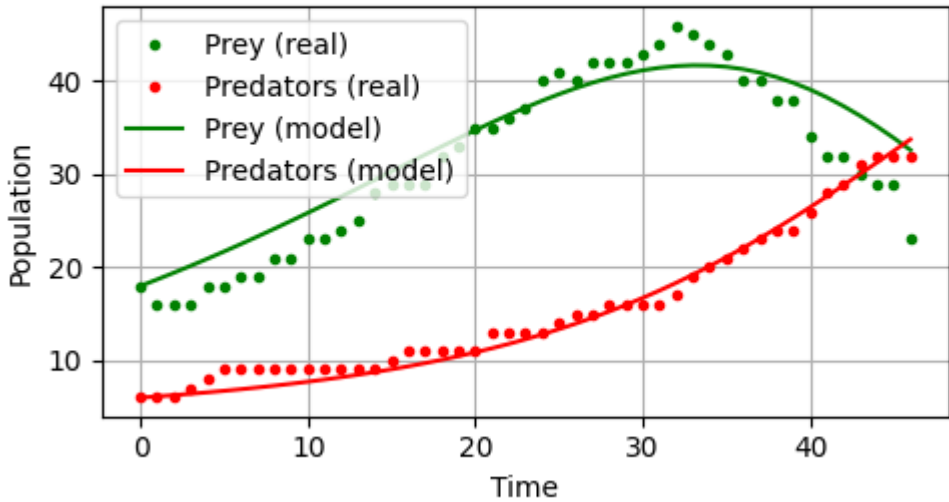
Test Number	Description	Evidence	RAG Rating
Test 19	The user should be taken to a screen which explains the simulation details to them. The user should be presented with the options of moving to the creature selection screen or to view their saved graphs	 Explanation Page opened!	
Test 20	If the user chooses to view their saved graphs, take them to the graph downloads page	 Graph Downloads Page opened!	
Test 21	If the user has any saved graphs, these should be downloaded to the user's downloads folder as a zip file containing all saved graphs.	 validuser_graphs.zip	

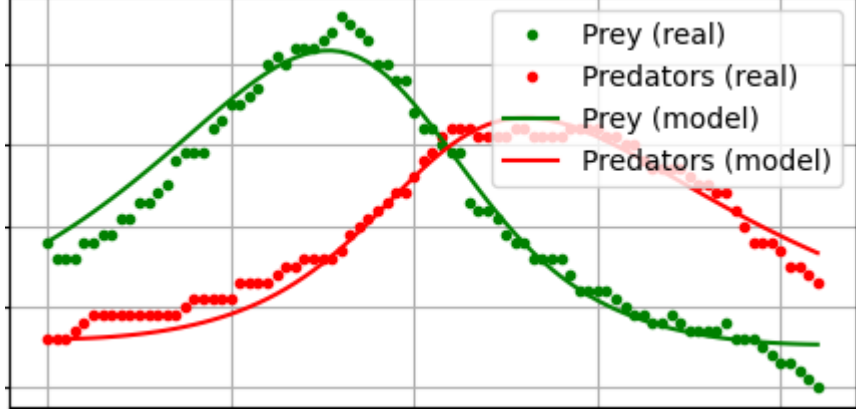
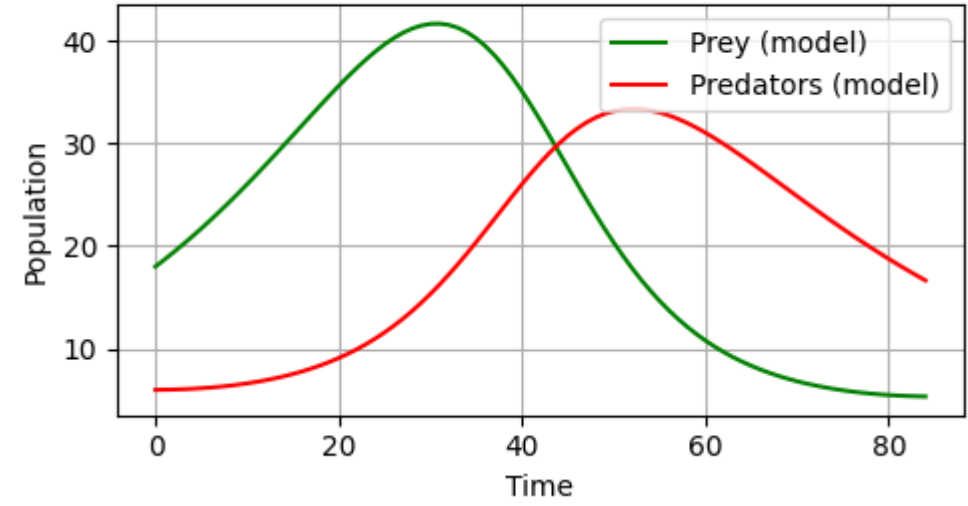
Test Number	Description	Evidence	RAG Rating
Test 22	If there are errors, or if the user has no saved graphs, display an error message.	 <pre>Graph IDs found for validuser + 06aa11@bourne-grammar.lincs.sch.uk: [] No graphs found in USER_GRAPHs table. No graphs found for this user.</pre>	
Test 23	The user should have the option to return to the main screen from the graph downloads page	 <pre>Welcome Page Opened! Users may log in or create a new account</pre>	
Test 24	If the user chooses to move to the creature selection screen, a list of choices of species of creatures should be given to the user.	 <pre>Creature Selection Page opened!</pre>	

Test Number	Description	Evidence	RAG Rating
Test 25	The user should be able to select which species they would like to include via check buttons	 Creature Selection Please select the types of creatures you would like: ☒ Simple Predators These are simple predators which search for prey by moving randomly until they see prey ☒ Simple Prey These are simple prey which search for food by moving randomly until they see prey ☒ Wolves These ferocious predators work together by sharing the location of nearby prey with others ☒ Deer These nimble beasts work together by sharing the location of nearby food with others Move to Settings	
Test 26	The next screen should display spin boxes for the speed of the simulation, the number of food and the number of animals, specifically only for the selected species.	 Settings Change Settings here. Simulation Speed Number of Food Number of Predators Number of Prey Number of Wolves Number of Deer Confirm Settings Settings Page opened!	

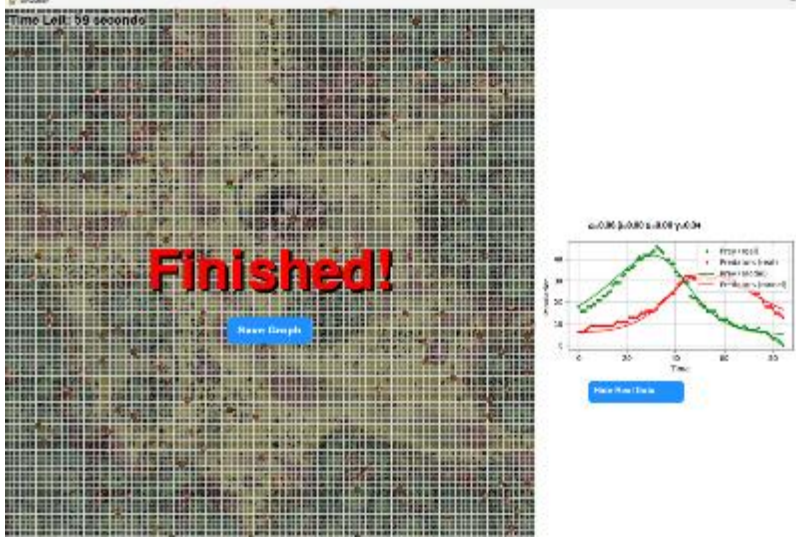
Test Number	Description	Evidence	RAG Rating
Test 27	If the numbers input by the user are valid, the user should be taken to the simulation screen.	 Simulation Time Left: 89 seconds Details were valid! Opened simulation!	
Test 28	The simulation should spawn the number of food and the number and types of creatures specified.	Added 18 simple prey! Added 6 simple predators! Added 0 deer! Added 0 wolves! Added 100 food!	

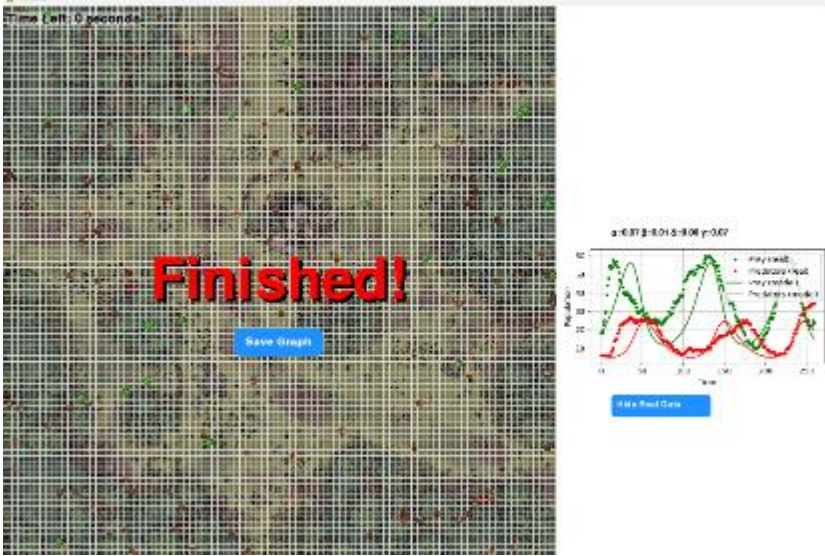
Test Number	Description	Evidence	RAG Rating
Test 29	The creatures should be able to move properly, according to their specified move logic.	<pre> Prey moving from (np.int64(670), np.int64(10)) to (np.int64(670), np.int64(40)) Prey moving from (np.int64(790), np.int64(230)) to (np.int64(800), np.int64(220)) Prey moving from (np.int64(740), np.int64(190)) to (np.int64(740), np.int64(160)) Prey moving from (np.int64(840), np.int64(250)) to (np.int64(850), np.int64(250)) Prey moving from (np.int64(320), np.int64(720)) to (np.int64(350), np.int64(720)) Prey moving from (np.int64(510), np.int64(960)) to (np.int64(530), np.int64(980)) Prey moving from (np.int64(150), np.int64(930)) to (np.int64(180), np.int64(960)) Prey moving from (np.int64(760), np.int64(180)) to (np.int64(790), np.int64(210)) Prey moving from (np.int64(200), np.int64(810)) to (np.int64(220), np.int64(810)) Prey moving from (np.int64(840), np.int64(350)) to (np.int64(840), np.int64(360)) Prey moving from (np.int64(240), np.int64(790)) to (np.int64(230), np.int64(780)) Prey moving from (np.int64(770), np.int64(900)) to (np.int64(770), np.int64(910)) Prey moving from (np.int64(10), np.int64(230)) to (np.int64(0), np.int64(240)) Predator moving from (np.int64(800), np.int64(80)) to (np.int64(790), np.int64(70)) Predator moving from (np.int64(150), np.int64(490)) to (np.int64(150), np.int64(510)) Predator moving from (np.int64(670), np.int64(140)) to (np.int64(680), np.int64(130)) Predator moving from (np.int64(390), np.int64(870)) to (np.int64(360), np.int64(870)) Predator moving from (np.int64(180), np.int64(350)) to (np.int64(190), np.int64(350)) Predator moving from (np.int64(400), np.int64(290)) to (np.int64(370), np.int64(320)) Predator moving from (np.int64(40), np.int64(200)) to (np.int64(60), np.int64(200)) Predator moving from (np.int64(860), np.int64(950)) to (np.int64(890), np.int64(950)) </pre>	
Test 30	The prey should be able to eat when they land on food, removing the food from the grid.	<pre> Prey at [40 760] ate food at [40. 760.] and gained 20 energy </pre>	
Test 31	The predators should be able to eat when they land on prey, removing the prey from the grid.	<pre> Predator at [210 550] ate prey at [210. 540.] and gained 103.84 energy </pre>	

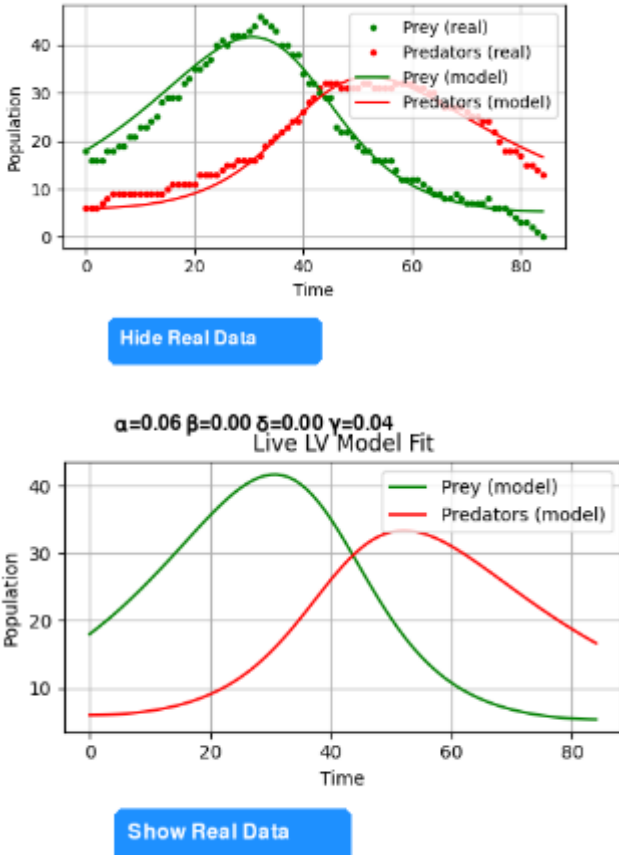
Test Number	Description	Evidence	RAG Rating
Test 32	The predators/prey should reproduce when they reach the reproduction threshold.	Prey at [40 760] reproduced, energy after reproduction: 116 Predator at [990 850] reproduced, energy after reproduction: 176.32	
Test 33	The predators/prey should die when their energy reaches 0 or below.	Prey at [480 280] died Predator at [710 870] died	
Test 34	The simulation should spawn food every 10 seconds	10 Food spawned after 10 seconds!	
Test 35	A graph should be produced side by side with the simulation.	$\alpha=0.06$ $\beta=0.00$ $\delta=0.00$ $\gamma=-0.00$ 	

Test Number	Description	Evidence	RAG Rating
Test 36	This graph should record current population values of the predator and prey classes over time.		
Test 37	This graph should display a curve fitted to the parameters found in the simulation.	$\alpha=0.06$ $\beta=0.00$ $\delta=0.00$ $\gamma=0.04$ Live LV Model Fit 	

Test Number	Description	Evidence	RAG Rating
Test 38	Using values from the simulation, the program should calculate the Lotka-Volterra differential equations and display the relevant parameters on the screen.	$\alpha=0.06 \quad \beta=0.00 \quad \delta=0.00 \quad \gamma=-0.00$	
Test 39	The simulation should be able to be paused using the spacebar. This should stop animals from moving, the graph from being generated, and the timer should stop.	 Time Left: 78 seconds Paused Spacebar pressed! Paused! alpha=0.06 beta=0.00 delta=0.00 gamma=-0.00 Prey (red) Predators (red) Paused Prey (red) Predators (red) Time Prey Predators	

Test Number	Description	Evidence	RAG Rating
Test 40	The simulation should stop if the number of food or the number of prey becomes 0 or if the time runs out.	 Simulation finished as prey reached 0	

Test Number	Description	Evidence	RAG Rating
		 Simulation finished as time reached 0	
Test 41	The simulation should then be stopped, and a visual should be shown to indicate this.		

Test Number	Description	Evidence	RAG Rating
Test 42	The user should be given the option to hide real values from the simulation, to just display the curve-fitted graph.	$\alpha=0.06 \beta=0.00 \delta=0.00 \gamma=0.04$  The evidence consists of two vertically stacked line graphs. Both graphs plot 'Population' on the y-axis (ranging from 0 to 40) against 'Time' on the x-axis (ranging from 0 to 80). The top graph displays four data series: 'Prey (real)' (green dots), 'Predators (real)' (red dots), 'Prey (model)' (green line), and 'Predators (model)' (red line). The bottom graph displays only the model curves: 'Prey (model)' (green line) and 'Predators (model)' (red line). Both graphs show a peak in the prey population around time 30, followed by a peak in the predator population around time 45. A blue button labeled 'Hide Real Data' is positioned below the top graph, and a blue button labeled 'Show Real Data' is positioned below the bottom graph. The parameter values $\alpha=0.06 \beta=0.00 \delta=0.00 \gamma=0.04$ are displayed above each graph.	

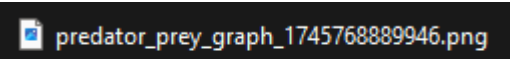
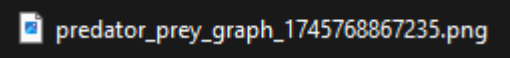
Test Number	Description	Evidence	RAG Rating
Test 43	The user should be able to save the graph, both downloading it to their downloads folder, in its current state of real values/no real values and saving it to their profile in the database.	  	

Table 11: Test table with evidence and RAG Rating

Chapter 5. Evaluation

The evaluation of this project focused on reviewing the objectives in light of the results achieved and going back to the user to capture their feedback on the program and to close the loop. In the following sections, the objectives that have been met are indicated, along with presenting the detailed verbatim feedback and its summary. Some easy-to-execute suggestions have been implemented (see section 5.1.1) which is followed by a brief commentary on further improvements that can be incorporated in future.

5.1. Key Takeaways

The following takeaways were noteworthy:-

- Users found the system easy to understand through the visualisation of the concept and by the embedded explanation page
- Users generally found the system easy to use and intuitive, but would have liked more explicit and easy-to-use UI controls. The email-onboarding system was sometimes difficult to understand, but users found that once an account was made, the system was highly intuitive.
- All users felt that the solution developed had met the purpose of the program

The end users' suggestions were very helpful, and there were a few that I implemented immediately that have been outlined below,

5.1.1. Quick Wins

- Increased the cell size to 20, which makes the simulation much easier to view
- Emails were set to be encrypted, just like passwords

5.1.2. Objectives Accomplished

The primary objectives set out in the analysis section have been met

- A. Create a secure login system for students to ensure their details are protected using hashing to encrypt passwords
- B. Create local normalised databases to store, insert and delete user details using SQL, allowing users to download previously saved graphs
- C. Create a simulation based on biological concepts using user-defined algorithms to simulate the behaviour of animals
- D. Create a user interface to help users interact with the program
- E. Create a feature to track the data of organisms and create graphs as the simulations continue

- F. Allow users to compare searching methods and determine their effectiveness in a competitive natural environment, and demonstrate the predator-prey relationship.

5.1.3. Detailed User Feedback

Feedback Questionnaire – End User

1. Understanding of the program/solution developed

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 9 (10)

Reasons for the rating (Please add comments below)

Problem well explained relevant equations applied & great use of free parameters to add fitting the data

2. Ease of Use

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 9 (10)

Reasons for the rating (Please add comments below)

Very intuitive software ; typical log in screen, all concepts explained in software. Great ability to save & retrieve data

3. Effective and meets the purpose

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 9 (10)

Reasons for the rating (Please add comments below)

4. Suggestions for further improvements

Maybe encrypt encrypt email addresses
Implement a zoom for the live 100x100 grid for real time change

Signature: 

Parameters	Respondent	Teacher with experience building teaching software
	Understanding	10
	Ease of Use	10
	Effectiveness	10
	Improvements suggested	Add encryption for the email addresses Implement a zoom feature, to be able to hone in on a particular area

Feedback Questionnaire – End User

1. Understanding of the program/solution developed

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 9 10

Reasons for the rating (Please add comments below)

contained description of the project's purpose at the start and is better understood by allowing the user to select their own parameter values.

2. Ease of Use

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 9 10

Reasons for the rating (Please add comments below)

Email onboarding system could be time consuming / confusing but since only occurs once, is valued. Simulation is easy to interpret.

3. Effective and meets the purpose

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 9 10

Reasons for the rating (Please add comments below)

Displays a live graph of predator, prey population sizes which effectively reinforces the simulation's purpose.

4. Suggestions for further improvements

could include a back button navigation system so user does not need to re-run code. A reset password capability would be desirable.

Signature: Sama

Parameters	Respondent	A Level Computer Science Student
	Understanding	10
	Ease of Use	8
	Effectiveness	9
	Improvements Suggested	Include a back button navigation system. Add a feature to reset the user's password.

Feedback Questionnaire – End User

1. Understanding of the program/solution developed

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 9 10

Reasons for the rating (Please add comments below)

Clearly explained good range of visual
explanations graph + simulation screen

2. Ease of Use

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 9 10

Reasons for the rating (Please add comments below)

Yes

3. Effective and meets the purpose

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 9 10

Reasons for the rating (Please add comments below)

It gives the possibility to expand on
ideas such as low amounts of prey
food meaning prey group around food
and are easier to catch by the predators

4. Suggestions for further improvements

Different Y axis scales for prey
and predator populations.
As prey should always be more abundant
than predators

Signature: _____

C. Pare

Parameters	Respondent	Biology Teacher
	Understanding	9
	Ease of Use	9
	Effectiveness	9
	Improvements Suggested	Different Y-axis scales for predators and prey Measures to ensure prey are always more abundant than predators

Feedback Questionnaire – End User

1. Understanding of the program/solution developed

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 9 **10**

Reasons for the rating (Please add comments below)

perfectly demonstrates the relationship between predator, prey, food & habitat in a way that is both useful for A-level students but also easy to understand.

2. Ease of Use

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 **6** 7 8 9 10

Reasons for the rating (Please add comments below)

- UI could be clearer for main menu page
- for selecting variables make clear what ranges are there

3. Effective and meets the purpose

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 **9** 10

Reasons for the rating (Please add comments below)

- Code helps with the education & understanding of predator-prey relationship but UI could be made easier to use in certain areas

4. Suggestions for further improvements

- have a key for each of the characters in simulation to make it easier to identify
- potentially add a rewind for paused so teacher can go to important events

Signature: _____

Parameters	Respondent	A Level Biology Student
	Understanding	10
	Ease of Use	6
	Effectiveness	9
	Improvements suggested	Add a key to identify creature types on the side of the simulation. Add a rewind feature to review important events

Feedback Questionnaire – End User

1. Understanding of the program/solution developed

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 **8** 9 10

Reasons for the rating (Please add comments below)

The actual simulation absolutely.

2. Ease of Use

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 **8** 9 10

Reasons for the rating (Please add comments below)

Once logged in very self explanatory.

3. Effective and meets the purpose

Rating (Indicate by circling a number of your choice)

1 2 3 4 5 6 7 8 **9** 10

Reasons for the rating (Please add comments below)

Great alternative teaching resource.

4. Suggestions for further improvements

The ecosystem visual.

Signature: Rk

Parameters	Respondent	Biology Teacher
	Understanding	8
	Ease of Use	8
	Effectiveness	9
	Improvements Suggested	Make the ecosystem cell size bigger.

5.1.4. Limitations

One of the limitations is the speed of the as the simulation involves the creation of many classes at runtime, and this potentially slows the program. The RAM usage of user devices spikes, and older systems are not able to run the simulation smoothly. While most devices were capable, the project requires a reasonable amount of memory to run. The users' systems also required setup to be able to run the software developed.

However, these limitations can be overcome by ensuring that other applications are not open whilst the simulation is running, and following the user instructions guide (See Section 5.1.5). Some of these basic steps will ensure that the users can execute the program uninterrupted.

5.1.5. Further Improvements for the future

A user guide document could be produced introducing users to the system and the required settings, which would allow for greater user understanding of the program and how to navigate it. This would help users to understand the logic behind the program better, and could include some recommended settings which give particularly good results for demonstrating the Lotka-Volterra curves.

More complex creatures could be created that better model real-life animals. These creatures could have a "flee" command, which would be more representative of prey in real life. Creatures could also be modelled by other types of algorithms explored in the design section, such as "Levy flights" or "Spiral Search". A new mechanic, such as thirst and the corresponding water object, could be introduced to also improve the simulation.

Another simulation could be developed which focuses on a different area of Biology. As this solution was deemed successful by the end users, it would be appropriate to develop another system to model a different biological concept. Disease would be a good example of this, as it would involve similar organism interaction, and was also highlighted as a topic which students wished to see a simulation of. The solution has been designed such that it could be implemented as a part of a wider set of such biological simulations.

Conclusion

The project was set out to create a tool (customisable simulation/model) for biology teachers to help them teach the dynamics of the predator-prey interaction by using a visual teaching aid to support learning.

This project has been able to meet its purpose, and a working solution has been developed to aid teachers. The end users (teachers) who evaluated the program are keen to use the simulation in their lessons going forward. With its use in the lessons, students will be able to understand predator-prey dynamics better, as they will be able to view a simulation and understand the interactions that lead to the theoretical results that are described by their teachers. In addition, teachers can produce physical copies of graphs, which students are likely to find very helpful.

The development of the program has been a great learning experience, and I am excited about the prospects of school teachers using the same. At an individual level, I have been able to build my skills in programming, report writing and project management.

----/---

ATIKSH AGNIHOTRI
BOURNE GRAMMAR SCHOOL

CANDIDATE NUMBER: 6002

CENTRE NUMBER: 26202

QUALIFICATION CODE: 7517/C

Bibliography

https://teaching.smp.uq.edu.au/scims/Appl_analysis/Lotka_Volterra.html#:~:text=Lotka%20Volterra%20variation&text=a%20is%20the%20growth%20rate,prey%20independent%20of%20the%20predators. Section 1.11

<https://www.youtube.com/@MikeSaintAntoine/videos> Section 1.15

<https://www.geeksforgeeks.org/how-to-center-a-window-on-the-screen-in-tkinter/>
Technical solution (tkManager.py)

<https://quantumnumbers.anu.edu.au/documentation> Technical solution
(emailManager.py)

<https://stackoverflow.com/questions/35851281/python-finding-the-users-downloads-folder> Technical solution (databaseManager.py)

<https://labs.minutelabs.io/evolution-simulator/> Section 1.10

<https://thelifeengine.net/> Section 1.10

<https://www.echalk.co.uk/> Section 1.10

<https://www.roboleary.net/architecture/2017/08/29/class-diagrams/> Section 2.10

<https://stackoverflow.com/questions/65200716/after-updating-my-classes-and-the-way-i-create-my-sprites-the-function-that-has?rq=3> Technical Solution
(simulationManager.py)

<https://stackoverflow.com/questions/41283541/using-bcrypt-password-hashing-for-user-authentication> Technical Solution (dbManager.py)

<https://stackoverflow.com/questions/65915695/how-do-i-make-sql-python-find-if-a-username-is-already-in-the-database> Technical Solution (dbManager.py)

<https://stackoverflow.com/questions/34275782/how-to-get-desktop-location> Technical Solution (dbManager.py)

Non-specific sources:

Wikipedia – Learning about Lotka-Volterra and other mathematical models

Isaac Computer Science – Learning about databases and normalisation

<https://www.aqa.org.uk/subjects/computer-science/a-level/computer-science-7517/specification/subject-content/non-exam-assessment-the-computing-practical-project> - AQA official NEA guidance website

<https://youtu.be/EgoXoMPVQc?si=PBTxKmuUwanv4Dqb> – Learning about hashing

Tackling A Level Projects in Computer Science (PG Online)

Library Documentation:

<https://www.pygame.org/docs/>

<https://numpy.org/doc/>

<https://tkdocs.com/>

<https://matplotlib.org/stable/index.html>

<https://docs.scipy.org/doc/scipy/tutorial/index.html#user-guide>

https://www.tutorialspoint.com/sqlite/sqlite_python.htm

<https://www.geeksforgeeks.org/hashing-passwords-in-python-with-bcrypt/>

<https://realpython.com/python-send-email/>

<https://pypi.org/project/email-validator/>

https://www.tutorialspoint.com/python/python_sending_email.htm

<https://realpython.com/python-requests/>

<https://docs.python.org/3/library/datetime.html>

<https://www.geeksforgeeks.org/python-time-module/>

<https://docs.python.org/3/library/random.html>

<https://docs.python.org/3/library/io.html>

https://www.w3schools.com/python/module_os.asp

<https://www.geeksforgeeks.org/platform-module-in-python/>

<https://realpython.com/python-zipfile/>